

Aula 11

*DataPrev (Perfil 3: Desenvolvimento de Software) Engenharia de Software - 2026
(Pós-Edital)*

Autor:

**Diego Carvalho, Equipe
Informática 2 (Diego Carvalho),
Equipe Informática 1 (Diego
Carvalho)**

21 de Julho de 2026

Índice

1) Padrões de Projeto - Teoria	3
2) Padrões de Projeto - Resumo	61
3) Padrões de Projeto - Questões Comentadas - FGV	66
4) Padrões de Projeto - Lista de Questões - FGV	90

APRESENTAÇÃO

Fala, pessoal! Tudo bem?

A disciplina de Padrões de Projeto é frequentemente considerada maçante e bastante "decoreba", exigindo muita memorização dos alunos. Para quem tem bastante tempo, como em um estudo pré-edital, a orientação é realizar uma leitura completa do material. O aluno deve focar inicialmente nos Conceitos Básicos, que apresentam incidência altíssima nas provas, e avançar para a compreensão detalhada de todos os padrões. Embora o conteúdo seja denso, ter tempo permite dominar os tópicos de incidência média e alta, garantindo pontos valiosos.

Para o aluno em reta final pós-edital com o tempo esgotado, a estratégia foca puramente no melhor custo-benefício. Como a aula é muito teórica, memorize as frases e mnemônicos iniciais, pois garantem o acerto de várias questões rapidamente. Além disso, descarte quase todos os Padrões Comportamentais, que é a mais extensa e toma muito tempo (foque-se naqueles com incidência maior). Aliás, analise com cuidado as incidências de cada padrão em prova.

Prof. Diego Carvalho

📷 **INSTAGRAM DO PROFESSOR- [WWW.INSTAGRAM.COM/PROFESSORDIEGOCARVALHO](https://www.instagram.com/professordiegocarvalho)**

AVENTURAS EM DESIGN PATTERNS: O CASO DO SINGLETON



PADRÕES DE PROJETO

Conceitos Básicos

INCIDÊNCIA EM PROVA: ALTÍSSIMA

PADRÕES DE PROJETO GOF

Os Padrões de Projeto GoF (Gang of Four) são um conjunto de 23 soluções reutilizáveis para problemas recorrentes no desenvolvimento de software orientado a objetos, apresentados no livro *Design Patterns: Elements of Reusable Object-Oriented Software* (1994), de Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides. Esses padrões não representam implementações prontas, mas sim modelos de solução que orientam a organização das classes, objetos e suas interações, promovendo maior reutilização, flexibilidade, baixo acoplamento e facilidade de manutenção dos sistemas.

Antes de estudar os principais padrões de projeto, é importante compreender o conceito de padrão. **Um padrão corresponde a uma solução recorrente para um problema que também ocorre de forma recorrente em determinado contexto.** O que torna essa ideia tão útil? Em vez de criar uma solução completamente nova sempre que o mesmo problema surgir, torna-se possível aproveitar um modelo já documentado, analisado e validado em diferentes situações. Christopher Alexander sintetiza essa proposta ao afirmar que cada padrão descreve um problema recorrente e o núcleo de uma solução que pode ser reutilizada inúmeras vezes, sem que sua implementação precise ser exatamente igual em todos os casos.

Com base nessa ideia, surgiram os padrões de projeto, conhecidos como Design Patterns. Segundo a definição apresentada por Gamma, Helm, Johnson e Vlissides, autores conhecidos como Gang of Four (GoF), **os padrões de projeto consistem em descrições de objetos que se comunicam e de classes personalizadas para resolver um problema genérico de projeto em um contexto específico.** Isso significa que um padrão corresponde a um trecho de código pronto para reutilização? Não. **Um padrão descreve uma solução abstrata que pode ser adaptada às necessidades de cada sistema** e de cada domínio de aplicação.

A utilidade dessa abordagem torna-se evidente quando se observa o desenvolvimento de sistemas para diferentes áreas de negócio. Embora as aplicações possuam finalidades distintas, muitos problemas de projeto se repetem ao longo do tempo. Em vez de desenvolver uma solução inteiramente nova para cada situação, os desenvolvedores podem recorrer a modelos já conhecidos e adaptá-los ao contexto específico do sistema. **Essa reutilização reduz o esforço de projeto, favorece a adoção de soluções consolidadas e facilita a comunicação entre profissionais** que compartilham uma mesma terminologia.

A origem desse conceito não está na Engenharia de Software. Como essa ideia chegou ao desenvolvimento de programas? A inspiração veio da Arquitetura. **Na década de 1970, Christopher Alexander dedicou-se ao estudo de problemas recorrentes no planejamento de cidades**, edifícios e outros espaços construídos. **Em 1977, juntamente com Sara Ishikawa e Murray Silverstein, publicou a obra A Pattern Language**, na qual apresentou um conjunto de padrões destinados a orientar a elaboração de projetos em diferentes escalas. Cada padrão descrevia um problema recorrente e uma solução que poderia ser adaptada às características específicas de cada projeto.

A proposta chamou a atenção da comunidade de Engenharia de Software, que enfrentava desafios semelhantes no desenvolvimento de sistemas orientados a objetos. **Em 1994, Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides publicaram o livro Design Patterns: Elements of Reusable Object-Oriented Software.** Qual foi a principal contribuição dessa obra? Em vez de apresentar bibliotecas ou componentes prontos, **o livro documentou vinte e três padrões de projeto que**

descrevem soluções recorrentes para problemas comuns de projeto em software orientado a objetos. Essa publicação tornou-se uma das principais referências sobre o tema.

Os padrões de projeto oferecem diversas vantagens durante o desenvolvimento de software. Entre elas estão a reutilização de soluções já consolidadas, a adoção de uma terminologia comum entre os desenvolvedores, a melhoria da documentação dos projetos e o aproveitamento da experiência acumulada pela comunidade de Engenharia de Software. Além disso, **muitos padrões favorecem a construção de sistemas com maior coesão, menor acoplamento e melhor organização estrutural**, contribuindo para facilitar a manutenção e a evolução do código.

A utilização de padrões de projeto, entretanto, não elimina a necessidade de análise crítica. Seria adequado aplicar um padrão sempre que ele estiver disponível? Nem sempre. **A adoção inadequada de um padrão pode aumentar a complexidade da solução sem produzir benefícios proporcionais.** Em algumas situações, recursos oferecidos pelas linguagens modernas de programação, como funções de primeira classe, expressões lambda, delegação, mecanismos de composição e outros recursos da linguagem, tornam desnecessária a aplicação de determinados padrões. Por esse motivo, alguns autores defendem que certos Design Patterns surgiram para suprir limitações existentes em linguagens mais antigas, embora essa interpretação não seja unânime na literatura.

Cada padrão de projeto é descrito segundo uma estrutura padronizada composta por quatro elementos fundamentais. O que cada um deles representa? O nome identifica o padrão e estabelece uma terminologia comum entre os desenvolvedores. O problema descreve a situação em que sua utilização é recomendada. A solução apresenta a estrutura geral capaz de resolver o problema, sem impor uma implementação específica. Por fim, **as consequências descrevem os impactos decorrentes da adoção do padrão**, incluindo suas vantagens, limitações e os efeitos sobre a arquitetura do sistema.

(FUNDATEC / IFC - 2023) O padrão de projeto de software ou Design Patterns é uma descrição do problema e da essência de sua solução, de modo que a solução possa ser reutilizada em diferentes contextos. Ao escrever um padrão, os quatro elementos essenciais foram definidos pela "Gangue dos Quatro", em seu livro de padrões. Assinale a alternativa que contém elemento que NÃO pertence a essa descrição.

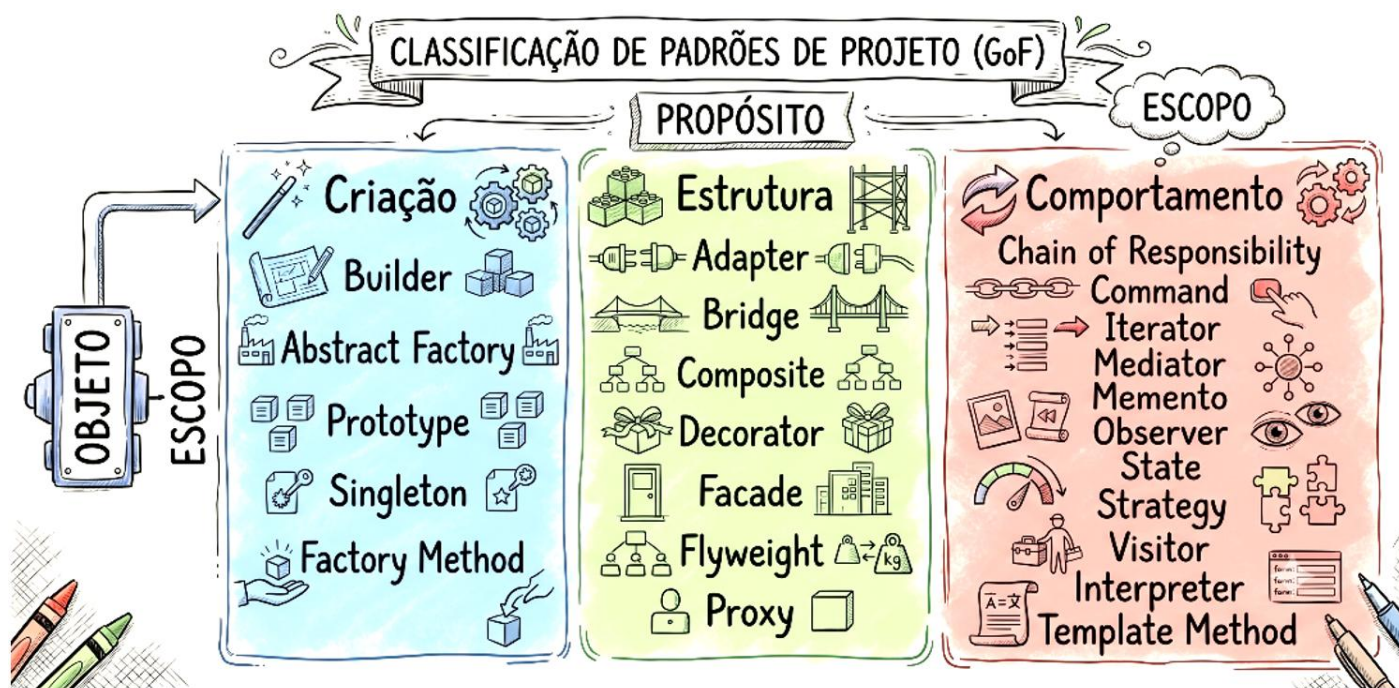
- a) Nome do padrão.
- b) Descrição do problema.
- c) Declaração de variáveis.
- d) Descrição da solução.
- e) Declaração das consequências.

Comentários: os quatro elementos essenciais de um padrão GoF são: nome, problema, solução e consequências. A declaração de variáveis é um detalhe de implementação e não integra a descrição formal de um padrão de projeto (Letra C).

Antes de estudar cada padrão individualmente, é importante compreender o alcance do catálogo elaborado pelo *Gang of Four*. Os vinte e três padrões podem ser aplicados em qualquer tipo de projeto de software? Não. **Esse catálogo foi desenvolvido especificamente para sistemas orientados a objetos** e utiliza conceitos característicos desse paradigma, como classes, objetos, herança, composição e polimorfismo. Isso não significa, entretanto, que os padrões de projeto existam apenas na orientação a objetos, pois há diversos outros padrões aplicáveis a diferentes paradigmas e estilos de desenvolvimento.

Os padrões do *Gang of Four* são tradicionalmente classificados em três grandes categorias, de acordo com a finalidade que desempenham durante o desenvolvimento de software. Como essa classificação é organizada? **Os padrões dividem-se em criacionais, estruturais e comportamentais.** Essa organização

facilita o estudo do catálogo, pois reúne padrões que procuram resolver problemas semelhantes dentro de uma mesma categoria.



Os padrões criacionais concentram-se na forma como os objetos são criados. Em vez de permitir que cada parte do sistema realize diretamente a instanciação das classes, esses padrões procuram encapsular e flexibilizar o processo de criação, reduzindo o acoplamento entre os componentes e tornando o código mais adaptável a mudanças. Dessa forma, o mecanismo de criação dos objetos pode evoluir sem exigir alterações significativas nas partes que os utilizam.

Os padrões estruturais voltam-se para a organização das classes e dos objetos. O que caracteriza esse grupo de padrões? Seu objetivo é estabelecer formas eficientes de compor estruturas maiores a partir de componentes menores, favorecendo a reutilização, a flexibilidade e a redução do acoplamento entre os elementos do sistema. Assim, torna-se possível construir arquiteturas mais organizadas e mais fáceis de manter ao longo do ciclo de vida do software.

Os padrões comportamentais concentram-se na interação entre os objetos e na distribuição de responsabilidades durante a execução do sistema. Eles descrevem mecanismos que organizam a comunicação entre os componentes, definem como determinadas tarefas devem ser distribuídas e estabelecem formas padronizadas de colaboração entre objetos. Por esse motivo, compreender a classificação dos vinte e três padrões é um passo importante para identificar qual categoria oferece a solução mais adequada para cada problema de projeto.

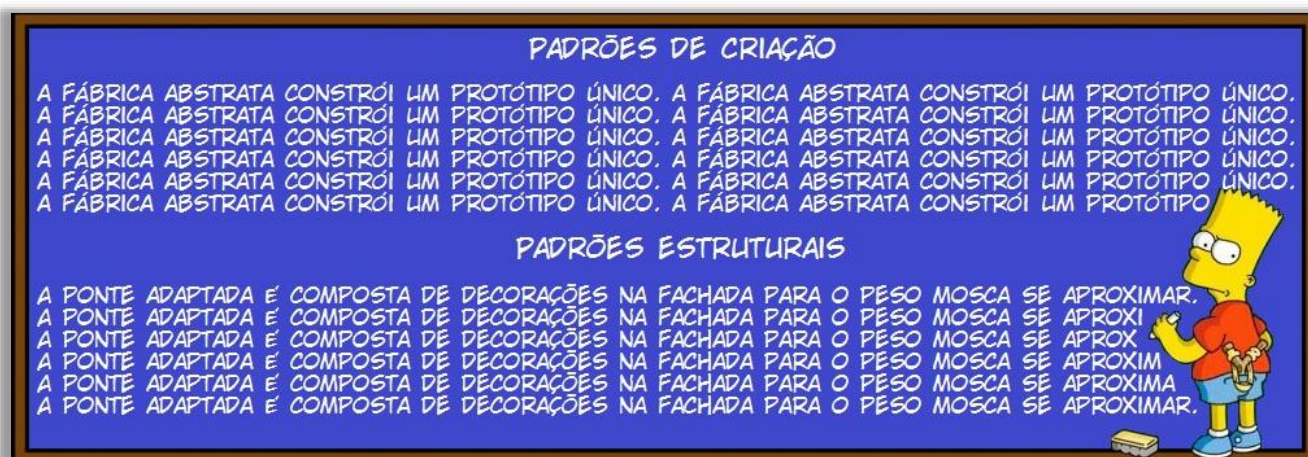
(FGV / Prefeitura de BH - 2024) Padrões de projeto são soluções consagradas que se baseiam nas estruturas da orientação a objetos para solucionar problemas comuns em projetos de software. Os padrões são agrupados em tipos. Assinale a opção que indica apenas padrões do tipo comportamental.

- a) Decorator, bridge e adapter.
- b) Singleton, prototype e builder.
- c) Iterator, memento e observer.
- d) Command, mediator e facade.

Comentários: Decorator, Bridge e Adapter são estruturais; Singleton, Prototype e Builder são criacionais; em Command, Mediator e Facade há mistura, pois Facade é estrutural. Somente Iterator, Memento e Observer são todos comportamentais, pois tratam da interação e da comunicação entre objetos (Letra C).



Quando eu estudei esse assunto na minha vida de concurseiro, utilizei de um mnemônico maneiríssimo criado pelo Prof. Rogério Araújo para memorizar:



Explicação: A fábrica (Factory Method) abstrata (Abstract Factory) constrói (Builder) um protótipo (Prototype) único (Singleton). A ponte (Bridge) adaptada (Adapter) é composta (Composite) de decorações (Decorator) na fachada (Façade) para o peso-mosca (Flyweight) se aproximar (Proxy). E não tem frase para o último? Não, porque não é necessária! **Se não é um padrão criacional ou estrutural, é um padrão comportamental.**

Padrões Criacionais

PADRÕES CRIACIONAIS

Os Padrões Criacionais são uma categoria dos padrões GoF voltada para abstrair e controlar o processo de criação de objetos. Seu principal objetivo é desacoplar o código cliente da forma como os objetos são instanciados, tornando o sistema mais flexível e facilitando a substituição ou extensão das classes concretas. Fazem parte dessa categoria os padrões Abstract Factory, Builder, Factory Method, Prototype e Singleton.

Abstract Factory

INCIDÊNCIA EM PROVA: MÉDIA

Descrição Resumida: esse padrão fornece uma interface para criar famílias de objetos relacionados ou dependentes sem especificar suas classes concretas.

O padrão de projeto Abstract Factory **fornece uma interface para criar famílias de objetos relacionados ou dependentes sem que o código cliente precise conhecer suas classes concretas**. Essa abordagem desacopla o processo de criação dos objetos de sua utilização, pois o cliente trabalha apenas com interfaces. Além disso, o padrão garante que todos os objetos produzidos pertençam à mesma família e sejam compatíveis entre si.

Uma dúvida bastante comum surge nesse ponto: o que significa uma família de objetos? **Uma família corresponde a um conjunto de componentes projetados para funcionar em conjunto**. Imagine uma aplicação que possa ser executada tanto em computadores quanto em smartphones. Cada ambiente possui botões, menus e barras de navegação próprios, mas todos esses componentes pertencem à mesma interface gráfica. Em vez de criar cada elemento individualmente, uma fábrica concreta produz toda a família correspondente ao ambiente selecionado.

Outro aspecto importante envolve a escolha da fábrica concreta. Como isso acontece? Normalmente, o código cliente identifica algum parâmetro de configuração, uma preferência do usuário ou o ambiente de execução e, a partir dessa informação, instancia a fábrica adequada. Depois dessa escolha, todas as solicitações de criação de objetos passam pela interface da fábrica abstrata, enquanto as implementações concretas permanecem ocultas do restante da aplicação.

Vale também comparar esse padrão com outro bastante conhecido. Seriam o Factory Method e o Abstract Factory a mesma solução? Apesar de ambos encapsularem a criação de objetos, eles possuem objetivos diferentes. **O Factory Method normalmente cria um único tipo de produto e delega essa decisão às subclasses; o Abstract Factory, por sua vez, cria famílias completas de objetos relacionados, garantindo que todos sejam compatíveis e possam ser utilizados em conjunto.**

```
class FabricaWindows:
    def criar_botao(self):
        return "Botão do Windows"

    def criar_menu(self):
        return "Menu do Windows"

class FabricaMac:
    def criar_botao(self):
        return "Botão do Mac"

    def criar_menu(self):
```

```
return "Menu do Mac"

def criar_interface(fabrica):
    print(fabrica.criar_botao())
    print(fabrica.criar_menu())

criar_interface(FabricaWindows())
```

Nesse exemplo, FabricaWindows e FabricaMac representam fábricas diferentes. Cada uma cria uma família de objetos relacionados: um botão e um menu do mesmo sistema operacional. A função criar_interface() não precisa saber se está trabalhando com Windows ou Mac. Ela apenas recebe uma fábrica e solicita os objetos por meio dos métodos criar_botao() e criar_menu(). Para trocar toda a família de componentes, basta alterar a fábrica recebida: criar_interface(FabricaMac()).

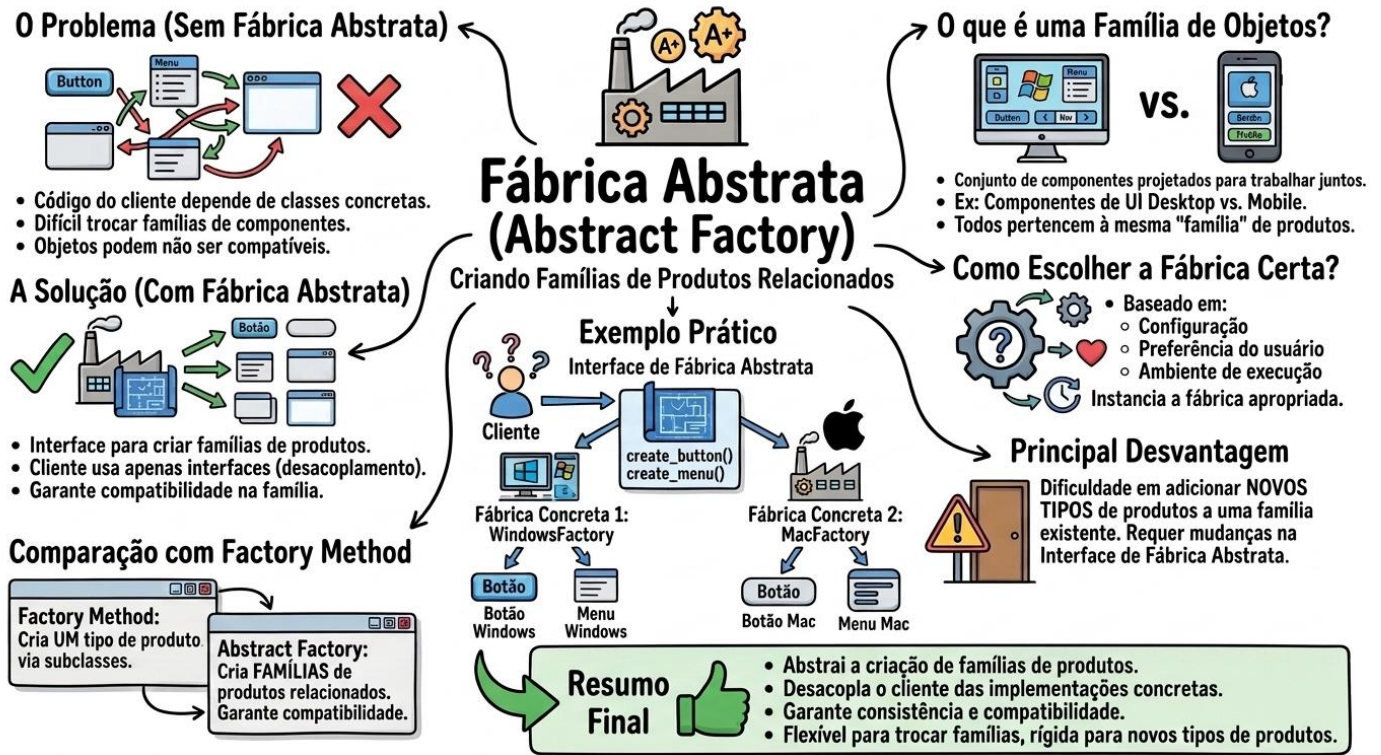
A ideia central do Abstract Factory é justamente essa: fornecer uma forma de criar conjuntos de objetos relacionados sem fazer o código cliente depender diretamente das classes concretas desses objetos. **É importante destacar, entretanto, a principal desvantagem do Abstract Factory:** estender o sistema para adicionar novas "categorias" (novos tipos) de produtos em uma família já existente é uma tarefa trabalhosa.

A interface central (a fábrica abstrata) condensa todos os métodos de criação (como criar_botao() e criar_menu()). Se o sistema precisar passar a produzir um novo componente genérico (como criar_janela()), essa mudança **exigirá a alteração forçada na interface da Fábrica Abstrata e, consequentemente, a reescrita de todas as subclasses de fábricas concretas** vinculadas a ela, o que fere o Princípio Aberto/Fechado (OCP).

(FCC / Analista do Ministério Público de Sergipe - 2026) Um Ministério Público está desenvolvendo um módulo ASP.NET Core (C#) para integrar diferentes sistemas externos de apoio. Cada sistema externo exige um conjunto consistente de objetos relacionados (por exemplo: ProtocoloClient, ConsultaService, ValidadorToken) que precisam ser compatíveis entre si conforme a origem dos dados. A equipe precisa alternar dinamicamente o sistema integrado em tempo de execução, sem acoplamento direto às classes concretas. A decisão de projeto que melhor atende esse cenário é

- a) empregar o padrão Prototype para clonar objetos base de um sistema externo e derivar variações configuráveis.
- b) definir um Abstract Factory para produzir famílias de objetos relacionados de cada sistema externo, mantendo o uso apenas de interfaces e escolhendo a fábrica concreta em tempo de execução.
- c) criar um único Factory Method no ProtocoloClient para instanciar os objetos necessários de cada sistema.
- d) adotar um Facade sobre cada SDK de sistema externo para simplificar o uso e centralizar a criação.
- e) implementar um Adapter para padronizar a interface de cada SDK, deixando a criação da família de objetos distribuída em módulos diferentes.

Comentários: o cenário exige criar famílias de objetos relacionados e compatíveis entre si, escolhendo a fábrica concreta em tempo de execução e sem acoplamento às classes concretas: essa é exatamente a definição do Abstract Factory. Prototype foca em clonagem; Factory Method cria um produto por hierarquia; Facade simplifica subsistemas; e Adapter apenas compatibiliza interfaces, sem organizar a criação unificada da família (Letra B).



Builder

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Descrição Resumida: esse padrão separa a construção de um objeto complexo da sua representação, de forma que o mesmo processo de construção possa criar diferentes tipos de representações.

O padrão de projeto Builder é utilizado quando um objeto complexo precisa ser construído em etapas e sua montagem deve permanecer separada da representação final. Em quais situações isso se torna útil? Sempre que o mesmo processo de construção puder gerar resultados diferentes, bastando alterar os componentes utilizados em cada etapa. Dessa forma, o algoritmo de construção permanece o mesmo, enquanto a representação final do objeto pode variar conforme a necessidade da aplicação.

O Builder costuma ser comparado ao Abstract Factory, mas essa semelhança pode gerar dúvidas. Afinal, ambos criam objetos, então qual é a diferença? O Abstract Factory produz famílias de objetos relacionados e compatíveis em uma única operação, enquanto **o Builder concentra-se na construção gradual de um único objeto complexo**. Em vez de entregar todos os componentes prontos de uma só vez, ele organiza a criação de cada parte até que o produto final esteja completamente montado.

Considere uma aplicação que possua interfaces gráficas para smartphone e desktop, cada uma formada por componentes como botões, barras de rolagem, caixas de seleção e ícones. Inicialmente, ao identificar o dispositivo utilizado, toda a família correspondente era criada automaticamente. E se um tablet precisasse combinar elementos das duas interfaces? Nesse caso, a construção integral de apenas uma família deixaria de atender aos requisitos da aplicação.

É justamente nesse cenário que o Builder demonstra sua principal vantagem. Seria necessário criar uma interface totalmente nova? Não necessariamente, pois **o padrão permite montar uma representação personalizada ao combinar componentes de diferentes origens**. Assim, podem ser utilizados, por exemplo, um botão da interface de smartphone, uma barra de rolagem da interface de desktop, caixas de seleção da interface de smartphone e ícones da interface de desktop. **Quando a implementação inclui um Director, esse componente coordena as etapas de construção** até que a interface esteja completamente montada.

```
class Casa:
    def __init__(self):
        self.comodos = []

class ConstrutorCasa:
    def __init__(self):
        self.casa = Casa()

    def adicionar_quarto(self):
        self.casa.comodos.append("Quarto")

    def adicionar_banheiro(self):
        self.casa.comodos.append("Banheiro")

    def obter_resultado(self):
        return self.casa

builder = ConstrutorCasa()
builder.adicionar_quarto()
builder.adicionar_banheiro()
```

```

casa = builder.obter_resultado()

print(casa.comodos)

```

Nesse exemplo, a classe Casa representa o objeto que será construído. Já a classe ConstrutorCasa (Builder) é responsável por montar esse objeto passo a passo, adicionando seus cômodos por meio de métodos específicos. A principal ideia do Builder é separar o processo de construção do objeto do próprio objeto. Em vez de criar uma casa completamente pronta em um único passo, o Builder permite construí-la gradualmente: primeiro adiciona-se um quarto, depois um banheiro, depois uma cozinha e assim por diante. Ao final, o método obter_resultado() devolve a casa completa.

Esse padrão é especialmente útil quando um objeto possui muitas partes ou diversas configurações possíveis. **Em vez de criar vários construtores diferentes ou um construtor com inúmeros parâmetros, o Builder permite montar o objeto de forma organizada**, flexível e legível, escolhendo apenas os componentes desejados.



(FCC / TRF 4ª Região - 2025) Uma equipe de desenvolvimento está criando uma aplicação que precisa gerar diferentes tipos de relatórios (PDF, Excel ou HTML). Cada tipo de relatório requer um processo de construção complexo e específico. Nesse cenário, o padrão de projeto criacional da Gang of Four (GoF) mais adequado para encapsular a criação de objetos complexos, permitindo a construção de diferentes representações, é o

- Object Pool, que mantém um conjunto de objetos prontos para uso.
- Builder, que separa a construção de um objeto complexo de sua representação, permitindo que o mesmo processo de construção crie diferentes representações.
- Prototype, que cria novos objetos copiando uma instância existente.

d) Factory Method, que define uma interface para criar um objeto, mas permite às subclasses decidirem qual classe instanciar.

e) Singleton, que garante que uma classe tenha apenas uma instância.

Comentários: O Builder separa o processo de construção da representação final, permitindo que a mesma lógica de montagem gere relatórios em PDF, Excel ou HTML. Object Pool apenas reaproveita instâncias; Prototype cria por cópia; Factory Method escolhe a classe concreta; e Singleton restringe a quantidade de instâncias (Letra B).

Factory Method

INCIDÊNCIA EM PROVA: ALTA

Descrição Resumida: esse padrão define uma interface para criar um objeto, mas deixa as subclasses decidirem qual classe instanciar.

O padrão de projeto Factory Method é utilizado quando uma classe não consegue determinar antecipadamente qual objeto concreto deverá instanciar. Em quais situações isso acontece? Geralmente quando diferentes objetos compartilham uma mesma interface ou classe base, mas a escolha da implementação concreta depende de informações conhecidas apenas durante a execução da aplicação. Assim, a lógica de criação permanece flexível e desacoplada do restante do sistema.

Considere uma concessionária que comercializa diversos modelos de automóveis. Em vez de criar uma classe independente para manipular cada modelo, define-se uma classe abstrata que representa um carro de forma genérica e, a partir dela, são criadas subclasses para modelos específicos, como Corsa, Celta, Cruze e Camaro. Por que adotar essa estrutura? Porque todos os modelos compartilham características comuns, mas cada um possui sua própria implementação.

A principal dificuldade surge quando o sistema precisa criar um objeto. Como decidir qual modelo deve ser instanciado? **O Factory Method resolve esse problema ao delegar essa responsabilidade às subclasses criadoras**, que implementam o método responsável pela criação do produto. Dessa forma, a escolha do objeto concreto pode ocorrer durante a execução da aplicação, conforme as regras de negócio, configurações ou informações disponíveis naquele momento.

A estrutura clássica desse padrão é formada pelos participantes Creator, ConcreteCreator, Product e ConcreteProduct. O que cada um representa? **O Product define a interface comum dos objetos criados; o ConcreteProduct implementa essa interface; o Creator declara o método fábrica**; por sua vez, o ConcreteCreator implementa esse método e determina qual produto concreto será instanciado em cada situação.

```
class Cachorro:
    def emitir_som(self):
        return "Au au!"

class Gato:
    def emitir_som(self):
        return "Miau!"

class FabricaAnimal:
    def criar_animal(self, tipo):
        if tipo == "cachorro":
            return Cachorro()
        elif tipo == "gato":
            return Gato()

fabrica = FabricaAnimal()

animal = fabrica.criar_animal("gato")
print(animal.emitir_som())
```

No exemplo, as classes Cachorro e Gato representam os produtos que podem ser criados. A classe FabricaAnimal implementa o Factory Method, isto é, um método (criar_animal()) responsável por decidir

qual objeto instanciar com base na solicitação recebida. A principal ideia do Factory Method é encapsular a criação de objetos em um método, evitando que o código cliente utilize diretamente comandos como Cachorro() ou Gato(). Em vez disso, o cliente solicita um objeto à fábrica, que decide qual classe concreta deve ser instanciada.

Esse padrão é útil quando o sistema pode precisar criar diferentes tipos de objetos sem que o código cliente conheça suas classes concretas. Assim, caso seja necessário adicionar um novo tipo de animal (como Pássaro), basta modificar ou estender a fábrica, mantendo o restante da aplicação desacoplado do processo de criação dos objetos.

A grande desvantagem de adotar o padrão Factory Method está relacionada com uma potencial proliferação e explosão de novas classes no projeto de software. A arquitetura exige que o cliente delegue a responsabilidade de instanciação; portanto, sempre que a aplicação precisar suportar o processamento de uma nova classe de "Produto" (ConcreteProduct), o desenvolvedor será quase sempre forçado a parametrizar o sistema gerando também uma nova subclasse paralela de "Criador" (ConcreteCreator).

A Jornada do "Eu Quero"

A transformação de um desejo em realidade através de passos, esforço e superação de desafios.

O Ponto de Partida: "Eu Quero"

Um desejo partida?

- Um desejo
- Sem plano definido
- Um destino sem rota



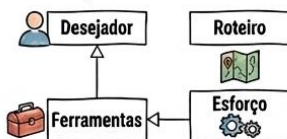
A Solução: O Caminho

Passos definidos:

- Passos definidos
- Ação consistente
- Adaptação a obstáculos



Componentes da Jornada



Desejador (indivíduo com o querer)
Roteiro (plano e passos)
Ferramentas (recursos)
Esforço (energia investida)

Benefícios da Jornada

- **Flexibilidade**
- **Crescimento Pessoal**
Encontrar novas formas de solucionar problemas e ver o progresso passo a passo.
- **Sentimento de Conquista**

Exemplo Prático - Conquista da Montanha



PlanejadorContexto planejador = new PlanejadorContexto();
planejador.setEstrategia(new ConquistaDaMontanha());
planejador.executar();

Desafios

- **Proliferação de Obstáculos**
- **Cansaço e Estresse**

Resumo da Jornada: A Jornada do "Eu Quero" é o processo de transformar um desejo vago em uma conquista tangível. Exige planejamento, ação consistente, superação de obstáculos e paciência.

(FGV / AMAZUL - 2026) Uma equipe de desenvolvimento identificou que o código de um módulo de autenticação está sendo duplicado em vários pontos do sistema. A equipe decide refatorar o código aplicando um padrão que centralize a criação de objetos de autenticação, permitindo que subclasses decidam qual tipo específico de autenticador instanciar (OAuth, LDAP ou local). O padrão de projeto que atende a essa necessidade é chamado

- Abstract Factory.
- Factory Method.
- Builder.
- Prototype.
- Singleton.

Comentários: O ponto central é centralizar a criação e delegar às subclasses a decisão sobre qual autenticador concreto instanciar, que é a essência do Factory Method. Abstract Factory cria famílias de objetos relacionados; Builder monta objetos complexos passo a passo; Prototype clona instâncias; e Singleton apenas garante instância única (Letra B).

Prototype

INCIDÊNCIA EM PROVA: BAIXA

Descrição Resumida: esse padrão especifica os tipos de objetos para criar usando uma instância como protótipo e cria novos objetos copiando este protótipo.

O padrão de projeto Prototype é indicado quando a criação de determinados objetos é trabalhosa ou quando vários deles compartilham praticamente o mesmo estado inicial. Como tornar esse processo mais eficiente? Em vez de criar cada objeto do zero, **a aplicação mantém objetos protótipos previamente configurados e gera novas instâncias por meio de sua clonagem**. Depois disso, apenas os atributos necessários são modificados para atender ao novo contexto.

Imagine uma classe Pessoa com centenas de atributos, como nome, idade, endereço, telefone residencial, nacionalidade e classe social. Após preencher completamente os dados referentes ao pai de uma família, ainda será necessário cadastrar a mãe e os filhos. Seria realmente necessário repetir todo esse preenchimento? Em muitos casos, diversos atributos possuirão exatamente os mesmos valores para todos os membros da família.

Nessa situação, o Prototype permite clonar o objeto já existente e alterar somente as informações que diferem, como nome e idade. Dessa forma, atributos compartilhados, como endereço, telefone residencial e nacionalidade, permanecem copiados para a nova instância, reduzindo o trabalho necessário para criar cada objeto. Esse mecanismo também diminui a probabilidade de erros decorrentes do preenchimento repetitivo de informações.

Além de simplificar a criação de objetos semelhantes, o Prototype reduz a dependência da instanciação direta de classes concretas e pode diminuir a necessidade de criar diversas subclasses para representar pequenas variações de configuração. Outra dúvida frequente é se os protótipos precisam ser definidos antecipadamente. Não necessariamente, pois **novos protótipos podem ser registrados durante a execução da aplicação** e utilizados como base para a clonagem de novos objetos sempre que necessário.

```
import copy

class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

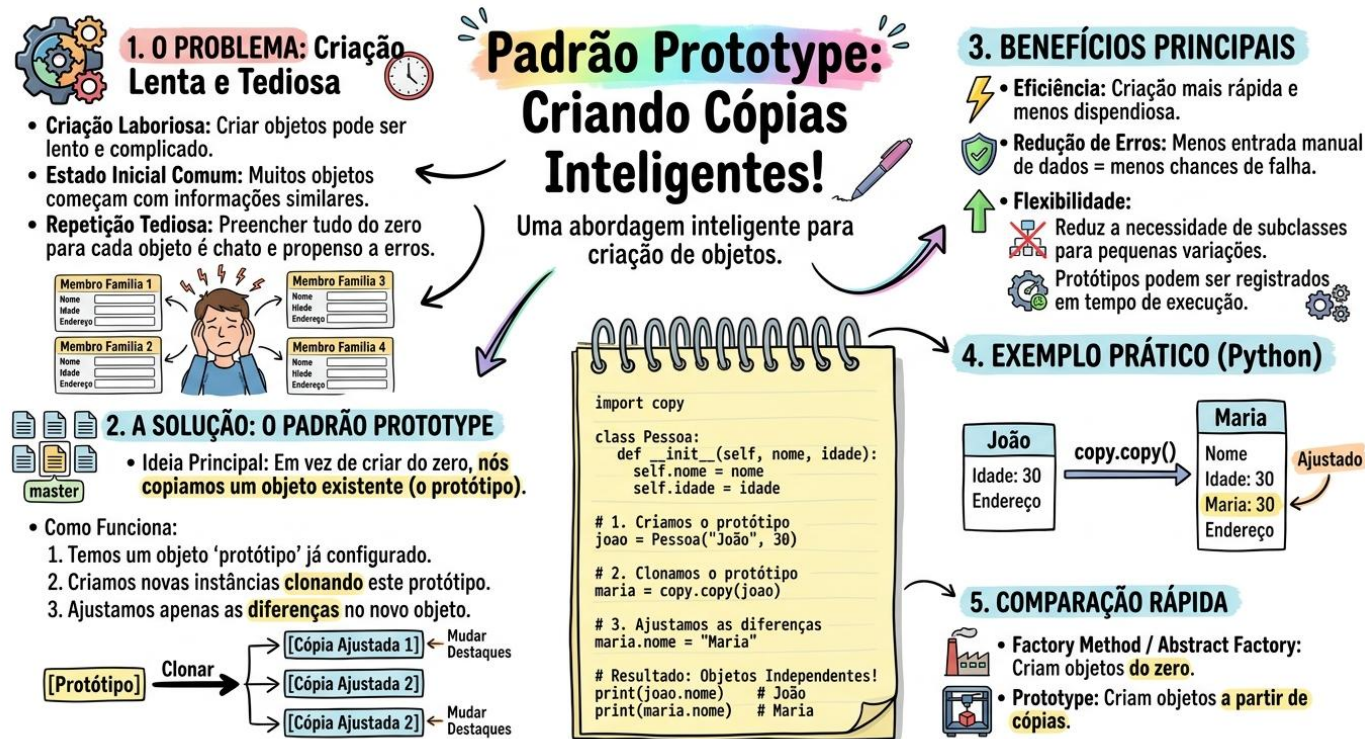
joao = Pessoa("João", 30)

maria = copy.copy(joao)
maria.nome = "Maria"

print(joao.nome)
print(maria.nome)
```

Nesse exemplo, a classe Pessoa representa o objeto que será copiado. Inicialmente, é criado um objeto chamado joao. Em vez de criar outro objeto do zero, utiliza-se a função copy.copy() para produzir uma cópia desse objeto. Após a clonagem, apenas o atributo nome é alterado, resultando em um novo objeto (maria) independente do original.

A principal ideia do Prototype é criar novos objetos por meio da cópia de objetos já existentes, em vez de instanciá-los diretamente. Esse padrão é especialmente útil quando a criação de um objeto é complexa, demorada ou envolve muitas configurações. Assim, basta clonar um objeto já configurado e modificar apenas os atributos necessários. Em suma, **enquanto padrões como Factory Method e Abstract Factory criam objetos do zero, o Prototype cria novos objetos a partir da clonagem de um objeto existente**, reduzindo o custo e simplificando o processo de criação quando muitos objetos possuem características semelhantes.



Ideal para: Objetos complexos, demorados para criar ou com um estado inicial semelhante.

FOOTER:

(FGV / TJ MS - 2024) O sistema instancia um objeto da classe Terceirizado de forma dinâmica, por meio da interface I3rd. Em outro módulo, é preciso instanciar um segundo objeto Terceirizado. Devido ao alto custo de uma nova inicialização dinâmica, Lucas optou por chamar o método cp3rd do objeto já criado, que retorna uma nova instância da classe com estado idêntico ao do objeto chamador. Ao permitir a criação de novas instâncias pelo método cp3rd de uma instância existente, Terceirizado implementa o padrão de projeto:

- a) builder;
- b) singleton;
- c) prototype;
- d) factory method;
- e) abstract factory.

Comentários: criar um novo objeto a partir da clonagem de uma instância já existente, copiando seu estado, caracteriza o Prototype. Builder monta objetos complexos por etapas; Singleton garante instância única; Factory Method delega a criação a subclasses; e Abstract Factory cria famílias de objetos (Letra C).

Singleton

INCIDÊNCIA EM PROVA: ALTÍSSIMA

Descrição Resumida: esse padrão garante que uma classe tenha apenas uma instância e provê um ponto de acesso global a ela.

O padrão de projeto Singleton é utilizado quando uma aplicação precisa garantir que exista apenas uma instância de determinada classe durante seu ciclo de execução. Em que situações isso faz sentido? Geralmente quando um recurso deve ser compartilhado por toda a aplicação, como um gerenciador de configurações, um serviço de registro de logs ou outro componente cuja duplicação poderia provocar inconsistências ou dificultar o controle de seu estado.

Uma característica importante desse padrão é a **existência de um ponto de acesso global à instância única**. Por que isso é necessário? Porque diferentes partes da aplicação podem precisar utilizar o mesmo objeto, sem criar novas instâncias a cada solicitação. Dessa forma, todos os clientes compartilham exatamente o mesmo objeto, o que mantém o estado centralizado e facilita seu gerenciamento.

É comum encontrar exemplos que associam o Singleton ao gerenciamento de conexões com bancos de dados. Essa associação está sempre correta? Nem sempre, pois aplicações modernas costumam utilizar pools de conexões, nos quais várias conexões permanecem disponíveis para reutilização. Nesses casos, **o Singleton pode ser empregado para gerenciar o pool, mas não necessariamente para representar uma única conexão com o banco de dados**.

O funcionamento do Singleton **baseia-se na restrição da instanciação da classe e na disponibilização de um método responsável por fornecer acesso à única instância existente**. Assim, independentemente da quantidade de solicitações realizadas pela aplicação, todas elas utilizam o mesmo objeto, que pode ser criado no início da execução ou apenas quando for solicitado pela primeira vez, conforme a estratégia adotada na implementação.

```
class Configuracao:
    _instancia = None

    def __new__(cls):
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
        return cls._instancia

config1 = Configuracao()
config2 = Configuracao()

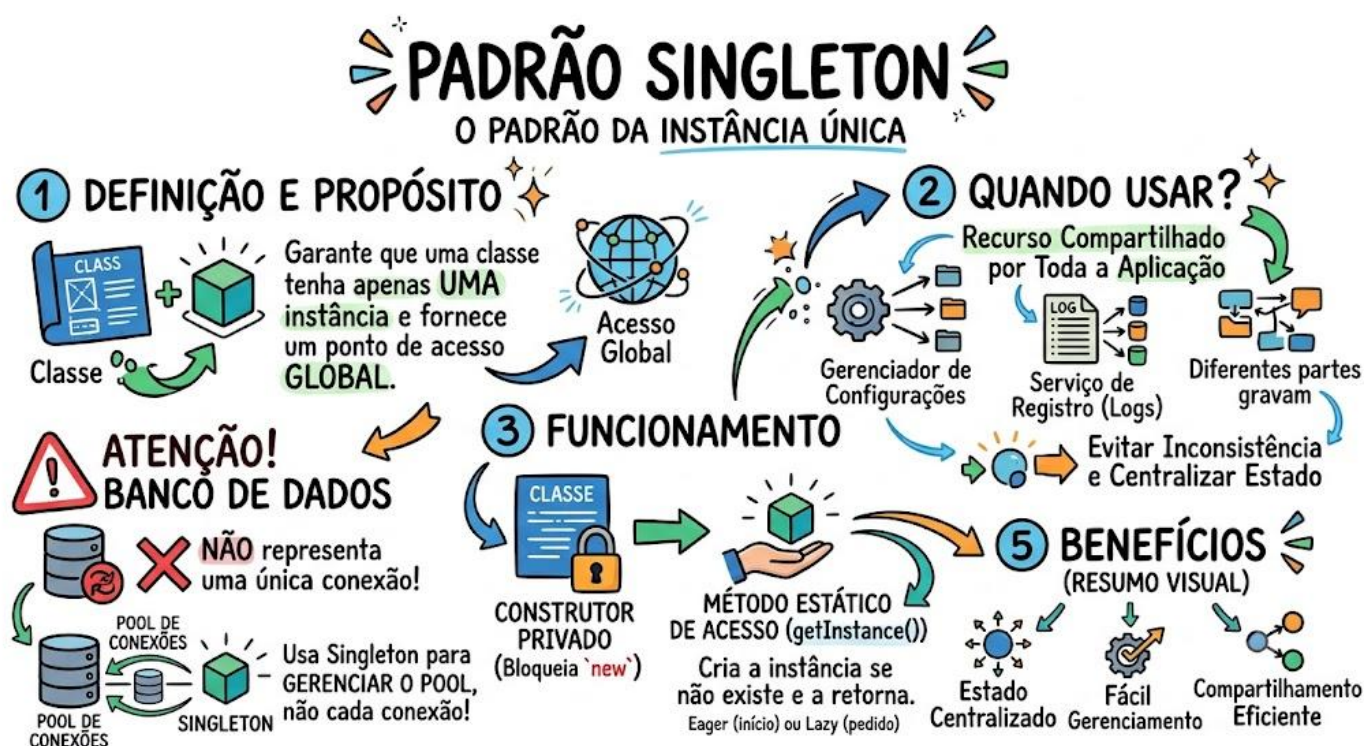
print(config1 is config2)
```

Nesse exemplo, a classe Configuracao implementa o padrão Singleton, garantindo que apenas uma única instância dessa classe seja criada. Isso é feito por meio do método especial __new__(), que verifica se já existe uma instância da classe. Se não existir, ela é criada; caso contrário, a instância já existente é retornada.

A principal ideia do Singleton é assegurar que uma classe possua apenas um único objeto durante toda a execução do programa, fornecendo um ponto único de acesso a ele. Assim, mesmo que o código execute Configuracao() várias vezes, todas as variáveis referenciarão exatamente o mesmo objeto, como demonstrado pela expressão config1 is config2, que retorna True.

Esse padrão é útil quando deve existir apenas uma instância de determinado objeto, como uma configuração global da aplicação, um gerenciador de logs, uma conexão com um banco de dados ou um cache compartilhado. Dessa forma, evita-se a criação desnecessária de múltiplas instâncias e garante-se que toda a aplicação utilize o mesmo objeto.

Apesar das suas utilidades, **o padrão Singleton é amplamente criticado em ambientes de Testes Unitários de software**, sendo comumente **caracterizado pela comunidade moderna como um anti-pattern** (antipadrão) nesse contexto. Como o Singleton introduz um estado global persistente que atravessa todo o ciclo de vida da aplicação, torna-se muito difícil garantir o isolamento e a repetibilidade entre diferentes testes automatizados. A execução de um teste altera os dados estáticos do objeto, o que pode fatalmente "contaminar" e quebrar a execução do próximo teste de maneira imprevisível.



(FGV / AMAZUL - 2026) Em programação orientada a objetos, um desenvolvedor precisa garantir que apenas uma instância de uma classe de conexão com banco de dados seja criada durante toda a execução da aplicação, evitando múltiplas conexões desnecessárias. O padrão de projeto que resolve esse problema é o

- a) Factory Method.
- b) Observer.
- c) Singleton.
- d) Strategy.
- e) Decorator.

Comentários: O Singleton é o padrão pensado para permitir que exista apenas uma instância de uma classe, com acesso global controlado. Factory Method trata da criação por subclasses; Observer notifica mudanças de estado; Strategy encapsula algoritmos intercambiáveis; e Decorator adiciona responsabilidades dinamicamente (Letra C).

Padrões Estruturais

PADRÕES ESTRUTURAIS

Os Padrões Estruturais são uma categoria dos padrões GoF que se concentram na forma como classes e objetos são organizados para formar estruturas maiores e mais flexíveis. Seu principal objetivo é facilitar a composição de classes e objetos, promovendo baixo acoplamento, reutilização de código e maior facilidade de manutenção. Esses padrões definem maneiras eficientes de estabelecer relacionamentos entre componentes, permitindo que sistemas complexos sejam construídos de forma modular. Fazem parte dessa categoria os padrões Adapter, Bridge, Composite, Decorator, Facade, Flyweight e Proxy.

Adapter

INCIDÊNCIA EM PROVA: MÉDIA

Descrição Resumida: esse padrão converte a interface de uma classe em outra interface que normalmente não poderiam trabalhar juntas por serem incompatíveis.

O padrão de projeto Adapter é utilizado para permitir que classes com interfaces incompatíveis possam trabalhar em conjunto. Como isso acontece? O Adapter atua como um intermediário que recebe chamadas utilizando a interface esperada pelo cliente e as converte para a interface disponibilizada pelo objeto adaptado. Dessa forma, componentes originalmente incompatíveis conseguem cooperar sem que suas implementações precisem ser modificadas.

Uma analogia bastante conhecida é a de um adaptador de tomada. O que fazer quando um equipamento possui um plugue incompatível com a tomada disponível? Em vez de alterar o equipamento ou a instalação elétrica, utiliza-se um adaptador que compatibiliza as interfaces. O mesmo princípio é aplicado em software, pois o Adapter permite integrar componentes desenvolvidos de forma independente.

Considere um sistema responsável por armazenar dados de formulários de visto utilizando o formato de data DD/MM/AAAA, enquanto outro sistema espera receber datas no formato MM/DD/AAAA. Como esses sistemas podem se comunicar sem alterações em suas implementações? Um Adapter pode receber os dados em um formato, realizar a conversão necessária e disponibilizá-los na forma esperada pelo sistema de destino.

Outra dúvida frequente é se o Adapter serve apenas para converter formatos de dados. Essa é apenas uma de suas aplicações. **O objetivo principal do padrão é adaptar interfaces incompatíveis**, permitindo que o código cliente continue utilizando a interface que conhece, enquanto o Adapter traduz as chamadas para a interface do componente que realmente executará a operação.

```
class TomadaEuropeia:
    def fornecer_energia(self):
        print("Fornecendo energia pela tomada europeia.")
```

```
class Adaptador:
    def __init__(self, tomada):
        self.tomada = tomada

    def ligar(self):
        self.tomada.fornecer_energia()
```

```
tomada = TomadaEuropeia()
```

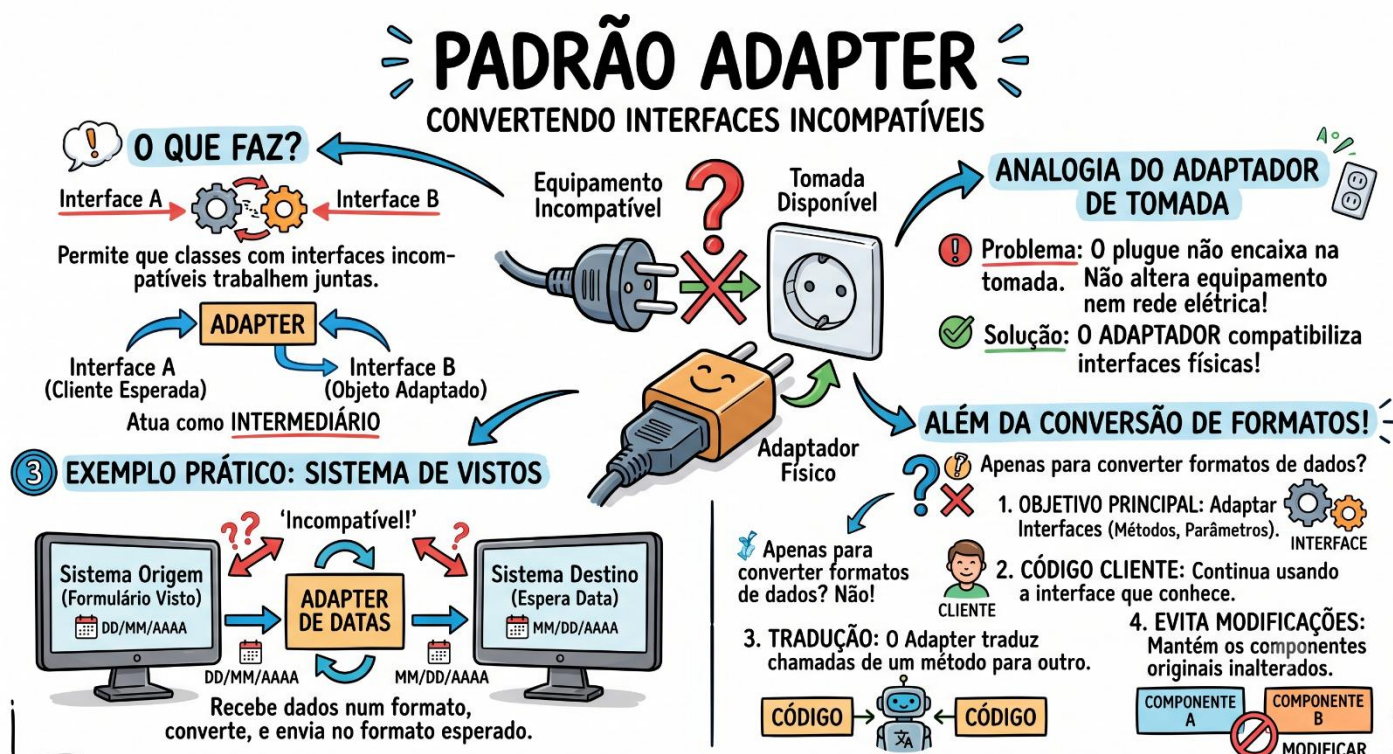
```
adaptador = Adaptador(tomada)
```

```
adaptador.ligar()
```

Nesse exemplo, a classe TomadaEuropeia representa um componente cuja interface não é compatível com a esperada pelo cliente. A classe Adaptador atua como um intermediário, oferecendo o método ligar(), que é o método esperado pelo cliente, mas internamente delegando a chamada ao método fornecer_energia() da tomada europeia.

A principal ideia do Adapter é permitir que classes com interfaces incompatíveis trabalhem juntas. **Em vez de modificar uma classe já existente, cria-se um adaptador que converte a interface disponível para a interface esperada pelo cliente.** Assim, o código cliente continua funcionando sem precisar conhecer os detalhes da adaptação.

Esse padrão é útil quando se deseja reutilizar componentes existentes cuja interface é diferente da esperada pela aplicação. Exemplos comuns incluem adaptadores de tomadas, integração com bibliotecas de terceiros, sistemas legados, APIs externas e drivers de dispositivos. Em todos esses casos, o Adapter atua como uma "ponte", traduzindo uma interface em outra compatível.



(FCC / MPE PB - 2023) Um Analista criou uma aplicação de monitoramento do mercado de ações, que faz o download dos dados em formato XML. Agora é necessário que a aplicação integre uma biblioteca de análise de terceiros, que trabalha apenas com dados em formato JSON. Para isso, o Analista deve utilizar um padrão de projeto GOF que converte a interface de uma classe para outra interface que o cliente espera encontrar. Este padrão é denominado

- a) Adapter.
- b) Proxy.
- c) Mediator.
- d) Prototype.

e) Builder.

Comentários: O Adapter é o padrão usado para traduzir uma interface para outra esperada pelo cliente, permitindo integrar classes incompatíveis, como no caso de dados XML consumidos por uma biblioteca que espera JSON. Proxy controla acesso; Mediator centraliza comunicação; Prototype clona objetos; e Builder constrói objetos complexos (Letra A).

Bridge

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Descrição Resumida: esse padrão desacopla uma interface de sua implementação, de forma que ambas possam variar independentemente.

O padrão de projeto Bridge é utilizado para separar uma abstração de sua implementação, permitindo que ambas evoluam de forma independente. Como essa separação é realizada? A abstração mantém uma referência para um objeto responsável pela implementação concreta, **utilizando composição em vez de herança para estabelecer essa relação**. Assim, alterações em uma hierarquia podem ocorrer sem exigir modificações na outra.

Imagine um sistema de janelas gráficas que precisa funcionar em plataformas como Windows, Linux e macOS. Seria conveniente criar uma subclasse para cada combinação entre tipo de janela e sistema operacional? Essa abordagem faria a quantidade de classes crescer rapidamente, pois cada novo tipo de janela precisaria ser combinado com todas as plataformas existentes, tornando a manutenção mais complexa.

O Bridge evita esse problema ao dividir o sistema em duas hierarquias independentes. Uma delas representa os diferentes tipos de janelas, como diálogo, aviso e erro; a outra representa as plataformas responsáveis por sua implementação. Quando uma operação é solicitada, a abstração delega sua execução ao objeto que representa a plataforma correspondente, preservando o desacoplamento entre essas duas dimensões.

Outra dúvida frequente é por que esse problema não deve ser resolvido com o Adapter. Embora ambos promovam integração entre componentes, seus objetivos são diferentes. **O Adapter adapta interfaces incompatíveis que já existem, enquanto o Bridge é utilizado desde o projeto da aplicação para separar abstrações e implementações** que podem variar independentemente. Dessa forma, novos tipos de janelas e novas plataformas podem ser incorporados sem provocar uma multiplicação de subclasses.

```
class TV:
    def ligar(self):
        print("Ligando a TV.")

class Radio:
    def ligar(self):
        print("Ligando o rádio.")

class ControleRemoto:
    def __init__(self, dispositivo):
        self.dispositivo = dispositivo

    def ligar(self):
        self.dispositivo.ligar()

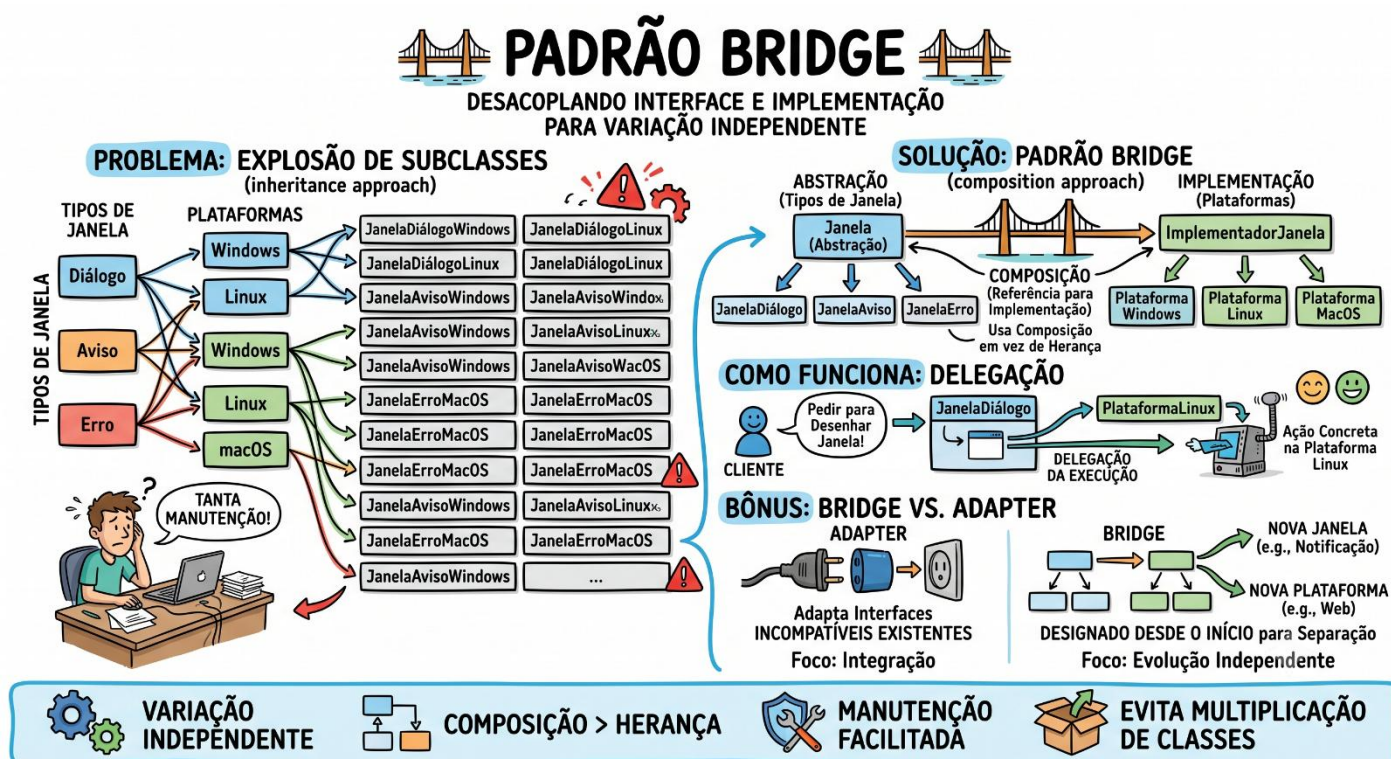
controle = ControleRemoto(TV())
controle.ligar()
```

Nesse exemplo, as classes TV e Radio representam diferentes dispositivos que podem ser controlados. A classe ControleRemoto não está vinculada a um dispositivo específico; ela apenas mantém uma referência

para um objeto que possua o método `ligar()`. Dessa forma, o mesmo controle remoto pode operar tanto uma televisão quanto um rádio.

A principal ideia do Bridge é separar uma abstração de sua implementação, permitindo que ambas evoluam de forma independente. Em vez de criar uma classe para cada combinação possível (como `ControleTV`, `ControleRadio`, `ControleAvancadoTV`, `ControleAvancadoRadio` etc.), **o padrão conecta a abstração (o controle remoto) à implementação (o dispositivo) por meio de uma associação**.

Esse padrão é útil quando existem duas dimensões de variação independentes. No exemplo, há diferentes tipos de controles remotos e diferentes tipos de dispositivos. O Bridge evita a explosão de subclasses e permite combinar qualquer controle com qualquer dispositivo, tornando o sistema mais flexível e fácil de expandir.



Composite

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Descrição Resumida: esse padrão compõe objetos em estruturas de árvore para representar hierarquias parte-todo, permitindo aos clientes tratarem objetos individuais e composições de objetos uniformemente.

O padrão de projeto Composite é utilizado para representar hierarquias do tipo parte todo por meio de estruturas em árvore. O que caracteriza esse tipo de organização? Um objeto pode representar um elemento simples ou uma composição formada por vários outros objetos do mesmo tipo. Para o código cliente, essa diferença é ocultada, pois todos os elementos são acessados por meio de uma interface comum.

Considere uma interface gráfica composta por painéis, botões, caixas de texto e outros componentes visuais. O que acontece quando um painel contém outros painéis? Cada painel pode reunir diversos componentes, inclusive novas composições, formando uma estrutura hierárquica semelhante a uma árvore. Essa organização permite representar interfaces complexas de maneira consistente e organizada.

Uma dúvida comum é se o cliente precisa identificar quando está manipulando um componente simples ou um composto. **Isso não é necessário, pois ambos implementam a mesma interface.** Assim, operações como desenhar, mover ou atualizar um componente podem ser executadas da mesma forma, independentemente de o objeto representar uma folha da árvore ou um nó composto por vários elementos.

Os elementos compostos normalmente oferecem operações para adicionar, remover e acessar seus componentes filhos, enquanto **as folhas representam os elementos indivisíveis da estrutura.** Essa organização facilita a construção de hierarquias complexas e reduz a necessidade de tratamentos especiais para cada tipo de componente, mantendo a interação uniforme em toda a árvore de objetos.

```
class Arquivo:
    def exibir(self):
        print("Arquivo")

class Pasta:
    def __init__(self):
        self.itens = []

    def adicionar(self, item):
        self.itens.append(item)

    def exibir(self):
        print("Pasta")
        for item in self.itens:
            item.exibir()

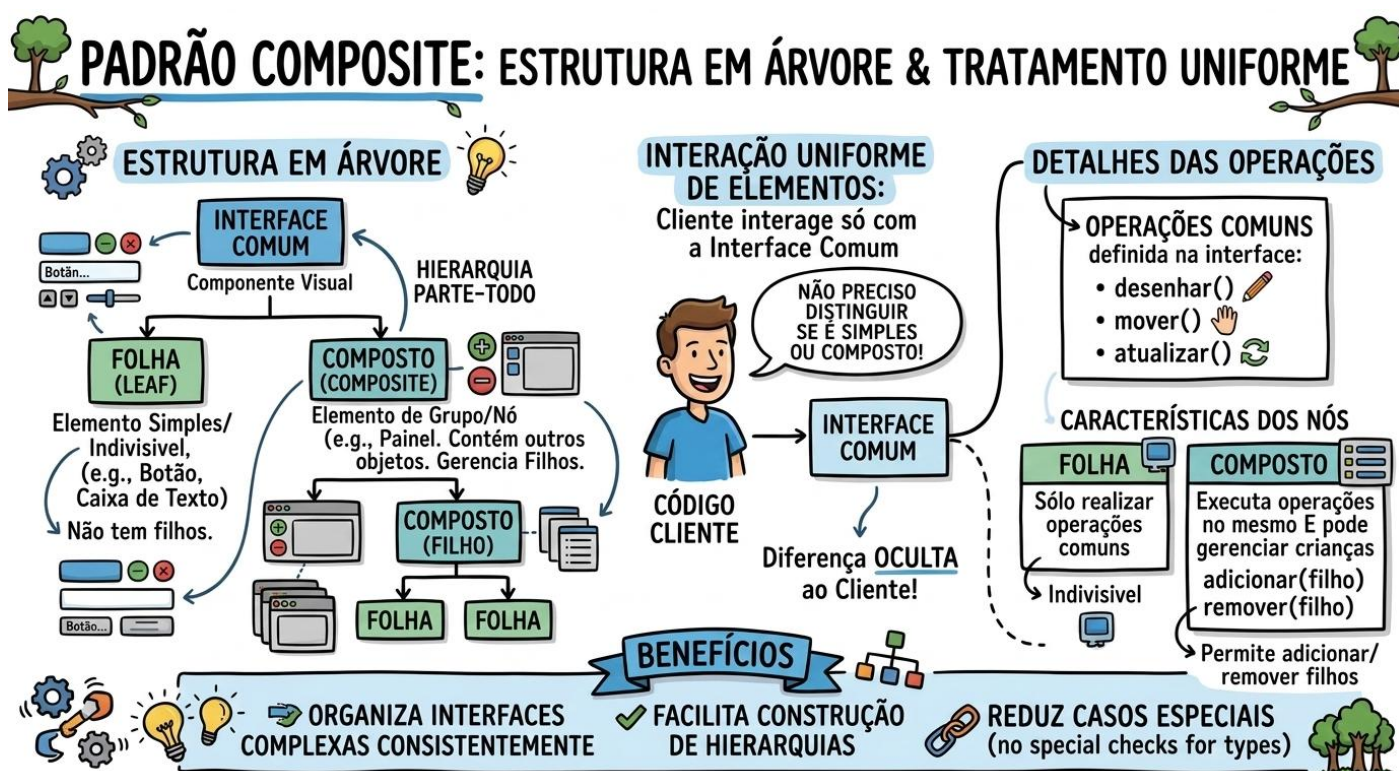
pasta = Pasta()
pasta.adicionar(Arquivo())
pasta.adicionar(Arquivo())
pasta.exibir()
```

Nesse exemplo, a classe Arquivo representa um objeto simples (folha), enquanto a classe Pasta representa um objeto composto, capaz de armazenar outros objetos, sejam eles arquivos ou até mesmo

outras pastas. Ao chamar o método `exibir()` da pasta, ela percorre todos os seus elementos e solicita que cada um exiba seu conteúdo.

A principal ideia do Composite é **permitir que objetos individuais e composições de objetos sejam tratados de maneira uniforme**. Assim, o cliente pode manipular um único arquivo ou uma pasta contendo diversos arquivos utilizando a mesma interface, sem precisar saber se está lidando com um objeto simples ou com um conjunto de objetos.

Esse padrão é muito utilizado para representar estruturas hierárquicas em forma de árvore, como sistemas de arquivos (pastas e arquivos), organogramas, menus, componentes gráficos e documentos. Seu principal benefício é simplificar o código cliente, que passa a tratar objetos individuais e coleções de objetos da mesma forma.



(CESGRANRIO / BANESE - 2025) J foi incumbido de criar um novo sistema de arquivos. Esse sistema seguirá uma estrutura de árvore com pastas que podem conter arquivos ou outras pastas. Além disso, tanto as pastas como os arquivos compartilharão operações como copiar, mover e excluir. A partir desse contexto, o padrão de projeto que poderia ajudá-lo nessa tarefa é o

- a) Adapter
- b) Command
- c) Composite
- d) DAO
- e) Singleton

Comentários: O Composite é o padrão ideal para estruturas em árvore do tipo parte-todo, permitindo tratar arquivos e pastas de forma uniforme nas operações como copiar, mover e excluir. Adapter compatibiliza interfaces; Command encapsula ações; DAO trata de persistência; e Singleton restringe instâncias (Letra C).

Decorator

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Descrição Resumida: esse padrão anexa responsabilidades adicionais a um objeto dinamicamente. Fornece uma alternativa flexível em relação à herança para estender funcionalidades.

O padrão de projeto Decorator é utilizado para adicionar funcionalidades a objetos de forma dinâmica, sem alterar sua classe original. Como isso é possível? Em vez de criar novas subclasses para cada combinação de comportamentos, o padrão utiliza objetos decoradores que envolvem o objeto original e acrescentam novas responsabilidades por meio de composição, mantendo a mesma interface utilizada pelo cliente.

Considere uma lanchonete em que um sanduíche pode receber ingredientes adicionais, como queijo, tomate ou bacon, além da possibilidade de remover determinados itens. Seria viável criar uma classe para cada combinação disponível? Essa abordagem faria a quantidade de subclasses crescer rapidamente, pois cada nova combinação exigiria uma implementação específica.

O Decorator evita esse problema permitindo que novos comportamentos sejam adicionados durante a execução da aplicação. Cada decorador representa uma funcionalidade adicional e pode envolver outro decorador ou o próprio objeto original. Dessa forma, diferentes combinações são construídas pela composição de objetos, sem modificar a implementação do sanduíche básico.

Outra dúvida frequente é qual a principal diferença entre Decorator e herança. Na herança, todas as instâncias de uma subclasse compartilham o mesmo comportamento definido por ela. No Decorator, apenas os objetos que forem explicitamente decorados recebem as funcionalidades adicionais. Essa característica oferece maior flexibilidade e reduz a necessidade de criar um grande número de subclasses para representar pequenas variações de comportamento.

```
class Cafe:
    def custo(self):
        return 5

class CafeComLeite:
    def __init__(self, cafe):
        self.cafe = cafe

    def custo(self):
        return self.cafe.custo() + 2

cafe = Cafe()
cafe_com_leite = CafeComLeite(cafe)

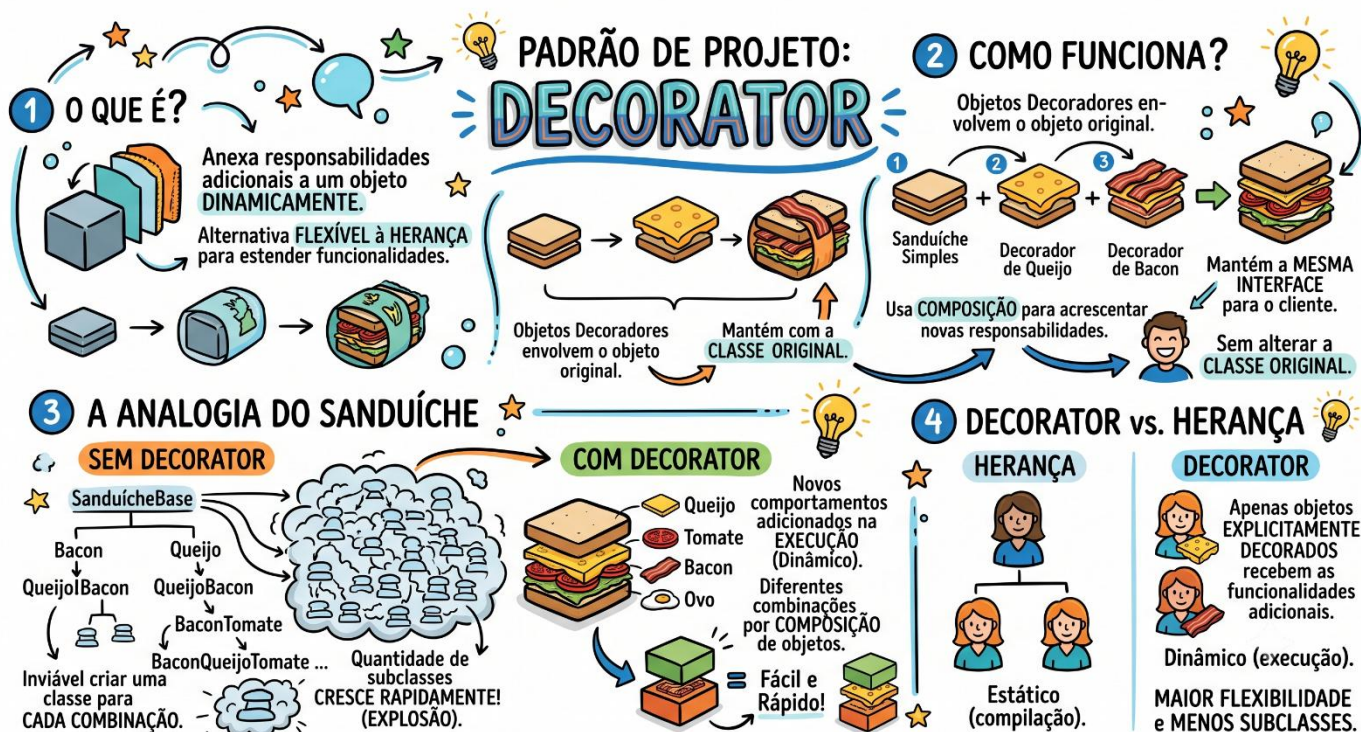
print(cafe_com_leite.custo())
```

Nesse exemplo, a classe Cafe representa o objeto original, cujo custo é de R\$ 5. A classe CafeComLeite atua como um Decorator, pois recebe um objeto Cafe e adiciona uma nova funcionalidade a ele: o acréscimo de R\$ 2 referente ao leite. O objeto original permanece inalterado, mas seu comportamento é enriquecido pelo decorador.

A principal ideia do Decorator é adicionar responsabilidades ou funcionalidades a um objeto de forma dinâmica, sem modificar sua classe original. Para isso, o decorador envolve o objeto existente e

intercepta suas chamadas, podendo manter o comportamento original e acrescentar novas funcionalidades.

Esse padrão é útil quando se deseja combinar funcionalidades de maneira flexível, evitando a criação de inúmeras subclasses. Exemplos comuns incluem cafés com diferentes complementos (leite, chantilly, chocolate), componentes gráficos com barras de rolagem ou bordas, fluxos de entrada e saída de dados (streams) e mecanismos de compressão ou criptografia, nos quais cada funcionalidade é adicionada por um novo decorador.



Façade

INCIDÊNCIA EM PROVA: MÉDIA

Descrição Resumida: esse padrão oferece uma interface unificada para um conjunto de interfaces em um subsistema, definindo uma interface de alto nível que facilita a utilização do subsistema.

O padrão de projeto Façade é utilizado para fornecer uma interface unificada e simplificada para um subsistema composto por diversos componentes. Qual é o objetivo dessa abordagem? Permitir que o código cliente acesse as funcionalidades mais comuns sem precisar conhecer os detalhes internos de funcionamento nem a forma como os componentes do subsistema interagem entre si.

Considere uma instituição financeira que possua um sistema legado responsável pelas informações de crédito e um novo sistema que precise utilizar esses dados. Seria conveniente que o novo sistema conhecesse todas as classes e regras do sistema legado? Essa dependência aumentaria a complexidade da integração e tornaria a manutenção mais difícil à medida que o subsistema evoluísse.

O Façade resolve esse problema ao disponibilizar um único ponto de acesso para as operações mais utilizadas. Assim, o cliente realiza suas solicitações por meio dessa interface de alto nível, enquanto a Façade coordena internamente as chamadas necessárias aos componentes do subsistema. Dessa forma, o cliente permanece desacoplado da complexidade existente na implementação.

Outra dúvida frequente é se o Façade impede o acesso direto aos componentes internos do subsistema. Isso não ocorre, pois **seu objetivo não é substituir essas interfaces, mas oferecer uma alternativa mais simples para os casos mais comuns.** Quando necessário, outros componentes da aplicação ainda podem acessar diretamente os serviços específicos do subsistema.

```
class TV:
    def ligar(self):
        print("TV ligada.")

class Som:
    def ligar(self):
        print("Som ligado.")

class HomeTheater:
    def __init__(self):
        self.tv = TV()
        self.som = Som()

    def assistir_filme(self):
        self.tv.ligar()
        self.som.ligar()

home = HomeTheater()
home.assistir_filme()
```

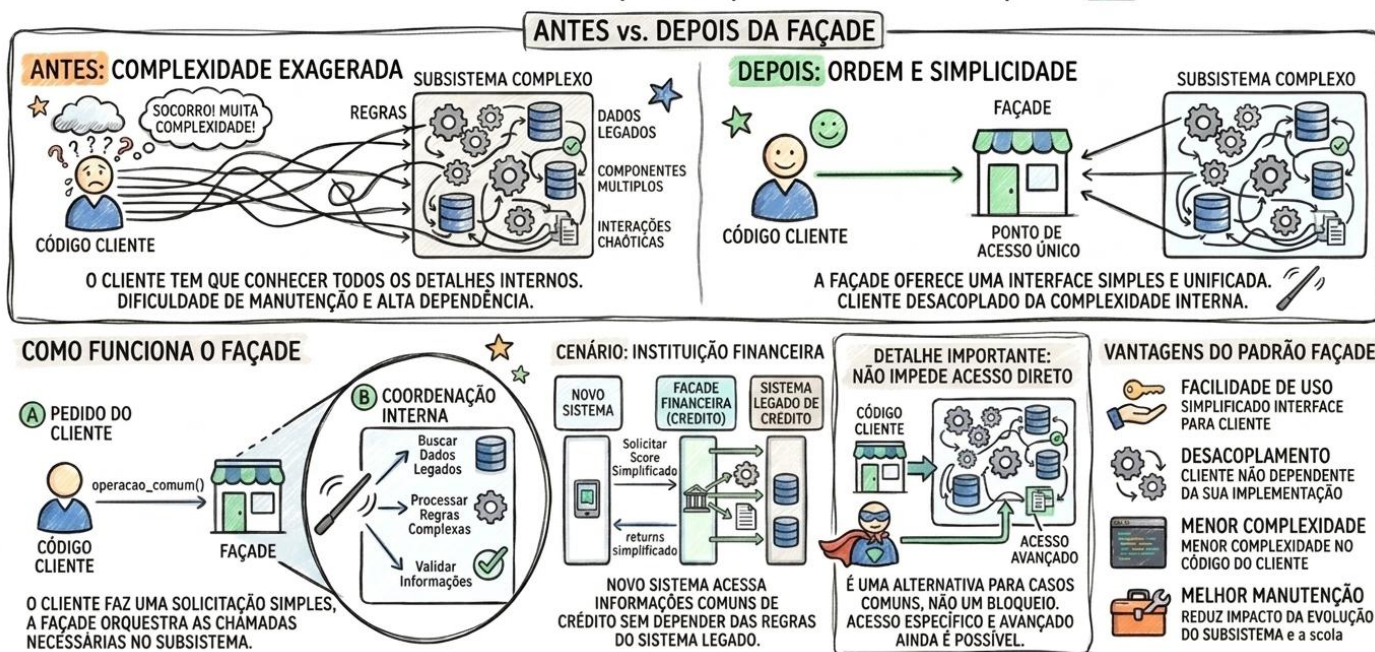
Nesse exemplo, as classes TV e Som representam componentes do sistema. Para assistir a um filme, seria necessário ligar cada equipamento separadamente. A classe HomeTheater atua como uma Façade, oferecendo um único método (assistir_filme()) que coordena todas as chamadas necessárias aos componentes internos.

A principal ideia do Façade é fornecer uma interface simplificada para um conjunto de classes ou subsistemas mais complexos. Em vez de o cliente precisar conhecer e interagir com vários objetos, ele utiliza apenas a fachada, que centraliza e organiza as chamadas aos componentes internos.

Esse padrão é útil quando um sistema possui muitos objetos ou operações que normalmente são utilizados em conjunto. Exemplos comuns incluem sistemas de home theater, bibliotecas de software, APIs complexas e serviços corporativos, nos quais a fachada oferece um ponto único de acesso, reduzindo a complexidade e o acoplamento do código cliente.

Padrão de Projeto: FAÇADE

Interface Unificada e Simplificada para Subsistemas Complexos



(QUADRIX / CRM MG - 2023) No que se refere ao padrão de projeto Facade, assinale a alternativa correta.

- a) Ele é usado especificamente na programação estruturada de sistemas.
- b) O padrão Facade não pode ser usado quando há o interesse em dividir seus subsistemas em camadas.
- c) Ele define uma interface comum para um conjunto de interfaces em um subsistema.
- d) Esse padrão promove forte acoplamento entre os subsistemas e seus clientes.
- e) Ele evita que as aplicações acessem as subclasses diretamente.

Comentários: a ideia central do Facade é oferecer uma interface única e mais simples para um conjunto de interfaces de um subsistema. Ele não é exclusivo da programação estruturada, pode sim ser usado com subsistemas em camadas, reduz o acoplamento em vez de aumentá-lo e não impede o acesso direto aos componentes internos (Letra C).

Flyweight

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Descrição Resumida: esse padrão utiliza compartilhamento para suportar eficientemente grandes quantidades de objetos de baixa granularidade.

O padrão de projeto Flyweight é utilizado para reduzir o consumo de memória quando uma aplicação precisa manipular uma grande quantidade de objetos semelhantes. Como isso é possível? O padrão identifica as informações que podem ser compartilhadas entre diferentes objetos, denominadas estado intrínseco, enquanto mantém separadamente os dados específicos de cada ocorrência, conhecidos como estado extrínseco.

Considere um editor de texto em que cada caractere seja representado por um objeto. Seria eficiente armazenar repetidamente as mesmas informações para todas as ocorrências da letra "A"? Essa abordagem desperdiçaria memória, pois muitos caracteres compartilham exatamente as mesmas características. Em vez disso, todas as ocorrências podem utilizar um único objeto Flyweight contendo os dados que permanecem iguais.

Outra dúvida frequente é quais informações podem ser compartilhadas. Características comuns, como o próprio símbolo do caractere e outras propriedades que não variam entre as ocorrências, fazem parte do estado intrínseco. **Já informações como a posição ocupada pelo caractere no documento dependem de cada ocorrência individual e constituem o estado extrínseco**, permanecendo armazenadas separadamente.

Essa organização reduz significativamente a duplicação de informações na memória, pois **cada ocorrência mantém apenas seus dados específicos e uma referência ao objeto Flyweight compartilhado**. Como consequência, aplicações que manipulam milhares ou milhões de objetos semelhantes conseguem utilizar os recursos computacionais de forma mais eficiente, sem alterar o comportamento observado pelo código cliente.

```
class Arvore:
    def __init__(self, especie):
        self.especie = especie

class FabricaArvores:
    def __init__(self):
        self.arvores = {}

    def obter_arvore(self, especie):
        if especie not in self.arvores:
            self.arvores[especie] = Arvore(especie)
        return self.arvores[especie]

fabrica = FabricaArvores()

arvore1 = fabrica.obter_arvore("Ipê")
arvore2 = fabrica.obter_arvore("Ipê")

print(arvore1 is arvore2)
```

Nesse exemplo, a classe Arvore representa um objeto que pode ser compartilhado. A classe FabricaArvores atua como uma fábrica de Flyweights, armazenando os objetos já criados. Quando uma

árvore da espécie Ipê é solicitada pela segunda vez, a fábrica não cria um novo objeto; ela retorna a mesma instância já existente.

A principal ideia do Flyweight é **compartilhar objetos que possuem o mesmo estado para reduzir o consumo de memória**. Em vez de criar milhares de objetos idênticos, cria-se apenas um objeto compartilhado e ele é reutilizado sempre que necessário.

Esse padrão é útil quando uma aplicação manipula uma grande quantidade de objetos semelhantes, como caracteres em um editor de texto, árvores em um jogo, ícones em uma interface gráfica ou partículas em uma simulação. Ao compartilhar os dados comuns entre esses objetos, o Flyweight reduz significativamente o uso de memória e melhora a eficiência da aplicação.



(FGV / TCE SP - 2023) O componente Board apresenta ao usuário indicadores visuais baseados em informações do banco de dados e instancia um novo objeto para cada indicador. Sabendo que determinado subconjunto de informações se repete em todos os indicadores, André aplicou no componente Board o padrão estrutural do GoF que otimiza a alocação de objetos na memória RAM, mediante o compartilhamento de atributos comuns entre os objetos. André aplicou o padrão do GoF:

- a) proxy;
- b) bridge;
- c) facade;
- d) flyweight;
- e) composite.

Comentários: o Flyweight é o padrão usado para reduzir o uso de memória ao compartilhar o estado intrínseco (atributos comuns) entre muitos objetos semelhantes. Proxy controla acesso; Bridge desacopla abstração e implementação; Facade simplifica subsistemas; e Composite organiza hierarquias em árvore (Letra D).

Proxy

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Descrição Resumida: esse padrão provê um substituto ou ponto através do qual um objeto pode controlar o acesso a outro objeto.

O padrão de projeto Proxy é utilizado para fornecer um objeto substituto que controla o acesso a outro objeto, mantendo a mesma interface utilizada pelo cliente. Como isso funciona? O cliente realiza suas solicitações ao Proxy, que decide como e quando encaminhá-las ao objeto real. Durante esse processo, podem ser executadas verificações de segurança, carregamento sob demanda, armazenamento em cache ou outras operações de controle.

Considere uma aplicação que precisa acessar um componente responsável pela comunicação com um banco de dados. Seria adequado permitir que qualquer parte da aplicação interagisse diretamente com esse componente? Em muitos cenários, essa abordagem dificultaria a aplicação de regras de segurança, controle de acesso ou otimizações de desempenho. O Proxy atua justamente como intermediário entre o cliente e o objeto responsável pela operação.

Outra dúvida frequente é se o cliente precisa conhecer a existência do Proxy. **Isso normalmente não ocorre, pois tanto o Proxy quanto o objeto real implementam a mesma interface.** Assim, o código cliente continua realizando chamadas da mesma maneira, enquanto o Proxy executa as ações necessárias antes ou depois de encaminhar a solicitação ao objeto original.

O Proxy pode assumir diferentes responsabilidades conforme a necessidade da aplicação. **Um Proxy virtual adia a criação de objetos cujo custo de inicialização é elevado; um Proxy de proteção controla o acesso ao objeto real com base em regras definidas.** Em ambos os casos, o objetivo permanece o mesmo, pois o acesso ao objeto original ocorre por meio de um componente intermediário responsável por exercer esse controle.

```
class Banco:
    def consultar_saldo(self):
        print("Saldo: R$ 1.000")

class ProxyBanco:
    def __init__(self):
        self.banco = Banco()

    def consultar_saldo(self, autenticado):
        if autenticado:
            self.banco.consultar_saldo()
        else:
            print("Acesso negado.")

proxy = ProxyBanco()

proxy.consultar_saldo(True)
```

Nesse exemplo, a classe Banco representa o objeto real que executa a operação de consultar o saldo. A classe ProxyBanco atua como um Proxy, isto é, um objeto intermediário que controla o acesso ao objeto real. Antes de encaminhar a solicitação, o proxy verifica se o usuário está autenticado. Somente após essa verificação a consulta ao saldo é realizada.

A principal ideia do Proxy é fornecer um substituto ou representante para outro objeto, controlando o acesso a ele. O cliente interage com o proxy como se estivesse acessando o objeto real, mas o proxy pode executar ações adicionais antes ou depois da chamada, como autenticação, controle de permissões, cache, registro de logs ou carregamento sob demanda.

Esse padrão é útil quando se deseja controlar ou otimizar o acesso a um objeto. Exemplos comuns incluem autenticação de usuários, proxies de acesso a serviços remotos, cache de consultas, carregamento preguiçoso (lazy loading) de imagens ou documentos e registro de logs de operações, permitindo adicionar essas funcionalidades sem alterar a implementação do objeto original.



(CESGRANRIO / BASA - 2022) Um sistema cliente-servidor precisa tratar de documentos que possuem imagens muito grandes, que não são necessariamente apresentadas ao usuário sempre que ele carrega um documento. A melhor estratégia é carregar a imagem apenas quando a visualização é necessária. O desenvolvedor decidiu usar um padrão de projeto que fornece, no cliente, um objeto substituto que controla o acesso ao objeto original. Qual foi o padrão de projeto utilizado?

- a) Decorator
- b) Facade
- c) Memento
- d) Observer
- e) Proxy

Comentários: o Proxy cria um objeto substituto no cliente para controlar o acesso ao objeto real, permitindo carregar a imagem somente quando necessário, o que caracteriza o proxy virtual (lazy loading). Decorator adiciona responsabilidades; Facade simplifica subsistemas; Memento guarda estados; e Observer notifica mudanças (Letra E).

Padrões Comportamentais

PADRÕES COMPORTAMENTAIS

Os Padrões Comportamentais são uma categoria dos padrões GoF que se concentram na comunicação, na colaboração e na distribuição de responsabilidades entre objetos durante a execução do sistema. Seu objetivo é definir formas eficientes de interação entre objetos, reduzindo o acoplamento e tornando o comportamento do software mais flexível e fácil de manter. Fazem parte dessa categoria os padrões Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method e Visitor.

Chain of Responsibility

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Descrição Resumida: esse padrão evita o acoplamento do remetente de uma requisição ao seu receptor ao dar a mais de um objeto a chance de lidar com a requisição.

O padrão de projeto Chain of Responsibility é utilizado quando uma solicitação pode ser tratada por diferentes objetos e o remetente não deve conhecer previamente qual deles será responsável pelo processamento. Como isso é possível? **Os objetos são organizados em uma cadeia, na qual cada elemento recebe a requisição, verifica se pode tratá-la e, caso não seja o responsável, encaminha a solicitação ao próximo elemento.**

Esse mecanismo é especialmente útil quando a ordem dos responsáveis pode ser alterada ou quando novos processadores precisam ser adicionados sem modificar o código do remetente. Por que essa flexibilidade é importante? Porque **a lógica de decisão deixa de ficar concentrada em grandes estruturas condicionais** e passa a ser distribuída entre os próprios objetos que compõem a cadeia, reduzindo o acoplamento entre os componentes da aplicação.

Considere uma loja virtual que recebe solicitações de pagamento. Como determinar qual componente deve processar cada operação? Em vez de concentrar toda a decisão em um único ponto do sistema, a solicitação pode percorrer uma cadeia de processadores. Cada um verifica se atende aos critérios necessários para processá-la e, caso contrário, encaminha a requisição ao próximo elemento da sequência.

Todos os elementos da cadeia normalmente implementam a mesma interface ou classe abstrata, o que garante um mecanismo uniforme para receber e encaminhar solicitações. Outra dúvida frequente é se apenas um objeto pode participar do processamento. Isso depende da implementação, pois **um manipulador pode tratar a requisição, encaminhá-la ao próximo elemento ou combinar essas duas ações**, conforme as necessidades da aplicação.

```
class Gerente:
    def aprovar(self, valor):
        if valor <= 1000:
            print("Gerente aprovou.")
            return False

class Diretor:
    def aprovar(self, valor):
        if valor <= 10000:
            print("Diretor aprovou.")
        else:
            print("Presidente aprovou.")
```

```
gerente = Gerente()
diretor = Diretor()

valor = 5000

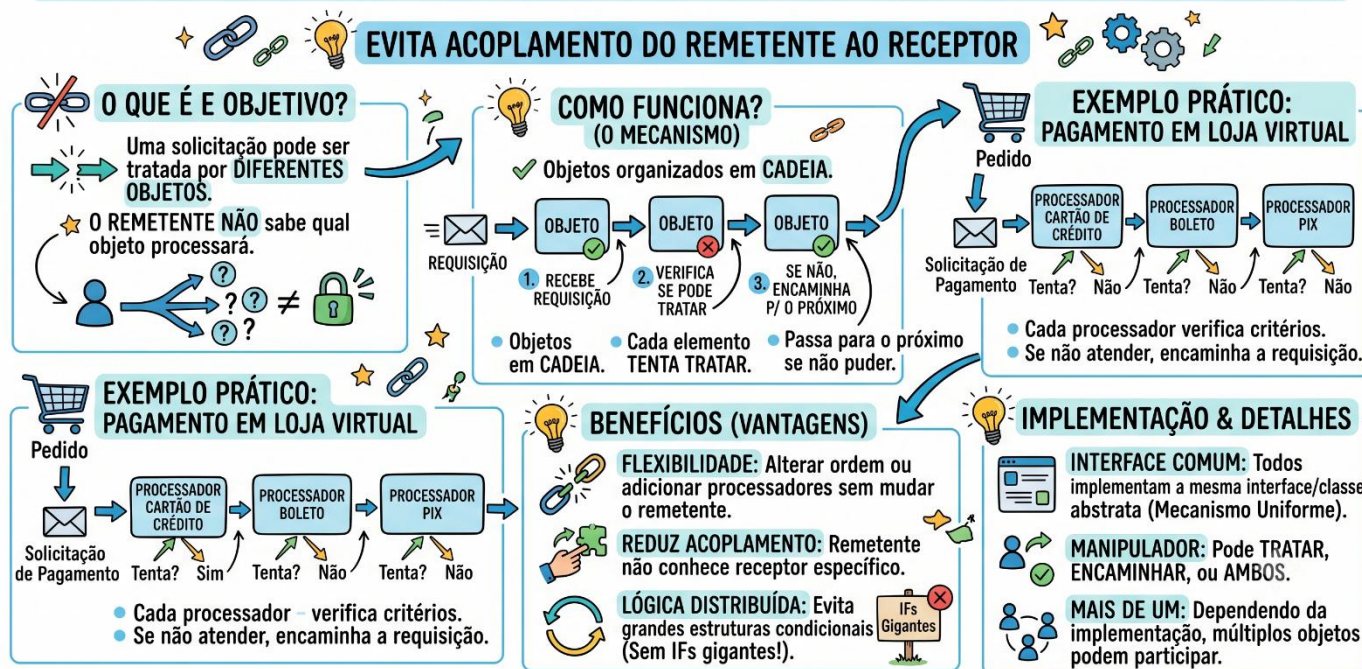
if gerente.aprovar(valor) is False:
    diretor.aprovar(valor)
```

Nesse exemplo, um pedido de aprovação é encaminhado inicialmente ao gerente. Se o valor for de até R\$ 1.000, ele próprio aprova a solicitação. Caso contrário, ele não resolve o problema e o pedido é encaminhado ao próximo responsável da cadeia, que é o diretor. Se o valor ultrapassar R\$ 10.000, o diretor o encaminha para o presidente.

A principal ideia do Chain of Responsibility é organizar vários objetos em uma cadeia, de modo que cada um tenha a oportunidade de tratar uma solicitação. Caso um objeto não consiga atendê-la, ele a repassa para o próximo da cadeia, até que algum objeto a processe ou a cadeia termine.

Esse padrão é útil quando uma solicitação pode ser tratada por diferentes objetos, mas não se sabe antecipadamente qual deles será o responsável. Dessa forma, o remetente da solicitação não precisa conhecer quem irá processá-la, bastando iniciar a requisição no primeiro elemento da cadeia. Isso reduz o acoplamento entre quem faz a solicitação e quem efetivamente a atende.

PADRÃO DE PROJETO: CHAIN OF RESPONSIBILITY (CADEIA DE RESPONSABILIDADE)



(FGV / CVM - 2024) Natália está desenvolvendo um sistema financeiro no qual os documentos podem receber diferentes tipos de tratamento. Para tratar os documentos, ela criou diversos processos, que serão colocados em uma fila. Assim, o documento passa por um dado processo da fila, é tratado e enviado para o processo seguinte. Adotando essa estratégia, Natália usou o padrão de desenvolvimento denominado:

- a) Chain of Responsibility;
- b) Prototype;
- c) Decorator;
- d) Abstract Factory;
- e) Factory Method.

Comentários: No Chain of Responsibility, a requisição percorre uma sequência de objetos e cada um trata sua parte antes de encaminhá-la ao próximo, exatamente como ocorre com os documentos na fila. Prototype, Abstract Factory e Factory Method são criacionais, e Decorator apenas adiciona responsabilidades a um objeto (Letra A).

Command

INCIDÊNCIA EM PROVA: BAIXA

Descrição Resumida: esse padrão encapsula a requisição de um objeto, portanto permitindo que se parametrize os clientes com diferentes requisições.

O padrão de projeto Command encapsula uma requisição na forma de um objeto, permitindo que ela seja armazenada, transmitida, enfileirada ou executada em momentos diferentes. Qual é a principal vantagem dessa abordagem? O objeto que solicita uma ação deixa de conhecer os detalhes de sua execução, pois apenas dispara um comando, enquanto outro objeto é responsável por realizar a operação correspondente.

Um exemplo clássico envolve um interruptor utilizado para controlar uma lâmpada. O que o interruptor realmente faz? Ele não executa diretamente a operação de ligar ou desligar, mas aciona um comando responsável por essa tarefa. Se o mesmo interruptor passar a controlar outro dispositivo, como um computador, basta associá-lo a um comando diferente que implemente as operações adequadas para esse equipamento, sem modificar o funcionamento do interruptor.

Outra característica importante do Command é a possibilidade de armazenar e organizar requisições. Por que isso é útil? Porque **comandos podem ser enfileirados para execução posterior, registrados em logs para auditoria** ou reaplicação após falhas e **utilizados na implementação de funcionalidades como desfazer e refazer operações**. Essas capacidades tornam o padrão especialmente útil em aplicações que executam diversas operações independentes.

Em aplicações que acessam bancos de dados, o Command costuma representar operações específicas, como inserir, atualizar, remover ou consultar registros. Isso significa que a conexão com o banco de dados é encapsulada pelo padrão? Não, pois **o objetivo do Command é encapsular a operação solicitada, mantendo desacoplados o objeto que solicita a execução e o objeto responsável por realizá-la**.

```
class Lampada:
    def ligar(self):
        print("Lâmpada ligada.")

class ComandoLigar:
    def __init__(self, lampada):
        self.lampada = lampada

    def executar(self):
        self.lampada.ligar()

lampada = Lampada()
comando = ComandoLigar(lampada)

comando.executar()
```

Nesse exemplo, a classe Lampada representa o objeto que sabe realizar uma ação, isto é, ligar a lâmpada. Já a classe ComandoLigar representa um comando, encapsulando a solicitação de ligar a lâmpada. Em vez de chamar diretamente lampada.ligar(), o cliente cria um objeto de comando e solicita sua execução por meio do método executar().

A principal ideia do Command é encapsular uma solicitação em um objeto. Dessa forma, a ação deixa de ser uma chamada direta a um método e passa a ser representada por um objeto que pode ser

armazenado, transmitido, agendado ou executado posteriormente. Além disso, quem solicita a execução não precisa conhecer os detalhes de como a ação será realizada.

Esse padrão é muito utilizado em situações como botões de interfaces gráficas, menus, atalhos de teclado, filas de tarefas, automação de processos e mecanismos de desfazer/refazer (undo/redo). Em todos esses casos, as ações são tratadas como objetos independentes, tornando o sistema mais flexível e desacoplado.



(VUNESP / EPC - 2023) Um determinado design pattern consiste em transformar uma requisição em um objeto que carrega toda a informação necessária sobre essa requisição, podendo ser usado para armazenar requisições em uma fila para processamento posterior. Esse design pattern é conhecido como

- a) Command.
- b) Singleton.
- c) Factory.
- d) Memento.
- e) Template Method.

Comentários: o Command encapsula uma requisição em um objeto, levando junto os dados necessários para executá-la depois, o que permite enfileirar, registrar em log ou desfazer ações. Singleton trata de instância única; Factory foca na criação de objetos; Memento guarda e restaura estados; e Template Method define o esqueleto de um algoritmo (Letra A).

Interpreter

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Descrição Resumida: esse padrão, dada uma linguagem, define uma representação para sua gramática em conjunto com um interpretador que utiliza a representação para interpretar sentenças na linguagem.

O padrão de projeto Interpreter é utilizado quando uma aplicação precisa interpretar expressões escritas em uma linguagem cuja gramática é conhecida e relativamente simples. O que significa interpretar uma linguagem nesse contexto? Significa analisar uma expressão de acordo com regras previamente definidas e produzir um resultado correspondente. Para isso, **as expressões costumam ser representadas por uma árvore sintática abstrata, na qual cada elemento corresponde a uma regra da gramática.**

Esse padrão é mais indicado quando a gramática possui baixa complexidade e o desempenho não representa um requisito crítico. Por que essa restrição existe? Porque, à medida que a linguagem cresce, também aumenta a quantidade de classes necessárias para representar suas regras gramaticais, tornando a solução mais difícil de manter. Em gramáticas muito extensas, normalmente são utilizados analisadores sintáticos e geradores de compiladores específicos.

É comum surgir uma dúvida ao estudar esse padrão: ele está relacionado ao funcionamento de linguagens como Java? A resposta é não. Embora a Máquina Virtual Java execute bytecode durante a execução da aplicação, **o padrão Interpreter possui outro objetivo, pois ele modela a gramática de uma linguagem por meio de classes capazes de interpretar expressões específicas.** Assim, não deve ser confundido com o processo de execução realizado por uma linguagem de programação.

Aplicações que utilizam expressões matemáticas, filtros de pesquisa, regras de negócio ou linguagens específicas de domínio são exemplos mais adequados para esse padrão. Como cada regra da gramática é representada? Normalmente, **cada regra corresponde a uma classe que implementa a lógica de interpretação de uma parte da linguagem.** Em conjunto, essas classes permitem analisar e interpretar expressões completas de forma estruturada.

```
class Soma:
    def interpretar(self, a, b):
        return a + b

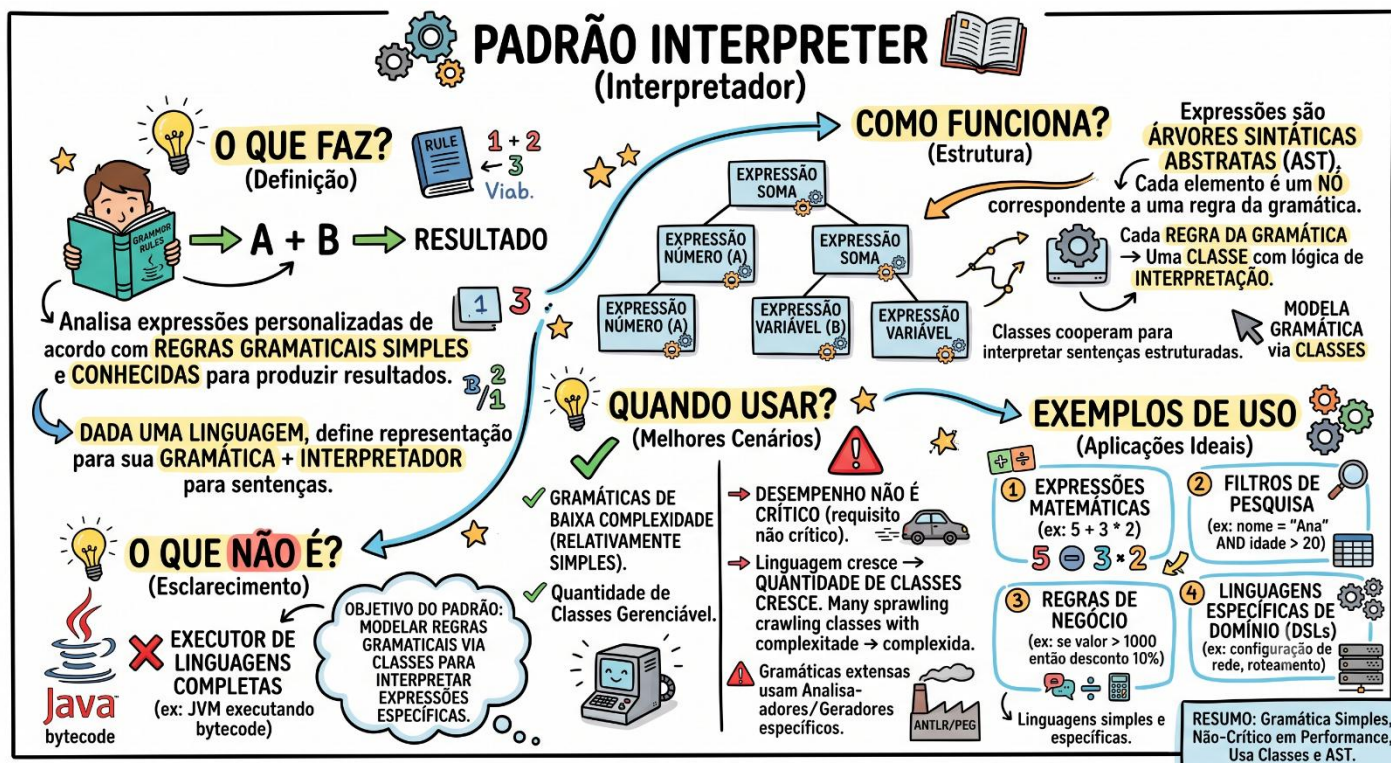
expressao = Soma()

resultado = expressao.interpretar(10, 5)

print(resultado)
```

No exemplo, a classe Soma representa uma expressão da linguagem que sabe interpretar uma operação específica: a soma. O método interpretar() recebe dois valores e devolve o resultado da operação. O cliente apenas cria a expressão e solicita sua interpretação. A principal ideia do Interpreter é representar elementos de uma linguagem por meio de classes, permitindo que cada classe saiba interpretar uma determinada regra gramatical. Em vez de escrever toda a lógica de interpretação em um único bloco de código, cada regra da linguagem é encapsulada em um objeto responsável por interpretá-la.

Esse padrão é útil quando se deseja criar interpretadores para linguagens simples ou domínios específicos (DSLs), como expressões matemáticas, filtros de busca, consultas, regras de negócio e comandos. À medida que a linguagem cresce, novas regras podem ser representadas por novas classes, tornando o interpretador mais organizado e extensível.



Iterator

INCIDÊNCIA EM PROVA: BAIXA

Descrição Resumida: esse padrão fornece uma maneira de acessar elementos de um objeto agregado sequencialmente sem expor sua representação interna.

O padrão de projeto Iterator é utilizado para percorrer os elementos de uma coleção sem expor a forma como ela está implementada internamente. O que isso significa na prática? Significa que o código responsável pela navegação conhece apenas a interface do iterador, enquanto os detalhes da estrutura utilizada para armazenar os elementos permanecem encapsulados na própria coleção.

Esse padrão também fornece uma interface uniforme para percorrer diferentes tipos de coleções. Por que isso é importante? Porque listas, arrays, conjuntos e mapas possuem mecanismos distintos de armazenamento e navegação. Com o Iterator, **o código cliente utiliza o mesmo padrão de acesso aos elementos, independentemente da estrutura de dados empregada.** No caso dos mapas, por exemplo, a iteração pode ocorrer sobre as chaves, os valores ou os pares chave e valor.

Outra característica importante é a possibilidade de realizar múltiplos percursos independentes sobre a mesma coleção. Como isso é possível? **Cada percurso pode utilizar uma instância diferente do iterador**, permitindo que vários processos naveguem pelos mesmos dados sem interferir uns nos outros. Essa abordagem aumenta a flexibilidade da aplicação e mantém desacoplados o mecanismo de navegação e a implementação da coleção.

Considere uma aplicação que precise percorrer sequencialmente listas, arrays, conjuntos e mapas contendo objetos complexos. Seria necessário conhecer a forma específica de navegação de cada estrutura? Com o Iterator, essa necessidade deixa de existir, pois todas as coleções disponibilizam um mecanismo uniforme para acessar seus elementos, enquanto seus detalhes internos permanecem ocultos do código cliente.

```
class Numeros:
    def __init__(self):
        self.lista = [10, 20, 30]

    def __iter__(self):
        return iter(self.lista)

numeros = Numeros()

for numero in numeros:
    print(numero)
```

Nesse exemplo, a classe Numeros representa uma coleção de elementos. O método __iter__() retorna um iterador para a lista armazenada no objeto, permitindo que seus elementos sejam percorridos utilizando um laço for. O cliente não precisa conhecer como os dados estão armazenados internamente; ele apenas percorre a coleção.

A principal ideia do Iterator é fornecer uma maneira de acessar sequencialmente os elementos de uma coleção sem expor sua estrutura interna. Assim, o código cliente pode percorrer listas, conjuntos, árvores ou outras estruturas de dados utilizando uma interface comum, sem depender da forma como esses elementos são armazenados.

Esse padrão é amplamente utilizado em bibliotecas e linguagens de programação. Em Python, por exemplo, estruturas como listas, tuplas, dicionários e conjuntos implementam iteradores, permitindo que sejam percorridas naturalmente com o comando `for`, tornando o código mais simples, desacoplado e reutilizável.

PADRÃO DE PROJETO: ITERATOR (ITERADOR)

1 O QUE FAZ? (O CONCEITO)



→ PRÓXIMO → PRÓXIMO → PRÓXIMO

Fornece uma maneira de acessar elementos **SEQUENCIALMENTE** de um objeto **AGREGADO**.

Sem expor sua **representação INTERNA**!

2 COMO FUNCIONA? (ENCAPSULAMENTO)



3 INTERFACE UNIFORME (PADRONIZAÇÃO)



[Listas]



[Arrays]



[Conjuntos]



[Mapas
(Chaves, Valores)]

Utiliza o **mesmo padrão** de acesso para **DIFERENTES** coleções!



ITERADOR
(Mecanismo Uniforme)

Listas, Arrays, Conjuntos, Mapas usam o **mesmo padrão de acesso**!

4 MÚLTIPLOS PERCURSOS INDEPENDENTES

Vários percursos **SIMULTÂNEOS** e **INDEPENDENTES** sobre os mesmos dados!

Cada percurso usa uma **instância DIFERENTE** do iterador.



Cada percurso usa uma **instância DIFERENTE** do iterador.

BENEFÍCIOS PRINCIPAIS:

- **Flexibilidade:** Aumenta a flexibilidade da aplicação.
- **Desacoplamento:** Mecanismo de navegação desacoplado da implementação da coleção.

Mediator

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Descrição Resumida: esse padrão define um objeto que encapsula a forma como um conjunto de objetos interagem, promovendo um fraco acoplamento ao evitar que objetos se refiram aos outros explicitamente.

O padrão de projeto Mediator é utilizado para centralizar a comunicação entre diversos objetos por meio de um componente intermediário. Em quais situações essa abordagem é vantajosa? Principalmente quando vários objetos precisam interagir entre si e a quantidade de dependências torna o sistema difícil de compreender, manter ou modificar. Ao concentrar as regras de interação em um mediador, **os participantes deixam de conhecer diretamente uns aos outros.**

Imagine uma aplicação composta por diversas classes que colaboram continuamente para executar uma funcionalidade. O que acontece quando cada classe passa a se comunicar diretamente com várias outras? A quantidade de dependências cresce, o código torna-se mais difícil de acompanhar e pequenas alterações podem exigir modificações em diferentes pontos da aplicação. Essa situação reduz o reúso dos componentes e aumenta o esforço de manutenção.

O Mediator resolve esse problema ao receber e coordenar as interações entre os objetos. Assim, quando um componente precisa solicitar uma operação, **ele envia a requisição ao mediador, que decide quais participantes deverão ser envolvidos.** Dessa forma, as classes permanecem desacopladas, pois deixam de manter referências diretas entre si, enquanto a lógica de comunicação fica concentrada em um único ponto.

Outra dúvida frequente é se o Mediator elimina completamente a complexidade da comunicação. **Isso não ocorre, pois essa complexidade é transferida para o objeto mediador.** Ainda assim, essa centralização costuma facilitar a manutenção da aplicação, já que as alterações nas regras de interação tendem a ser realizadas em um único componente, sem exigir mudanças simultâneas em diversas classes participantes.

```
class TorreControle:
    def enviar(self, aviao, mensagem):
        print(f"{aviao}: {mensagem}")

class Aviao:
    def __init__(self, nome, torre):
        self.nome = nome
        self.torre = torre

    def comunicar(self, mensagem):
        self.torre.enviar(self.nome, mensagem)

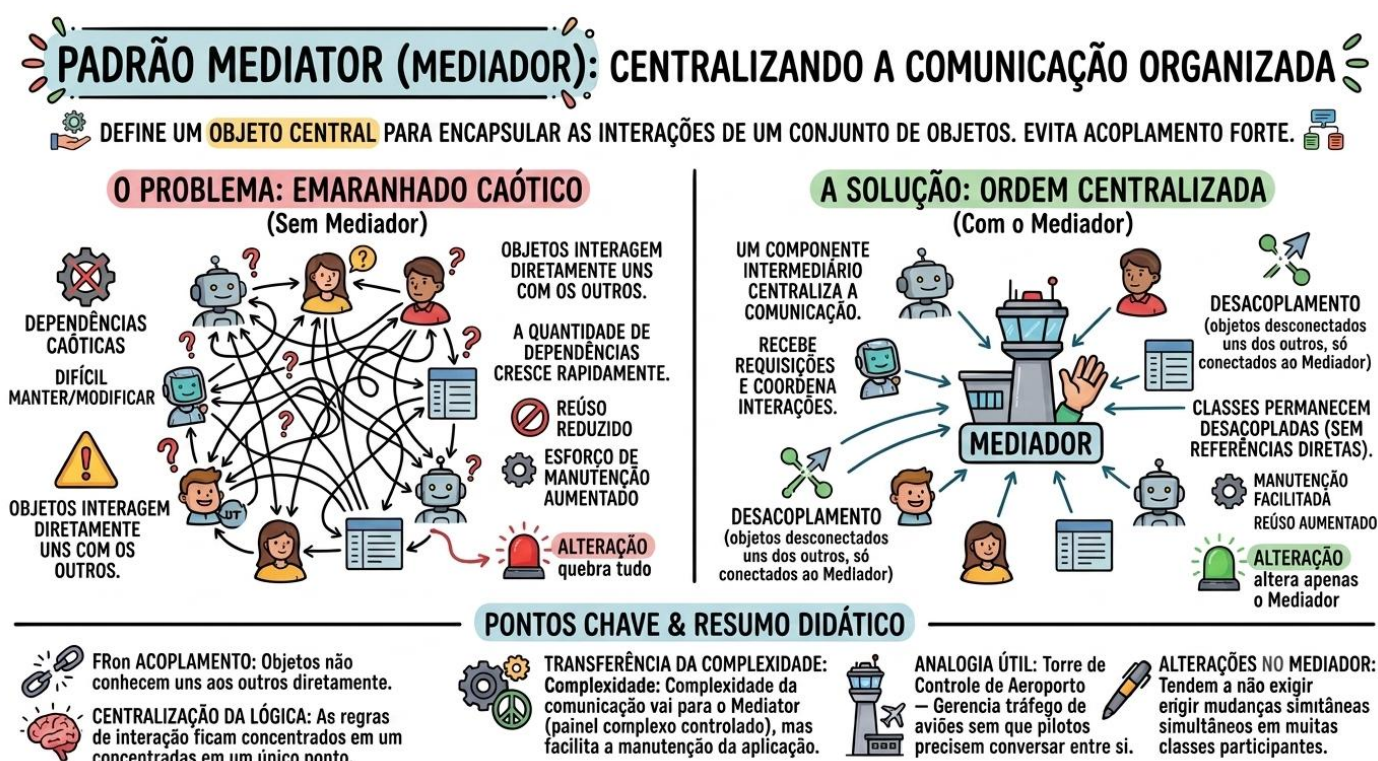
torre = TorreControle()

aviao1 = Aviao("Avião A", torre)
aviao2 = Aviao("Avião B", torre)

aviao1.comunicar("Solicitando autorização para pouso.")
aviao2.comunicar("Aguardando instruções.")
```

Nesse exemplo, a classe TorreControle representa o Mediator (mediador). Em vez de os aviões se comunicarem diretamente entre si, toda comunicação passa pela torre de controle. Cada objeto Aviao conhece apenas o mediador, e não precisa conhecer ou manter referências aos demais aviões. A principal ideia do Mediator é centralizar a comunicação entre vários objetos, reduzindo o acoplamento entre eles. Assim, em vez de cada objeto precisar conhecer e interagir diretamente com todos os outros, eles apenas enviam suas solicitações ao mediador, que decide como tratá-las e para quem encaminhá-las.

Esse padrão é útil quando há muitos objetos que precisam interagir entre si, pois evita uma grande quantidade de dependências diretas. Exemplos comuns incluem janelas de interfaces gráficas, chats, salas de videoconferência, torres de controle de aeroportos e sistemas de comunicação entre componentes de software.



(CESGRANRIO / BNDES - 2024) Um desenvolvedor percebeu que o padrão Façade não atendia os requisitos do projeto, já que era necessário um comportamento cooperativo e um protocolo multidirecional, além de evitar que os objetos interagissem de forma explícita uns com os outros. Considerando esse contexto, o desenvolvedor deve corrigir o sistema implementando o padrão

- a) Bridge
- b) Composite
- c) Decorator
- d) Mediator
- e) Strategy

Comentários: O Mediator é indicado quando se deseja comportamento cooperativo entre objetos, com comunicação centralizada e multidirecional, sem que os participantes se refiram explicitamente uns aos outros. Bridge separa abstração e implementação; Composite organiza hierarquias; Decorator adiciona responsabilidades; e Strategy encapsula algoritmos (Letra D).

Memento

INCIDÊNCIA EM PROVA: BAIXA

Descrição Resumida: esse padrão captura e externaliza o estado interno de um objeto, sem violar seu encapsulamento, de maneira que o objeto possa ser restaurado posteriormente.

O padrão de projeto Memento é utilizado para armazenar o estado interno de um objeto, permitindo que ele seja restaurado posteriormente sem comprometer seu encapsulamento. Como isso é possível? O estado é registrado em um objeto específico denominado memento, enquanto os detalhes internos permanecem protegidos e inacessíveis aos demais componentes da aplicação.

Um exemplo bastante conhecido é o funcionamento de um editor de texto. O que acontece quando o usuário deseja desfazer uma alteração? Antes que a modificação seja realizada, o estado atual do documento pode ser armazenado em um memento. Se o usuário solicitar o desfazer da operação, o documento recupera o estado previamente salvo, retornando exatamente à condição anterior sem que outros objetos precisem manipular sua estrutura interna.

Outra dúvida frequente é se o memento expõe os dados armazenados para qualquer componente da aplicação. Isso não ocorre, pois o padrão preserva o encapsulamento do objeto. O estado é armazenado no memento, mas **sua interpretação e restauração permanecem sob responsabilidade do próprio objeto que o originou, conhecido como Originator**. Um terceiro participante, **denominado Caretaker, normalmente é responsável apenas por armazenar e gerenciar os mementos**.

Além de implementar funcionalidades de desfazer e refazer operações, o Memento pode ser utilizado sempre que for necessário preservar versões anteriores do estado de um objeto para recuperação futura. Assim, o histórico de estados permanece disponível sem que seja necessário expor os detalhes internos do objeto ou permitir que outros componentes modifiquem diretamente suas informações.

```
class Documento:
    def __init__(self):
        self.texto = ""

    def salvar(self):
        return self.texto

    def restaurar(self, memento):
        self.texto = memento

documento = Documento()
documento.texto = "Versão 1"

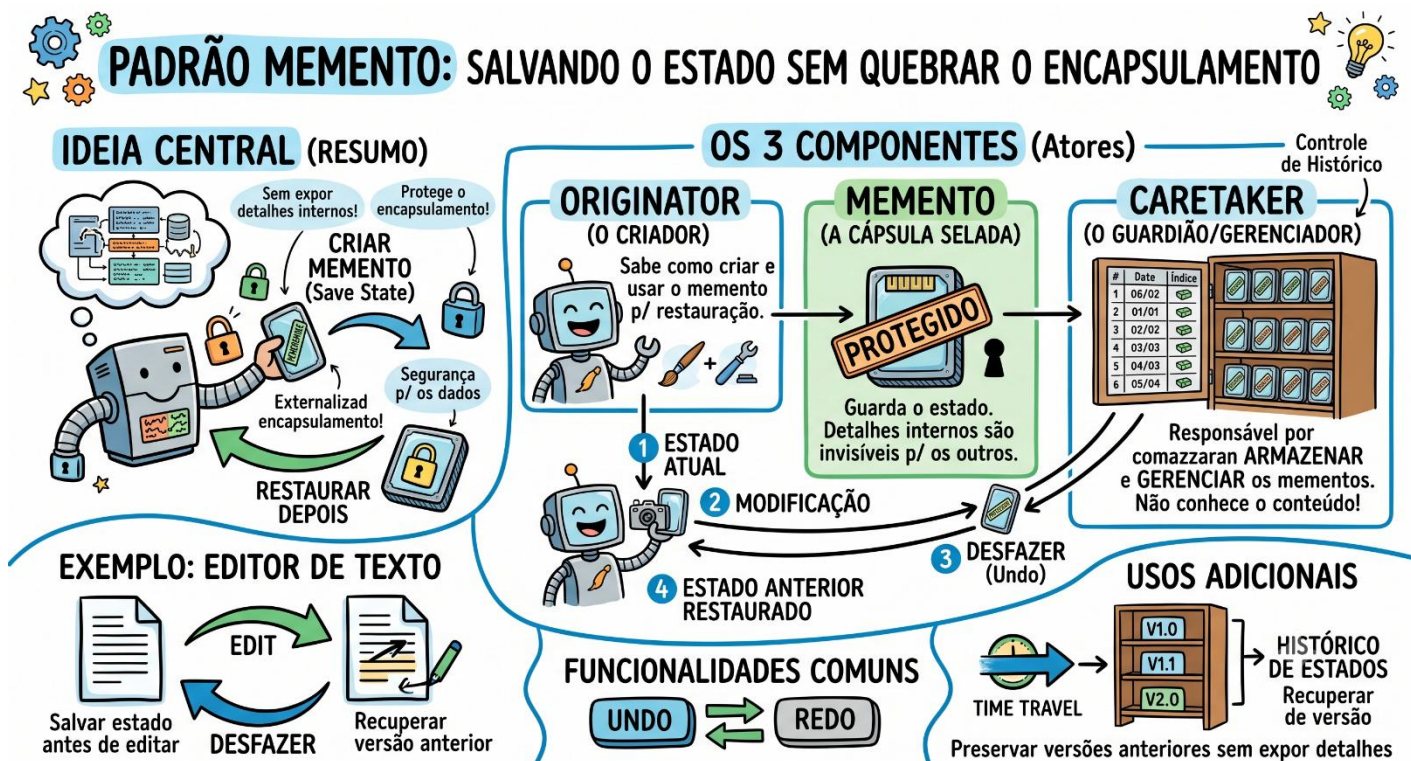
backup = documento.salvar()

documento.texto = "Versão 2"
documento.restaurar(backup)

print(documento.texto)
```

Nesse exemplo, a classe Documento representa o objeto cujo estado pode ser salvo e restaurado. O método salvar() cria um memento, que nada mais é do que uma cópia do estado atual do objeto. Posteriormente, o método restaurar() permite retornar exatamente ao estado salvo anteriormente.

A principal ideia do Memento é **capturar e armazenar o estado interno de um objeto sem expor sua estrutura interna**, possibilitando que esse estado seja restaurado futuramente. Assim, é possível desfazer alterações ou retornar a uma configuração anterior sem que outras partes do sistema precisem conhecer os detalhes de implementação do objeto. Esse padrão é muito utilizado em funcionalidades de desfazer/refazer, histórico de edições, pontos de restauração de jogos, snapshots de configurações e sistemas que precisam recuperar estados anteriores de forma simples e segura.



(FGV / Câmara dos Deputados - 2023) Assinale a opção que indica o padrão de projeto GoF que, com finalidade comportamental, sem violar o encapsulamento, captura e externaliza um estado interno de um objeto, de modo que o mesmo possa posteriormente ser restaurado para este estado.

- a) bridge.
- b) facade.
- c) flyweight.
- d) memento.
- e) visitor.

Comentários: o Memento é o padrão comportamental que permite capturar e externalizar o estado interno de um objeto para restaurá-lo depois, sem quebrar o encapsulamento. Bridge, Facade e Flyweight são estruturais, e o Visitor, embora comportamental, separa operações da estrutura dos objetos, sem armazenar estado (Letra D).

Observer

INCIDÊNCIA EM PROVA: MÉDIA

Descrição Resumida: esse padrão define uma dependência um-para-muitos entre objetos para que, quando um objeto mudar de estado, os seus dependentes sejam notificados e atualizados automaticamente.

O padrão de projeto Observer é utilizado quando vários objetos precisam ser informados automaticamente sobre alterações ocorridas em outro objeto. Como isso funciona? **O objeto que sofre as alterações, denominado Subject ou Observable, mantém uma lista de observadores registrados.** Sempre que seu estado é modificado, todos os observadores são notificados para que possam atualizar suas próprias informações.

Imagine uma tabela de classificação do Campeonato Brasileiro acompanhada por diferentes gráficos, como um gráfico de pizza com a quantidade de vitórias, empates e derrotas e outro que apresenta a evolução da posição de um time ao longo da competição. O que acontece quando os resultados da rodada são atualizados? Todas essas representações precisam refletir imediatamente os novos dados para permanecerem consistentes.

O Observer automatiza esse processo ao **estabelecer uma dependência do tipo um para muitos entre a tabela e os gráficos.** Assim, sempre que a tabela sofre uma alteração, todos os observadores registrados recebem uma notificação e executam sua atualização. Dessa forma, a tabela não precisa conhecer a implementação de cada gráfico, bastando interagir com a interface comum implementada pelos observadores.

Outra dúvida frequente é se a quantidade de observadores precisa ser conhecida antecipadamente. Isso não é necessário, pois **novos observadores podem ser registrados ou removidos durante a execução da aplicação.** Essa característica torna o padrão bastante flexível, já que diferentes componentes podem começar ou deixar de acompanhar as alterações do objeto observado sem modificar sua implementação.

```
class Canal:
    def __init__(self):
        self.inscritos = []

    def inscrever(self, usuario):
        self.inscritos.append(usuario)

    def publicar_video(self):
        for usuario in self.inscritos:
            usuario.notificar()

class Usuario:
    def __init__(self, nome):
        self.nome = nome

    def notificar(self):
        print(f"{self.nome} recebeu uma notificação.")

canal = Canal()

canal.inscrever(Usuario("Ana"))
canal.inscrever(Usuario("Carlos"))
canal.publicar_video()
```

Nesse exemplo, a classe Canal representa o objeto observado (Subject), enquanto a classe Usuario representa os observadores (Observers). Os usuários se inscrevem no canal por meio do método inscrever(). Quando o canal publica um novo vídeo, ele percorre sua lista de inscritos e notifica automaticamente cada um deles.

A principal ideia do Observer é **estabelecer uma relação de dependência do tipo um-para-muitos entre objetos**. Assim, quando o estado de um objeto é alterado, todos os objetos que dependem dele são automaticamente notificados, sem que o objeto observado precise conhecer detalhes sobre cada observador.

Esse padrão é muito utilizado em sistemas de notificações, eventos de interfaces gráficas, atualização de painéis em tempo real, publicação de notícias, redes sociais e aplicações baseadas em eventos. Em todos esses casos, diversos objetos precisam ser avisados automaticamente quando ocorre uma determinada mudança de estado.



(FCC / TRT 15ª Região - 2025) Um Tribunal Regional do Trabalho precisa implementar um sistema de notificação para diferentes eventos processuais (novos documentos apresentados, audiências agendadas, decisões publicadas). O sistema deve ser flexível, permitindo adicionar novos tipos de notificações sem alterar significativamente o código existente. O padrão de projeto mais adequado para alcançar essa flexibilidade é o

- Factory Method.
- Decorator.
- Singleton.
- Observer.
- Strategy.

Comentários: O Observer é ideal quando vários elementos precisam ser avisados automaticamente sobre eventos, facilitando incluir novos tipos de notificação sem grandes mudanças no código. Factory Method cria objetos; Decorator adiciona responsabilidades; Singleton garante instância única; e Strategy trata da troca de algoritmos (Letra D).

State

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Descrição Resumida: esse padrão permite a um objeto alterar o seu comportamento quando o seu estado interno for modificado.

O padrão de projeto State é utilizado quando o comportamento de um objeto depende de seu estado interno e precisa mudar dinamicamente durante a execução da aplicação. Como isso acontece? Em vez de concentrar todas as regras em grandes estruturas condicionais, **cada estado é representado por uma classe específica que implementa o comportamento correspondente**. Assim, o objeto altera suas ações simplesmente ao mudar de estado.

Um exemplo conhecido pode ser observado nos jogos da série Super Mario. O que ocorre quando o personagem obtém um item especial? O resultado depende do estado em que ele se encontra naquele momento. Um personagem pequeno pode passar para um estado diferente de um personagem que já possui um poder especial, enquanto outras situações podem produzir efeitos distintos conforme as regras definidas pelo jogo.

Essa característica demonstra que **a próxima ação não depende apenas do evento recebido, mas também do estado atual do objeto**. Por que isso é importante? Porque a decisão sobre qual comportamento executar ocorre durante a execução da aplicação. Dessa forma, cada estado encapsula sua própria lógica e pode definir tanto as operações disponíveis quanto as transições para outros estados.

Outra dúvida frequente é se o State elimina completamente as estruturas condicionais. Isso não ocorre em todos os casos, mas **o padrão reduz significativamente a necessidade de grandes blocos de decisão centralizados, substituindo-os por polimorfismo**. Como consequência, novos estados e novas transições podem ser adicionados com menor impacto sobre o código existente, favorecendo a manutenção e a evolução da aplicação.

```
class Semaforo:
    def __init__(self):
        self.estado = "Verde"

    def proximo(self):
        if self.estado == "Verde":
            self.estado = "Amarelo"
        elif self.estado == "Amarelo":
            self.estado = "Vermelho"
        else:
            self.estado = "Verde"

        print(f"Semaforo: {self.estado}")

semaforo = Semaforo()

semaforo.proximo()
semaforo.proximo()
semaforo.proximo()
```

Nesse exemplo, a classe Semaforo possui um atributo chamado estado, que indica sua condição atual. Sempre que o método proximo() é chamado, o estado é alterado de acordo com regras pré-definidas: de Verde para Amarelo, de Amarelo para Vermelho e, por fim, de Vermelho para Verde. A principal ideia do State é permitir que um objeto altere seu comportamento quando seu estado interno muda. Em vez

de o cliente precisar decidir qual ação executar para cada situação, o próprio objeto controla suas transições de estado e adapta seu comportamento automaticamente.

Esse padrão é útil quando um objeto pode assumir diferentes estados ao longo de sua vida, como um semáforo, uma máquina de vendas, um pedido de compra (novo, pago, enviado e entregue) ou um reprodutor de mídia (tocando, pausado e parado). Em implementações completas do padrão GoF, **cada estado costuma ser representado por uma classe distinta**, tornando o código mais organizado e facilitando a inclusão de novos estados sem modificar a lógica existente.

PADRÃO STATE: COMPORTAMENTO ADAPTÁVEL



DEFINIÇÃO RESUMIDA: OBJETO ALTERA COMPORTAMENTO QUANDO SEU ESTADO INTERNO MUDA.



POR QUE USAR?

PROBLEMA: CAOS DE CONDICIONAIS

```
if ... else if  
if ... else if  
else  
}  
if ... else if  
if ... else if  
else  
if ... else if  
if ... else if  
else  
else
```

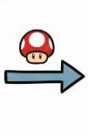
COMPORTAMENTO
DEPENDE DO
ESTADO.

Sem o State, temos GRANDES
ESTRUTURAS CONDICIONAIS.
DIFÍCIL DE MANTER E EVOLUIR.



EXEMPLO CLÁSSICO: SUPER MARIO

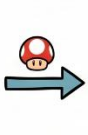
ESTADO
PEQUENO



REAGE: PASSA PARA
ESTADO GRANDE



Ação: QUEBRA BLOCOS



REAGE: PASSA PARA
ESTADO FIRE



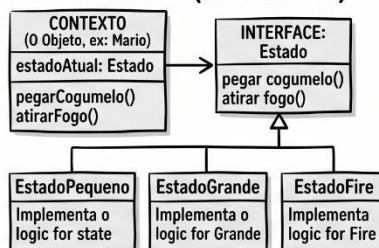
Ação: ATIRA BOLAS
DE FOGO

O resultado de um ITEM depende do ESTADO atual.



COMO FUNCIONA:

ESTRUTURA (Polimorfismo)



ESTADOS representados por
CLASSES ESPECÍFICAS.
Cada classe implementa o comportamento
correspondente.

PONTOS CHAVE E RESUMO



Ação depende de EVENTO
+ ESTADO ATUAL.
Lógica encapsulada e
descentralizada.



Novos estados/transições
são fáceis de adicionar.
MENOR IMPACTO no
código existente.



Favorece MANUTENÇÃO
e EVOLUÇÃO.
Polimorfismo reduz grandes
blocos de decisão centralizados.



Decisão de comportamento
ocorre durante a EXECUÇÃO.

Strategy

INCIDÊNCIA EM PROVA: BAIXA

Descrição Resumida: esse padrão define uma família de algoritmos, encapsula cada um e faz deles intercambiáveis.

O padrão de projeto Strategy é utilizado quando um problema pode ser resolvido por diferentes algoritmos e a aplicação deve ser capaz de escolher qual deles utilizar durante a execução. O que caracteriza esse padrão? **Cada algoritmo é encapsulado em uma classe independente que implementa uma interface comum, permitindo que todos sejam utilizados de forma intercambiável** sem modificar o código que os utiliza.

Considere um sistema escolar que precisa ordenar uma coleção de alunos por idade. Seria necessário implementar todos os algoritmos de ordenação em uma única classe? Não, pois existem diferentes algoritmos, como Bubble Sort, Quick Sort, Heap Sort e Merge Sort, capazes de realizar essa mesma tarefa. Cada um pode ser implementado como uma estratégia distinta, mantendo separada a lógica de cada algoritmo.

Outra dúvida frequente é como a aplicação escolhe o algoritmo mais adequado. **O objeto responsável pela ordenação recebe uma estratégia como parâmetro e utiliza sua implementação para executar a operação.** Dessa forma, o mesmo código pode ordenar a coleção empregando diferentes algoritmos, bastando substituir a estratégia utilizada, sem alterar a lógica principal da aplicação.

Essa abordagem também **reduz a necessidade de grandes estruturas condicionais utilizadas apenas para selecionar um algoritmo específico.** Em vez de concentrar todas as implementações em uma única classe, cada estratégia permanece isolada e pode ser adicionada, removida ou substituída de forma independente, o que facilita a manutenção e a evolução do sistema.

```
class PagamentoCartao:
    def pagar(self, valor):
        print(f"Pagamento de R${valor} no cartão.")

class PagamentoPix:
    def pagar(self, valor):
        print(f"Pagamento de R${valor} via PIX.")

class Loja:
    def __init__(self, estrategia):
        self.estrategia = estrategia

    def finalizar_compra(self, valor):
        self.estrategia.pagar(valor)

loja = Loja(PagamentoPix())
loja.finalizar_compra(100)
```

Nesse exemplo, as classes PagamentoCartao e PagamentoPix representam diferentes estratégias para realizar um pagamento. A classe Loja não implementa a lógica de pagamento; ela apenas recebe uma estratégia e delega a ela a responsabilidade de efetuar o pagamento. A principal ideia do Strategy é encapsular diferentes algoritmos ou formas de executar uma mesma tarefa em classes distintas, permitindo que eles sejam substituídos em tempo de execução. Assim, o objeto cliente pode alterar seu comportamento simplesmente trocando a estratégia utilizada, sem modificar seu próprio código.

Esse padrão é útil quando existem várias maneiras de realizar uma mesma operação, como diferentes formas de pagamento, algoritmos de ordenação, estratégias de compactação, cálculo de frete ou autenticação. Em vez de utilizar grandes estruturas de decisão (if ou switch), cada comportamento é implementado em uma estratégia independente, tornando o código mais flexível, organizado e fácil de estender.



(FGV / TJ SE - 2023) Existem vários algoritmos, conhecidos e disponíveis, capazes de implementar a funcionalidade C, e a aplicação deve deixar o usuário escolher qual algoritmo usar. Maria decidiu aplicar o padrão GoF comportamental que modela algoritmos similares como objetos independentes, porém passíveis de troca em tempo de execução. Maria decidiu aplicar o padrão GoF:

- a) Visitor;
- b) Strategy;
- c) Mediator;
- d) Interpreter;
- e) Template Method.

Comentários: o Strategy encapsula algoritmos em objetos independentes e permite trocá-los em tempo de execução, exatamente como descrito. Visitor adiciona operações a estruturas de objetos; Mediator centraliza comunicação; Interpreter interpreta gramáticas; e Template Method fixa o esqueleto de um algoritmo na superclasse (Letra B).

Template Method

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Descrição Resumida: esse padrão define o esqueleto de um algoritmo dentro de uma operação, deixando alguns passos a serem preenchidos pelas subclasses.

O padrão de projeto Template Method é utilizado quando diferentes implementações compartilham a mesma sequência geral de um algoritmo, mas algumas etapas precisam variar. Como isso é organizado?

A superclasse define o método que estabelece toda a ordem de execução do algoritmo, enquanto delega às subclasses apenas a implementação das operações que podem apresentar comportamentos diferentes.

Considere o processo de preparação de um sanduíche em uma franquia de restaurantes. O que permanece igual em qualquer pedido? Etapas como abrir o pão, posicionar o hambúrguer e fechar o sanduíche fazem parte da sequência básica de preparo e são executadas sempre na mesma ordem. Em contrapartida, ingredientes como queijo, picles, molhos e outros complementos podem variar conforme a receita ou o tipo de sanduíche preparado.

Essa organização permite separar claramente o que é fixo do que é variável no algoritmo. Por que isso é vantajoso? Porque **a lógica comum permanece concentrada na superclasse, evitando duplicação de código**, enquanto cada subclasses implementa apenas as etapas específicas pelas quais é responsável. Dessa forma, alterações nas partes variáveis não exigem mudanças na estrutura principal do algoritmo.

Outra dúvida frequente é se as subclasses podem modificar completamente a sequência das etapas. Normalmente isso não ocorre, pois **a ordem de execução é controlada pelo método definido na superclasse**. As subclasses apenas implementam ou especializam determinadas operações previstas nesse fluxo, preservando a estrutura geral do algoritmo e garantindo um comportamento consistente entre todas as implementações.

```
class Bebida:
    def preparar(self):
        self.ferver_agua()
        self.adicionar_ingredientes()
        self.servir()

    def ferver_agua(self):
        print("Fervendo água...")

    def adicionar_ingredientes(self):
        pass

    def servir(self):
        print("Servindo a bebida.")

class Cafe(Bebida):
    def adicionar_ingredientes(self):
        print("Adicionando café.")

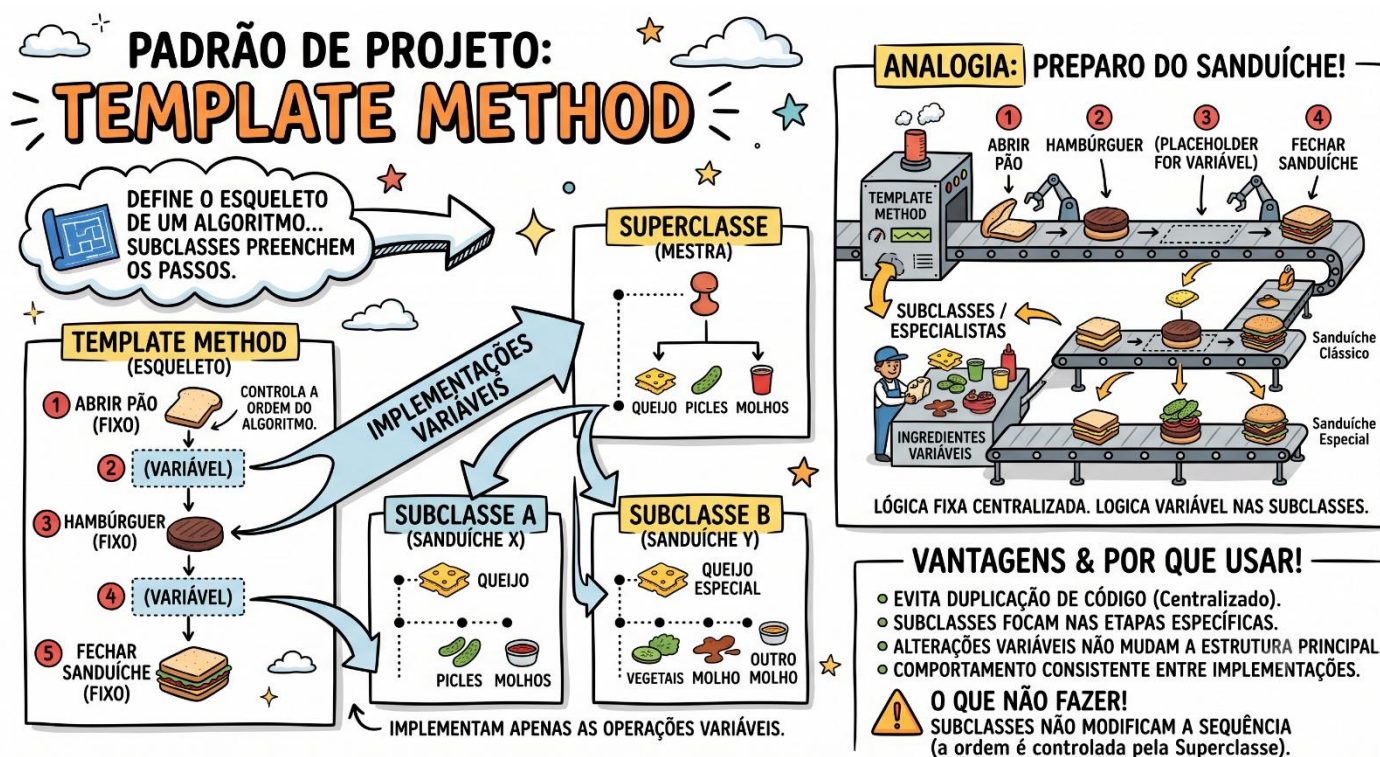
cafe = Cafe()
cafe.preparar()
```

Nesse exemplo, a classe Bebida define o método preparar(), que estabelece a sequência de etapas para preparar uma bebida: ferver a água, adicionar o ingrediente principal e servir. Entretanto, a etapa

adicionar_ingrediente() não é implementada na classe base, ficando a cargo das subclasses definir seu comportamento.

A principal ideia do Template Method é definir o esqueleto de um algoritmo em uma classe base, deixando que algumas etapas sejam implementadas ou personalizadas pelas subclasses. Dessa forma, a estrutura geral do algoritmo permanece fixa, enquanto partes específicas podem variar conforme a necessidade.

Esse padrão é útil quando diferentes objetos executam processos muito semelhantes, mas apresentam pequenas variações em determinadas etapas. Exemplos comuns incluem o preparo de bebidas (café, chá e chocolate quente), rotinas de autenticação, geração de relatórios e processos de importação de dados, nos quais a sequência de passos é sempre a mesma, mas algumas etapas possuem implementações diferentes.



(CESGRANRIO / BANESE - 2025) M percebeu a necessidade de desenvolver um esqueleto de um algoritmo, composto por um conjunto de passos na sua superclasse, sendo que a definição de alguns desses passos deverá ser postergada para as subclasses. A ordem em que os passos serão executados, definida na superclasse, não poderá ser mudada. Nesse caso, o padrão de projeto utilizado por M e sua equipe é o

- a) Façade
- b) Observer
- c) Abstract Factory
- d) Factory Method
- e) Template Method

Comentários: o Template Method define na superclasse o esqueleto do algoritmo, com a ordem dos passos já estabelecida, deixando que as subclasses implementem ou redefinam apenas alguns desses passos sem alterar a estrutura geral. Façade simplifica subsistemas; Observer notifica interessados; e Abstract Factory e Factory Method são criacionais (Letra E).

Visitor

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

Descrição Resumida: esse padrão representa uma operação a ser realizada sobre elementos de uma estrutura de objetos e permite definir uma operação sem mudar as classes dos elementos sobre os quais opera.

O padrão de projeto Visitor é utilizado quando diversas operações precisam ser executadas sobre uma estrutura de objetos sem que seja necessário modificar as classes que representam seus elementos. Como isso é possível? **As operações são implementadas em objetos visitantes, que percorrem a estrutura e executam o comportamento adequado para cada tipo de elemento encontrado.** Dessa forma, novas funcionalidades podem ser adicionadas sem alterar a implementação dos objetos visitados.

Considere um carrinho de compras de um supermercado contendo produtos industrializados, frutas e verduras. O que acontece quando a compra é finalizada? Um visitante percorre todos os itens do carrinho para calcular o valor da compra. Durante esse percurso, cada produto recebe o visitante e direciona a execução para a operação correspondente ao seu próprio tipo, permitindo que o cálculo seja realizado de acordo com suas características.

Essa abordagem permite que diferentes tipos de produtos executem operações específicas sem concentrar toda a lógica em suas próprias classes. Por exemplo, um produto industrializado pode fornecer diretamente seu preço, enquanto frutas e verduras exigem uma etapa adicional de pesagem antes do cálculo do valor. Embora o processamento seja diferente, todos os elementos continuam fazendo parte da mesma estrutura de objetos.

Outra dúvida frequente é quando o Visitor deve ser adotado. **Esse padrão é mais indicado quando a estrutura de objetos permanece relativamente estável e novas operações precisam ser adicionadas com frequência. Se novos tipos de elementos forem criados constantemente, cada visitante deverá ser atualizado** para contemplá-los, o que pode reduzir as vantagens oferecidas por esse padrão.

```
class Livro:
    def aceitar(self, visitor):
        visitor.visitar_livro()

class Filme:
    def aceitar(self, visitor):
        visitor.visitar_filme()

class Relatorio:
    def visitar_livro(self):
        print("Gerando relatório do livro.")

    def visitar_filme(self):
        print("Gerando relatório do filme.")

visitor = Relatorio()

Livro().aceitar(visitor)
Filme().aceitar(visitor)
```

Nesse exemplo, as classes Livro e Filme representam os elementos que serão visitados. Cada uma possui o método aceitar(), que recebe um objeto visitante (Relatorio) e delega a ele a execução da operação correspondente. Assim, a lógica para gerar o relatório não fica dentro das classes Livro e Filme, mas sim na classe visitante.

A principal ideia do Visitor é **separar os dados das operações realizadas sobre eles**. Em vez de adicionar um novo método em todas as classes sempre que surgir uma nova funcionalidade, cria-se um novo Visitor contendo essa operação. Dessa forma, é possível adicionar novos comportamentos sem modificar as classes dos objetos visitados.

Esse padrão é útil quando uma estrutura de objetos permanece relativamente estável, mas novas operações precisam ser adicionadas com frequência. Exemplos comuns incluem geração de relatórios, cálculo de impostos, exportação para diferentes formatos (PDF, XML ou JSON), validação de estruturas e análise de árvores sintáticas em compiladores.



Por fim, algumas dicas para "matar" questões rapidamente...

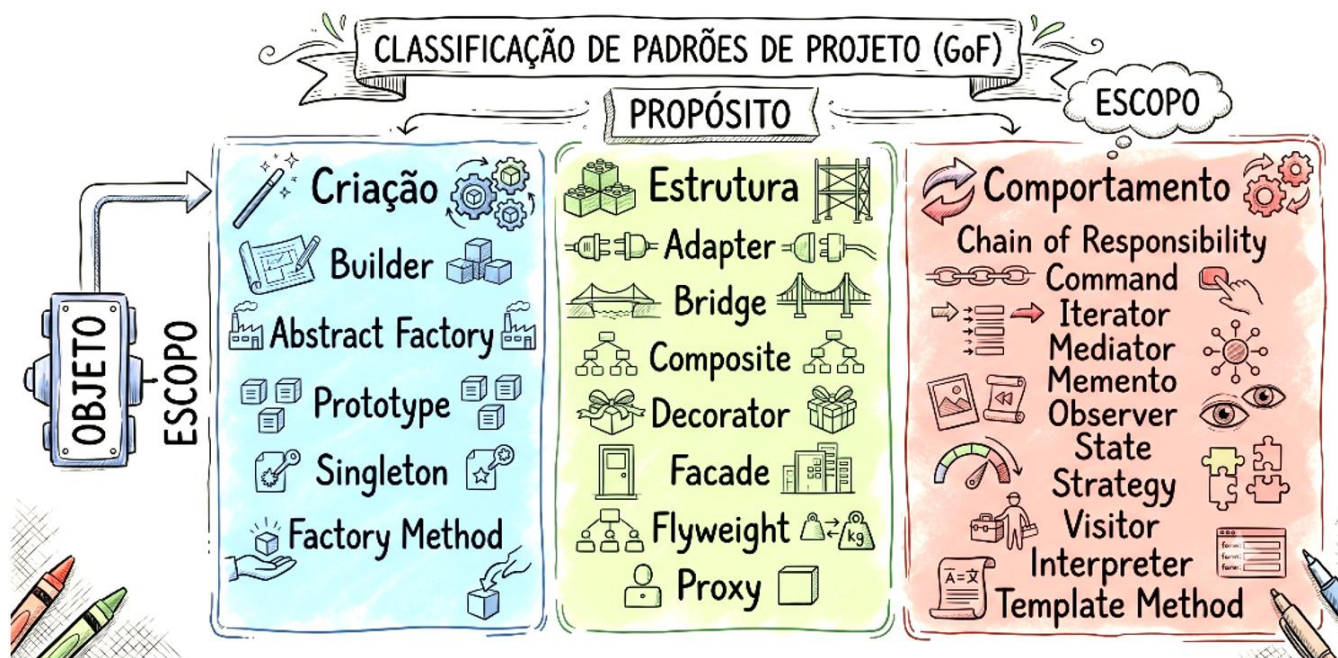
Falou em...	... provavelmente será:
Criar famílias de objetos relacionados...	Abstract factory
Construção de um objeto complexo com diferentes representações...	Builder
Deixar subclasses decidirem...	Factory Method
Criar uma instância prototípica...	Prototype
Apenas uma instância com um ponto global a ela...	Singleton
Converte uma interface em outra, por serem incompatíveis...	Adapter
Desacoplar interface da implementação...	Bridge
Estruturas de árvore em hierarquia parte-todo...	Composite
Anexa responsabilidades adicionais dinamicamente...	Decorator
Interface unificada de alto nível para simplificar outra complexa...	Façade
Compartilhamento para suportar grandes quantidades de objetos...	Flyweight
Prover substituto para controlar um objeto...	Proxy
Evitar o acoplamento dando oportunidade a outros objetos...	Chain of Responsibility
Encapsula requisição de objetos...	Command

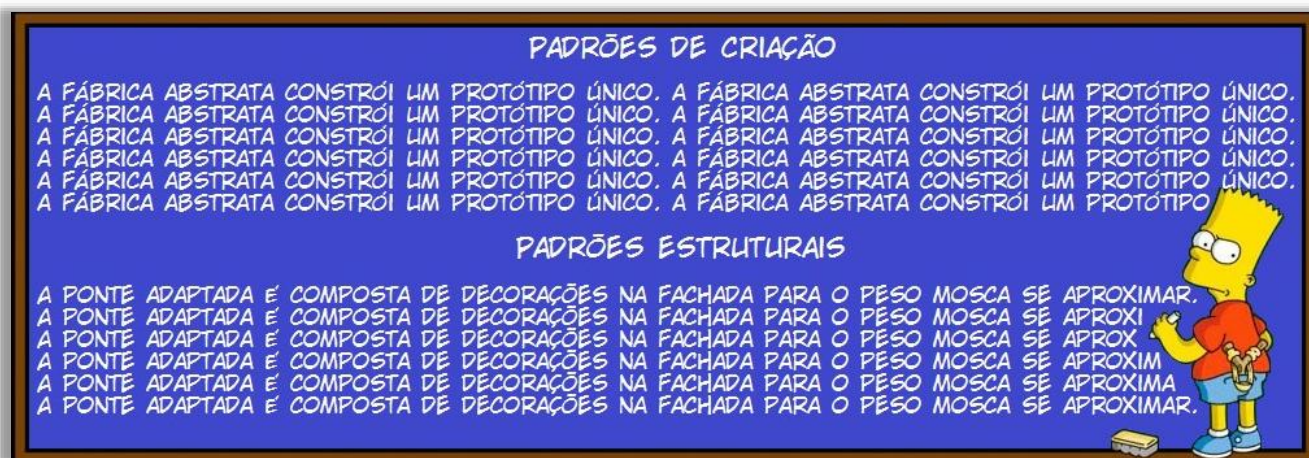
Representação de uma gramática...	Interpreter
Interface única para acessar coleções sequencialmente...	Iterator
Encapsula a forma como objetos interagem...	Mediator
Captura o estado interno de um objeto...	Memento
Quando objeto mudar de estado, notifica os dependentes...	Observer
Altera comportamentos quando modificar o estado interno...	State
Família de algoritmos...	Strategy
Esqueleto de algoritmos...	Template Method
Operação a ser realizada sobre uma estrutura de objetos...	Visitor

RESUMO

PADRÕES DE PROJETO GOF

Os Padrões de Projeto GoF (Gang of Four) são um conjunto de 23 soluções reutilizáveis para problemas recorrentes no desenvolvimento de software orientado a objetos, apresentados no livro Design Patterns: Elements of Reusable Object-Oriented Software (1994), de Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides. Esses padrões não representam implementações prontas, mas sim modelos de solução que orientam a organização das classes, objetos e suas interações, promovendo maior reutilização, flexibilidade, baixo acoplamento e facilidade de manutenção dos sistemas.





PADRÕES CRIACIONAIS

Os Padrões Criacionais são uma categoria dos padrões GoF voltada para abstrair e controlar o processo de criação de objetos. Seu principal objetivo é desacoplar o código cliente da forma como os objetos são instanciados, tornando o sistema mais flexível e facilitando a substituição ou extensão das classes concretas. Fazem parte dessa categoria os padrões Abstract Factory, Builder, Factory Method, Prototype e Singleton.

ABSTRACT FACTORY

O Abstract Factory fornece uma interface para criar famílias de objetos relacionados ou dependentes sem especificar suas classes concretas. Seu objetivo é garantir que objetos pertencentes à mesma família sejam criados de forma consistente, desacoplando o código cliente das implementações específicas. Esse padrão é útil quando o sistema precisa trabalhar com diferentes famílias de produtos que devem ser utilizadas em conjunto, como interfaces gráficas para diferentes sistemas operacionais.

BUILDER

O Builder separa a construção de um objeto complexo de sua representação, permitindo que o mesmo processo de construção gere diferentes representações do objeto. Em vez de criar um objeto completamente em um único passo, ele é montado gradualmente por meio de etapas bem definidas. Esse padrão é indicado quando um objeto possui muitas partes ou diversas configurações possíveis, tornando sua construção mais organizada e flexível.

FACTORY METHOD

O Factory Method define uma interface para criar objetos, permitindo que subclasses decidam qual classe concreta será instanciada. Dessa forma, o processo de criação dos objetos é encapsulado, reduzindo o acoplamento entre o código cliente e as classes concretas. Esse padrão é útil quando uma aplicação deve criar diferentes tipos de objetos sem que o cliente conheça previamente suas implementações.

PROTOTYPE

O Prototype cria novos objetos por meio da clonagem de um objeto existente, em vez de instanciá-los diretamente. Esse padrão é especialmente útil quando a criação de um objeto é custosa, complexa ou envolve muitas configurações, permitindo que um objeto previamente configurado seja copiado e adaptado conforme necessário.

SINGLETON

O Singleton garante que uma classe possua apenas uma única instância durante toda a execução da aplicação, fornecendo um ponto único de acesso a ela. Esse padrão é utilizado quando deve existir apenas um objeto

responsável por determinada funcionalidade, como um gerenciador de configuração, um sistema de logs ou um controlador de acesso compartilhado.

PADRÕES ESTRUTURAIS

Os Padrões Estruturais são uma categoria dos padrões GoF que se concentram na forma como classes e objetos são organizados para formar estruturas maiores e mais flexíveis. Seu principal objetivo é facilitar a composição de classes e objetos, promovendo baixo acoplamento, reutilização de código e maior facilidade de manutenção. Esses padrões definem maneiras eficientes de estabelecer relacionamentos entre componentes, permitindo que sistemas complexos sejam construídos de forma modular. Fazem parte dessa categoria os padrões Adapter, Bridge, Composite, Decorator, Facade, Flyweight e Proxy.

ADAPTER

O Adapter converte a interface de uma classe em outra interface esperada pelo cliente, permitindo que classes incompatíveis trabalhem juntas sem modificar suas implementações. Esse padrão é muito utilizado na integração entre sistemas, bibliotecas ou componentes desenvolvidos de forma independente.

BRIDGE

O Bridge desacopla uma abstração de sua implementação, permitindo que ambas evoluam de forma independente. Em vez de criar diversas subclasses para combinar diferentes abstrações e implementações, o padrão estabelece uma composição entre elas, tornando o sistema mais flexível e reduzindo a explosão de subclasses.

COMPOSITE

O Composite organiza objetos em estruturas hierárquicas do tipo árvore, permitindo que objetos individuais e grupos de objetos sejam tratados de maneira uniforme. Esse padrão é especialmente útil para representar estruturas compostas, como sistemas de arquivos, organogramas e menus hierárquicos.

DECORATOR

O Decorator adiciona novas responsabilidades ou funcionalidades a um objeto de forma dinâmica, sem alterar sua classe original. Isso é feito envolvendo o objeto em outros objetos decoradores, que ampliam seu comportamento. Esse padrão oferece uma alternativa flexível à herança para estender funcionalidades.

FAÇADE

O Façade fornece uma interface simplificada para um conjunto complexo de classes ou subsistemas. Seu objetivo é ocultar a complexidade interna do sistema, oferecendo ao cliente um ponto único e simples de acesso às funcionalidades mais utilizadas, reduzindo o acoplamento entre o cliente e o subsistema.

FLYWEIGHT

O Flyweight utiliza o compartilhamento de objetos para reduzir o consumo de memória quando uma aplicação precisa manipular um grande número de objetos semelhantes. Em vez de criar diversas cópias de informações repetidas, os dados comuns são compartilhados entre os objetos, aumentando a eficiência do sistema.

PROXY

O Proxy fornece um objeto substituto que controla o acesso a outro objeto. Esse intermediário pode realizar tarefas adicionais, como controle de acesso, carregamento sob demanda, cache, registro de operações ou comunicação remota, mantendo transparente para o cliente o acesso ao objeto real.

PADRÕES COMPORTAMENTAIS

Os Padrões Comportamentais são uma categoria dos padrões GoF que se concentram na comunicação, na colaboração e na distribuição de responsabilidades entre objetos durante a execução do sistema. Seu objetivo é definir formas eficientes de interação entre objetos, reduzindo o acoplamento e tornando o comportamento do

software mais flexível e fácil de manter. Fazem parte dessa categoria os padrões Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method e Visitor.

CHAIN OF RESPONSIBILITY

O Chain of Responsibility organiza objetos em uma cadeia de processamento, permitindo que uma solicitação percorra essa cadeia até que algum objeto seja capaz de tratá-la. Dessa forma, o remetente da solicitação não precisa conhecer qual objeto será responsável pelo processamento, reduzindo o acoplamento entre quem solicita e quem executa a ação.

COMMAND

O Command encapsula uma solicitação em um objeto, transformando operações em objetos independentes. Isso permite armazenar, agendar, desfazer, refazer ou transmitir comandos sem que o cliente conheça os detalhes de sua execução. É amplamente utilizado em interfaces gráficas, filas de tarefas e mecanismos de undo/redo.

INTERPRETER

O Interpreter representa elementos de uma linguagem por meio de classes capazes de interpretar suas regras gramaticais. Cada expressão da linguagem é modelada como um objeto responsável por sua própria interpretação, tornando esse padrão adequado para linguagens simples, expressões matemáticas, consultas e regras de negócio.

ITERATOR

O Iterator fornece um mecanismo para percorrer sequencialmente os elementos de uma coleção sem expor sua estrutura interna. Dessa forma, listas, árvores, conjuntos e outras estruturas podem ser percorridos por meio de uma interface comum, promovendo encapsulamento e desacoplamento.

MEDIATOR

O Mediator centraliza a comunicação entre diversos objetos, evitando que eles se comuniquem diretamente entre si. Em vez de manter múltiplas dependências, cada objeto interage apenas com o mediador, que coordena as mensagens e ações, reduzindo significativamente o acoplamento do sistema.

MEMENTO

O Memento captura e armazena o estado interno de um objeto, permitindo que ele seja restaurado posteriormente sem violar seu encapsulamento. Esse padrão é muito utilizado em funcionalidades de desfazer/refazer, histórico de alterações, pontos de restauração (checkpoints) e salvamento de jogs.

OBSERVER

O Observer estabelece uma relação de dependência do tipo um-para-muitos entre objetos. Quando o estado de um objeto é alterado, todos os objetos interessados são automaticamente notificados e atualizados. Esse padrão é amplamente empregado em sistemas de eventos, notificações, interfaces gráficas e aplicações reativas.

STATE

O State permite que um objeto altere seu comportamento quando seu estado interno muda. Em vez de utilizar grandes estruturas condicionais, cada estado pode encapsular seu próprio comportamento, tornando o código mais organizado, flexível e fácil de manter à medida que novos estados são adicionados.

STRATEGY

O Strategy define uma família de algoritmos, encapsulando cada um deles em uma classe independente para que possam ser substituídos durante a execução do programa. Esse padrão permite alterar o comportamento de um objeto sem modificar sua implementação, sendo muito utilizado para diferentes formas de pagamento, algoritmos de ordenação e cálculo de fretes.

TEMPLATE METHOD

O Template Method define o esqueleto de um algoritmo em uma classe base, permitindo que determinadas etapas sejam implementadas ou personalizadas pelas subclasses. Assim, a estrutura geral do algoritmo permanece inalterada, enquanto partes específicas podem variar conforme a necessidade da aplicação.

VISITOR

O Visitor permite adicionar novas operações a uma estrutura de objetos sem modificar suas classes. Para isso, as operações são encapsuladas em um objeto visitante, que percorre os elementos da estrutura executando a ação apropriada em cada um deles. Esse padrão é bastante utilizado em compiladores, geração de relatórios, validações e exportação de dados.

Falou em...	... provavelmente será:
Criar famílias de objetos relacionados...	Abstract factory
Construção de um objeto complexo com diferentes representações...	Builder
Deixar subclasses decidirem...	Factory Method
Criar uma instância prototípica...	Prototype
Apenas uma instância com um ponto global a ela...	Singleton
Converte uma interface em outra, por serem incompatíveis...	Adapter
Desacoplar interface da implementação...	Bridge
Estruturas de árvore em hierarquia parte-todo...	Composite
Anexa responsabilidades adicionais dinamicamente...	Decorator
Interface unificada de alto nível para simplificar outra complexa...	Façade
Compartilhamento para suportar grandes quantidades de objetos...	Flyweight
Prover substituto para controlar um objeto...	Proxy
Evitar o acoplamento dando oportunidade a outros objetos...	Chain of Responsibility
Encapsula requisição de objetos...	Command
Representação de uma gramática...	Interpreter
Interface única para acessar coleções sequencialmente...	Iterator
Encapsula a forma como objetos interagem...	Mediator
Captura o estado interno de um objeto...	Memento
Quando objeto mudar de estado, notifica os dependentes...	Observer
Altera comportamentos quando modificar o estado interno...	State
Família de algoritmos...	Strategy
Esqueleto de algoritmos...	Template Method
Operação a ser realizada sobre uma estrutura de objetos...	Visitor

QUESTÕES COMENTADAS

1. (FGV / AMAZUL - 2026) Em programação orientada a objetos, um desenvolvedor precisa garantir que apenas uma instância de uma classe de conexão com banco de dados seja criada durante toda a execução da aplicação, evitando múltiplas conexões desnecessárias. O padrão de projeto que resolve esse problema é o

- a) Factory Method.
- b) Observer.
- c) Singleton.
- d) Strategy.
- e) Decorator.

Comentários:

(a) Errado. Factory Method trata da criação de objetos por meio de subclasses, mas não tem como foco garantir uma única instância durante toda a execução.

(b) Errado. Observer é usado para notificar objetos sobre mudanças de estado, não para controlar quantidade de instâncias.

(c) Correto. Singleton é o padrão pensado justamente para permitir que exista apenas uma instância de uma classe, com acesso global controlado.

(d) Errado. Strategy serve para encapsular algoritmos e trocar comportamentos em tempo de execução, não para restringir instâncias.

(e) Errado. Decorator adiciona responsabilidades a objetos de forma dinâmica, sem relação com limitar a criação a uma única instância.

Gabarito: Letra C

2. (FGV / AMAZUL - 2026) Uma equipe de desenvolvimento identificou que o código de um módulo de autenticação está sendo duplicado em vários pontos do sistema, dificultando a manutenção. A equipe decide refatorar o código aplicando um padrão que centralize a criação de objetos de autenticação, permitindo que subclasses decidam qual tipo específico de autenticador instanciar (OAuth, LDAP, ou local). O padrão de projeto que atende a essa necessidade é chamado

- a) Abstract Factory.

- b) Factory Method.
- c) Builder.
- d) Prototype.
- e) Singleton.

Comentários:

(a) Errado. Abstract Factory cria famílias de objetos relacionados. Aqui a ideia central é deixar que subclasses escolham qual autenticador concreto será instanciado.

(b) Correto. Factory Method encaixa direitinho nesse caso: ele centraliza a criação e permite que subclasses decidam entre OAuth, LDAP ou autenticação local.

(c) Errado. Builder é mais usado quando o foco está na construção passo a passo de objetos complexos, não na escolha do tipo concreto por subclasses.

(d) Errado. Prototype trabalha com clonagem de objetos já existentes, o que não é o problema descrito no enunciado.

(e) Errado. Singleton garante uma única instância de uma classe, mas não resolve a variação na criação de autenticadores diferentes.

Gabarito: Letra B

3. (FGV / TCE-RR - 2025) Um bom desenvolvedor de sistemas deve ter conhecimento sobre refatoração, padrões de arquitetura e de projeto, princípios SOLID e outros tópicos importantes sobre programação. Em relação a padrões de projeto, considere o seguinte trecho de código:

```
public interface Heroi {  
    void falar();  
}
```

```
public class HomemDeFerro implements Heroi {  
    public void falar() {  
        System.out.println("Eu sou o Homem de Ferro, da Marvel!");  
    }  
}
```

```
public class SuperHomem implements Heroi {  
    public void falar() {  
        System.out.println("Eu sou o Super Homem, da DC!");  
    }  
}
```

```
    }  
}  
  
public class MarvelOuDC {  
    public static Heroi getHeroi(String nome) {  
        if (nome.equalsIgnoreCase("HomemDeFerro")) {  
            return new HomemDeFerro();  
        } else if (nome.equalsIgnoreCase("SuperHomem"))  
        {  
            return new SuperHomem();  
        }  
    }  
}
```

Marque a opção que corresponde ao padrão de projeto implementado no código acima.

- a) Singleton
- b) Factory
- c) Observer
- d) Flyweight
- e) Builder

Comentários:

- (a) Errado. Singleton trata de garantir uma única instância de uma classe, o que não é o foco do código mostrado.
- (b) Correto. O método getHeroi centraliza a criação dos objetos e devolve a implementação conforme o nome informado, que é a ideia do padrão Factory.
- (c) Errado. Observer envolve relação de notificação entre objetos observadores e observado, algo que não aparece no trecho.
- (d) Errado. Flyweight busca compartilhar objetos para economizar memória, o que não é o caso desse exemplo.
- (e) Errado. Builder é usado para construir objetos complexos em etapas, diferente da simples escolha e criação feita no código.

Gabarito: Letra B

4. (FGV / TCE-PE - 2025) Leia a descrição a seguir.

Trata-se de um padrão que fornece uma interface para criar famílias de objetos relacionados ou dependentes, sem especificar suas classes concretas, seja porque estas ainda são desconhecidas ou para permitir maior escalabilidade no futuro.

O padrão de projeto descrito é o:

- a) Builder.
- b) Singleton.
- c) Abstract Factory.
- d) Decorator.
- e) Template Method.

Comentários:

(a) Errado. Builder foca a construção passo a passo de objetos mais complexos, não a criação de famílias de objetos relacionados.

(b) Errado. Singleton serve para garantir uma única instância de uma classe, o que é outra proposta.

(c) Correto. Abstract Factory cria famílias de objetos relacionados sem depender das classes concretas, exatamente como diz o enunciado.

(d) Errado. Decorator é usado para adicionar responsabilidades a objetos de forma flexível, não para criar famílias de objetos.

(e) Errado. Template Method define o esqueleto de um algoritmo, deixando etapas para subclasses, o que não é o caso aqui.

Gabarito: Letra C

5. (FGV / AgSUS - 2025) Os princípios GRASP (General Responsibility Assignment Software Patterns) são padrões de design de software orientados a objetos que servem como diretrizes para a atribuição de responsabilidades a classes e objetos. Dentre os princípios GRASP podemos identificar os seguintes:

- a) Baixo Acoplamento (Low Coupling, Proteção contra Variações (Protected Variations) e Responsabilidade única (Single Responsibility).
- b) Alta Coesão (High Cohesion), Indireção (Indirection) e Substituição de Liskov (Liskov Substitution).

- c) Controlador (Controller), Especialista na Informação (Information Expert) e Inversão da dependência (Dependency Inversion).
- d) Controlador (Controller), Criador (Creator) e Fabricação Pura/Pura Fabricação (Pure Fabrication).
- e) Baixa Coesão (Low Cohesion), Indireção (Indirection) e Polimorfismo (Polymorphism).

Comentários:

- (a) Errado. A alternativa mistura princípios GRASP com outro princípio de design, como Responsabilidade Única, que é mais associado ao SOLID.
- (b) Errado. Embora traga itens de GRASP, inclui Substituição de Liskov, que pertence ao SOLID, então o conjunto não fica correto.
- (c) Errado. Controlador e Especialista na Informação são GRASP, mas Inversão de Dependência é princípio do SOLID, não entra aqui.
- (d) Correto. Controlador, Criador e Fabricação Pura são, sim, princípios clássicos do GRASP, cobrados justamente para orientar a distribuição de responsabilidades.
- (e) Errado. Indireção e Polimorfismo aparecem no contexto de GRASP, mas “Baixa Coesão” contraria a ideia do padrão, que trabalha com Alta Coesão.

Gabarito: Letra D

6. (FGV / Prefeitura de BH - 2024) Padrões de projeto são soluções consagradas que se baseiam nas estruturas da orientação a objetos para solucionar problemas comuns em projetos de software. Os padrões são agrupados em tipos. Assinale a opção que indica apenas padrões do tipo comportamental.

- a) Decorator, bridge e adapter.
- b) Singleton, prototype e builder.
- c) Iterator, memento e observer.
- d) Command, mediator e facade.

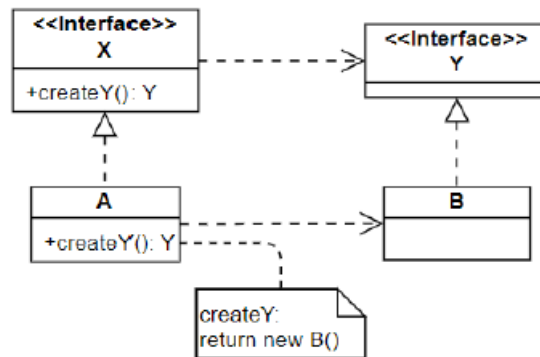
Comentários:

- (a) Errado. Decorator, Bridge e Adapter não entram no grupo dos comportamentais; eles são mais ligados à organização estrutural entre classes e objetos.

- (b) Errado. Singleton, Prototype e Builder são padrões voltados à criação de objetos, ou seja, pertencem ao grupo dos criacionais.
- (c) Correto. Iterator, Memento e Observer são padrões comportamentais, pois tratam da interação entre objetos e da forma como eles se comunicam.
- (d) Errado. Command e Mediator são comportamentais, mas Facade é estrutural, então a alternativa mistura tipos diferentes.

Gabarito: Letra C

7. (FGV / TJ AP - 2024) Padrões de projeto são representados por meio de diagramas da UML, sendo possível identificá-los através da análise de métodos e relacionamentos entre classes ou interfaces. Observe o diagrama de classes apresentado a seguir.



No diagrama é representado o padrão de projeto:

- a) Singleton;
- b) Prototype;
- c) Data Access Object;
- d) Observer;
- e) Abstract Factory.

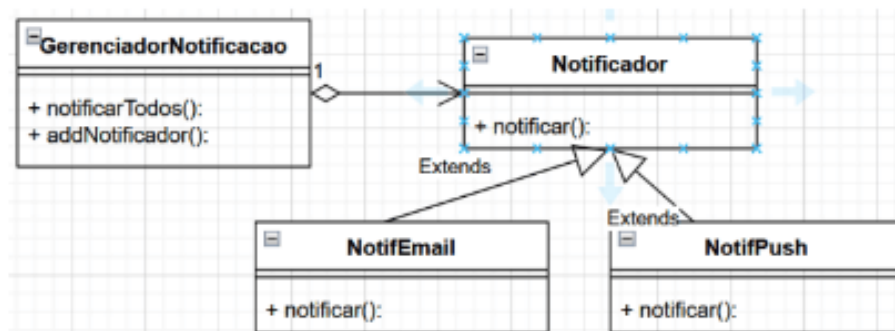
Comentários:

(a) Errado. O padrão Singleton garante a existência de uma única instância de uma classe, normalmente por meio de um construtor privado e um método de acesso à instância. O diagrama não apresenta essas características.

- (b) Errado. O padrão Prototype cria novos objetos por clonagem de uma instância existente, utilizando operações como clone(). O diagrama mostra um método de criação (createY()), e não de clonagem.
- (c) Errado. Data Access Object (DAO) é um padrão de acesso a dados e envolve a abstração das operações de persistência. O diagrama não representa acesso a banco de dados.
- (d) Errado. O padrão Observer modela a relação entre um sujeito e seus observadores, com notificações automáticas de mudanças de estado. Esses elementos não aparecem no diagrama.
- (e) Correto. O diagrama representa o padrão Abstract Factory. A interface X declara o método de fábrica createY(): Y, e a classe A implementa esse método retornando uma instância de B (return new B()), enquanto B implementa a interface Y. Assim, uma fábrica concreta cria um produto concreto por meio de interfaces abstratas, desacoplando a criação dos objetos de sua utilização.

Gabarito: Letra E

8. (FGV / ALETO - 2024) O diagrama a seguir mostra um módulo de notificação para uma aplicação de e-commerce para permitir diversos serviços de notificação, tais como, e-mail e push.



Assinale a opção que indica o padrão de projeto usado nesse módulo.

- a) Singleton.
- b) Factory Method.
- c) Abstract Factory.
- d) Observer.
- e) Prototype.

Comentários:

- (a) Errado. O padrão Singleton garante uma única instância de uma classe. O diagrama não apresenta mecanismo de instância única nem método de acesso global.

- (b) Errado. Factory Method é voltado à criação de objetos por meio de métodos de fábrica. O diagrama não evidencia classes criadoras nem métodos de criação de notificadores.
- (c) Errado. Abstract Factory cria famílias de objetos relacionados por meio de interfaces de fábrica. O diagrama não mostra fábricas abstratas nem produtos abstratos.
- (d) Correto. O padrão Observer é caracterizado por um objeto manter uma coleção de observadores e notificá-los sobre determinado evento. No diagrama, o GerenciadorNotificacao agrega notificadores (addNotificador()) e dispara notificarTodos(), enquanto NotifEmail e NotifPush implementam o comportamento notificar(), permitindo adicionar novos tipos de notificação sem alterar o gerenciador.
- (e) Errado. O padrão Prototype baseia-se na clonagem de objetos existentes, característica que não aparece no diagrama.

Gabarito: Letra D

9. (FGV / ALETO - 2024) Assinale a opção que indica o padrão responsável pela criação de objetos de várias classes graças à especialização de sua classe principal.

- a) Abstract Factory.
- b) Decorator.
- c) Strategy.
- d) Builder.
- e) Adapter.

Comentários:

- (a) Correto. Abstract Factory trata justamente da criação de famílias de objetos relacionados, permitindo gerar objetos de várias classes por meio da especialização da fábrica principal.
- (b) Errado. Decorator serve para adicionar responsabilidades a objetos, sem focar na criação de várias classes por especialização.
- (c) Errado. Strategy organiza algoritmos intercambiáveis, ou seja, o foco é comportamento, não criação de objetos.
- (d) Errado. Builder é voltado à construção passo a passo de objetos complexos, com outra proposta.
- (e) Errado. Adapter é usado para compatibilizar interfaces diferentes, não para criar objetos de várias classes por especialização.

Gabarito: Letra A

10.(FGV / TJ MS - 2024) O analista Lucas está desenvolvendo o sistema orientado a objeto JustiSul. Em determinado módulo do JustiSul, o sistema instancia um objeto da classe Terceirizado de forma dinâmica, em tempo de execução, por meio da interface I3rd. A classe Terceirizado pertence a uma biblioteca proprietária externa, de forma que o JustiSul conhece apenas a especificação de I3rd, e não a de Terceirizado. Dentro de outro módulo, o sistema precisa instanciar um segundo objeto Terceirizado. Devido ao alto custo de uma nova inicialização dinâmica, Lucas optou por chamar o método cp3rd do objeto Terceirizado já criado. O cp3rd, declarado em I3rd e implementado na classe Terceirizado, retorna uma nova instância da classe, com um estado idêntico ao do objeto chamador.

Ao permitir a criação de novas instâncias pelo método cp3rd de uma instância existente, Terceirizado implementa o padrão de projeto:

- a) builder;
- b) singleton;
- c) prototype;
- d) factory method;
- e) abstract factory.

Comentários:

(a) Errado. Builder é usado para montar objetos complexos passo a passo, e não para gerar cópias de uma instância já existente.

(b) Errado. Singleton busca garantir uma única instância da classe, o que não combina com a criação de um novo objeto por cópia.

(c) Correto. Prototype é justamente o padrão em que um objeto cria outro a partir de si mesmo, clonando seu estado por meio de uma operação como cp3rd.

(d) Errado. Factory Method trata da criação por subclasses ou métodos-fábrica, não da duplicação de um objeto já instanciado.

(e) Errado. Abstract Factory cria famílias de objetos relacionados, mas não tem como foco copiar uma instância existente.

Gabarito: Letra C

11.(FGV / Prefeitura de Caraguatatuba - 2024) Leia o fragmento a seguir sobre Design Patterns.

Os padrões de design desempenham um papel crucial na arquitetura de software, proporcionando soluções testadas e comprovadas para desafios recorrentes no desenvolvimento de sistemas. Esses padrões são geralmente classificados em três categorias principais: padrões de design _____, que são estratégias que se concentram nos padrões de comunicação e colaboração entre objetos; padrões de design _____, que oferece abordagens flexíveis e reutilizáveis na instancição e composição de objetos e os padrões de design _____, que referem as estratégias voltadas para a eficiente organização de classes e objetos, visando aprimorar a composição e simplificar a interação entre os componentes de um sistema. Cada categoria oferece um conjunto valioso de diretrizes e abordagens que promovem a _____, e manutenção do código, contribuindo para o desenvolvimento de sistemas robustos e eficientes.

Assinale a opção cujos itens completam corretamente as lacunas do fragmento lido.

- a) comportamental – criacional – estrutural – flexibilidade e segurança
- b) estrutural – criacional – comportamental – reusabilidade e segurança
- c) estrutural – comportamental – criacional – flexibilidade e reusabilidade
- d) comportamental – criacional – estrutural – flexibilidade e reusabilidade
- e) criacional – comportamental – estrutural – reusabilidade e acessibilidade

Comentários:

(a) Errado. A sequência inicial até coincide em parte, mas o fechamento com “segurança” não acompanha a ideia central do texto, que destaca ganhos mais ligados ao reaproveitamento e adaptação do código.

(b) Errado. A ordem das categorias não bate com as definições do enunciado. O trecho fala primeiro de comunicação entre objetos, depois de criação e composição, e por fim de organização estrutural.

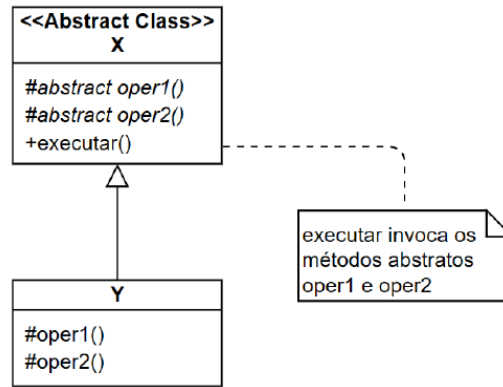
(c) Errado. Aqui há troca entre as categorias do meio e do fim. Além disso, a combinação apresentada não acompanha a sequência conceitual descrita no fragmento.

(d) Correto. “Comportamental – criacional – estrutural – flexibilidade e reusabilidade” encaixa direitinho nas definições dadas e nos benefícios clássicos associados aos design patterns.

(e) Errado. A ordem das categorias não corresponde ao texto, e “acessibilidade” não é o benefício mais diretamente associado ao uso de padrões nesse contexto.

Gabarito: Letra D

12.(FGV / CVM - 2024) Os padrões de projeto são extremamente úteis para organizar a arquitetura do sistema e o modelo de programação. Eles são projetados em diagramas da UML, como no modelo a seguir.



O diagrama expressa o padrão de projeto:

- a) Observer;
- b) Singleton;
- c) Data Access Object;
- d) Intercept Filter;
- e) Template Method.

Comentários:

(a) Errado. O padrão Observer estabelece uma relação entre um objeto e seus observadores para notificações automáticas de mudanças de estado. O diagrama não apresenta esse mecanismo.

(b) Errado. O padrão Singleton garante que uma classe possua apenas uma instância e um ponto global de acesso. Esses elementos não aparecem no diagrama.

(c) Errado. Data Access Object (DAO) é um padrão para abstração do acesso a dados e persistência. O diagrama não representa operações de banco de dados.

(d) Errado. O padrão Intercept Filter organiza filtros para pré e pós-processamento de requisições, característica ausente no diagrama.

(e) Correto. O diagrama representa o padrão Template Method. A classe abstrata X define o método concreto executar(), que estabelece o fluxo do algoritmo chamando os métodos abstratos oper1() e oper2(). A classe Y implementa esses métodos, personalizando partes do algoritmo sem alterar sua estrutura geral.

Gabarito: Letra E

13.(FGV / CVM - 2024) Natália está desenvolvendo um sistema financeiro, no qual os documentos podem receber diferentes tipos de tratamento, e pretende tirar proveito dos padrões de desenvolvimento. Para tratar os documentos, ela criou diversos processos, que serão colocados em uma fila. Assim, o documento passa por um dado processo da fila, é tratado e enviado para o processo seguinte. Adotando essa estratégia, Natália usou o padrão de desenvolvimento denominado:

- a) Chain of Responsibility;
- b) Prototype;
- c) Decorator;
- d) Abstract Factory;
- e) Factory Method.

Comentários:

- (a) Correto. No Chain of Responsibility, a requisição passa por uma sequência de objetos/processos, e cada um trata sua parte antes de encaminhar ao próximo, exatamente como ocorre com os documentos na fila.
- (b) Errado. Prototype é usado para criar novos objetos a partir da clonagem de um modelo existente, não para encadear etapas de processamento.
- (c) Errado. Decorator serve para adicionar responsabilidades a um objeto de forma dinâmica, sem envolver esse fluxo em fila entre vários processos.
- (d) Errado. Abstract Factory é voltado à criação de famílias de objetos relacionados, não ao tratamento sequencial de documentos.
- (e) Errado. Factory Method também é um padrão de criação, focado em definir a forma de instanciar objetos, e não em organizar processamento em cadeia.

Gabarito: Letra A

14.(FGV / SEF MG - 2023) Os padrões de projeto de software (design patterns) tiram proveito máximo dos pilares da orientação a objetos. Usemos como exemplo o padrão Abstract Factory, que é um padrão de projeto criacional que permite produzir famílias de objetos relacionados sem especificar suas classes concretas. O principal pilar da orientação a objetos usado nesse design pattern é

- a) o polimorfismo.
- b) a herança.
- c) o encapsulamento.
- d) a abstração.
- e) a sublevação.

Comentários:

- (a) Errado. Polimorfismo ajuda no uso de interfaces comuns, mas não é o ponto central destacado no Abstract Factory.
- (b) Errado. Herança pode aparecer na implementação, porém não é o pilar principal explorado pelo padrão.
- (c) Errado. Encapsulamento organiza a ocultação de detalhes, mas não é a base mais marcante desse caso.
- (d) Correto. A abstração é o foco do Abstract Factory, pois o padrão trabalha com interfaces e famílias de objetos sem depender das classes concretas.
- (e) Errado. Sublevação não é um pilar clássico da orientação a objetos.

Gabarito: Letra D

15.(FGV / DPE RS - 2023) Os padrões de projeto fornecem soluções para problemas recorrentes no desenvolvimento de sistemas. Maria está desenvolvendo o sistema ComprasWeb e precisa resolver um problema comum de sistemas Web que é a recepção assíncrona de dados, onde deve ocorrer a atualização dos dados na interface do usuário quando ocorre a resposta do servidor, sem que haja o bloqueio das demais funções da interface.

Para tratar o problema do ComprasWeb, Maria deve usar o padrão de projeto:

- a) Observer;
- b) Chain of Responsibility;
- c) Flyweight;
- d) Data Access Object;
- e) Builder.

Comentários:

- (a) Correto. Observer é o padrão indicado quando uma mudança de estado ou a chegada de uma resposta precisa disparar a atualização automática da interface, sem travar o restante da aplicação.
- (b) Errado. Chain of Responsibility serve para encadear tratadores de requisições, não para atualizar interface de forma assíncrona.
- (c) Errado. Flyweight é usado para economizar memória com compartilhamento de objetos, o que não resolve esse cenário.
- (d) Errado. Data Access Object organiza o acesso a dados e ao banco, mas não trata a atualização assíncrona da interface.
- (e) Errado. Builder é voltado à construção passo a passo de objetos complexos, sem relação direta com resposta assíncrona na interface.

Gabarito: Letra A

16.(FGV / DPE RS - 2023) Ana está implementando um sistema de comércio eletrônico e agora está definindo a operação de pagamento para a compra, podendo aceitar cartão de crédito, PIX ou boleto. Como a ação de pagamento envolve apenas a conciliação do valor final da compra, mas segundo diferentes meios de pagamento, Ana, com o objetivo de organizar seu código, permitiu a escolha do meio de pagamento por intermédio do padrão:

- a) State;
- b) Interpreter;
- c) Prototype;
- d) Memento;
- e) Strategy.

Comentários:

- (a) Errado. State trata mudança de comportamento conforme o estado interno do objeto, não da escolha entre formas alternativas de executar um pagamento.
- (b) Errado. Interpreter é usado para interpretar linguagens ou expressões, o que não é o caso da seleção entre meios de pagamento.
- (c) Errado. Prototype serve para criar objetos por clonagem, sem relação direta com definir a forma de pagamento.

(d) Errado. Memento é voltado a salvar e restaurar estados de um objeto, não a variar o algoritmo de pagamento.

(e) Correto. Strategy encaixa direitinho aqui, porque permite escolher entre diferentes formas de pagamento, como cartão, PIX ou boleto, mantendo a mesma operação com estratégias distintas.

Gabarito: Letra E

17.(FGV / ALEMA - 2023) Um padrão de projeto é uma solução geral para um problema que ocorre com frequência dentro de um determinado contexto no projeto de software. O padrão de projeto de software denominado Bridge é um padrão

- a) estrutural que desacopla uma abstração de sua implementação para que os dois possam variar independentemente.
- b) comportamental destinado a armazenar o estado interno de um objeto em um dado momento para que seja possível retorná-lo a este estado.
- c) de controle de objetos utilizado para garantir que determinada classe só tenha uma única instância e prover um ponto de acesso global a ela.
- d) de criação para instanciar famílias de objetos relacionados por meio de uma única interface e sem que a classe concreta seja especificada.
- e) de interação para prover uma maneira de acessar os elementos de um objeto agregado sequencialmente sem expor sua representação interna.

Comentários:

(a) Correto. Bridge é um padrão estrutural cuja ideia central é separar a abstração da implementação, permitindo que ambas evoluam de forma independente.

(b) Errado. Essa descrição corresponde ao Memento, usado para guardar e restaurar o estado interno de um objeto.

(c) Errado. Aqui a ideia é do Singleton, que garante uma única instância e um ponto global de acesso.

(d) Errado. Essa definição é do Abstract Factory, voltado à criação de famílias de objetos relacionados sem expor classes concretas.

(e) Errado. Esse enunciado descreve o Iterator, que permite percorrer elementos sequencialmente sem mostrar a estrutura interna do agregado.

Gabarito: Letra A

18.(FGV / TCE SP - 2023) O analista André é o arquiteto da aplicação web TCEDash. O componente Board, da TCEDash, apresenta ao usuário indicadores visuais e manipuláveis, baseados em informações contidas no banco de dados da aplicação. O Board instancia um novo objeto para cada indicador apresentado ao usuário. Sabendo que determinado subconjunto de informações se repete em todos os indicadores, André aplicou no componente Board o padrão de projeto estrutural do Gang Of Four (GoF), que otimiza a alocação de objetos na memória RAM, mediante o compartilhamento de atributos comuns entre os objetos.

André aplicou no componente Board o padrão do GoF:

- a) proxy;
- b) bridge;
- c) facade;
- d) flyweight;
- e) composite.

Comentários:

(a) Errado. Proxy atua como representante de outro objeto, controlando acesso, e não tem como foco principal compartilhar estado comum para economizar memória.

(b) Errado. Bridge serve para desacoplar abstração e implementação, permitindo evolução independente, não para reaproveitar atributos entre vários objetos.

(c) Errado. Facade cria uma interface mais simples para um subsistema, facilitando o uso, mas não resolve compartilhamento de dados comuns entre instâncias.

(d) Correto. Flyweight é justamente o padrão usado para reduzir uso de memória ao compartilhar atributos comuns entre muitos objetos parecidos.

(e) Errado. Composite organiza objetos em estruturas de árvore, tratando partes e todo de forma uniforme, sem foco em otimização de memória por compartilhamento.

Gabarito: Letra D

19.(FGV / TCE SP - 2023) Marcos está efetuando a manutenção de um sistema e notou que em diversos pontos são efetuadas chamadas HTTP para diferentes servidores, utilizando configurações específicas. Ele resolveu refatorar o código, criando uma classe que oferece métodos para cada elemento da configuração, os quais podem ser combinados para gerar um conector personalizado para cada servidor acessado.

Adotando essa abordagem, Marcos aumentou o reuso de processos intermediários na construção de objetos complexos, de acordo com o padrão:

- a) Singleton;
- b) Mediator;
- c) Builder;
- d) Observer;
- e) Chain of Responsibility.

Comentários:

(a) Errado. Singleton serve para garantir uma única instância de uma classe, não para montar objetos complexos em etapas.

(b) Errado. Mediator centraliza a comunicação entre objetos, reduzindo acoplamento, mas não foca na construção configurável de conectores.

(c) Correto. Builder é justamente o padrão usado para construir objetos complexos passo a passo, combinando partes da configuração com reuso.

(d) Errado. Observer trata de notificação entre objetos, quando um muda de estado e os outros precisam ser avisados.

(e) Errado. Chain of Responsibility organiza o tratamento em cadeia, passando requisições entre objetos, sem foco em construção de objetos complexos.

Gabarito: Letra C

20.(FGV / CAM DEP - 2023) Assinale a opção que indica o padrão de projeto GoF ('Gang of Four') que, com finalidade comportamental, sem violar o encapsulamento, captura e externaliza um estado interno de um objeto, de modo que o mesmo possa posteriormente ser restaurado para este estado.

- a) bridge.
- b) facade.

- c) flyweight.
- d) memento.
- e) visitor.

Comentários:

- (a) Errado. Bridge é um padrão estrutural, usado para separar abstração de implementação, não para guardar e restaurar estado interno.
- (b) Errado. Facade também é estrutural e serve para simplificar o acesso a subsistemas, sem relação com captura de estado.
- (c) Errado. Flyweight é estrutural e busca economizar memória com compartilhamento de objetos, não salvar estados.
- (d) Correto. Memento é o padrão comportamental que permite capturar e externalizar o estado interno de um objeto para restaurá-lo depois, sem quebrar o encapsulamento.
- (e) Errado. Visitor é comportamental, mas sua função é separar operações da estrutura dos objetos, não armazenar estado para restauração.

Gabarito: Letra D

21.(FGV / TJ SE - 2023) A analista Maria está desenvolvendo a funcionalidade C de uma aplicação, utilizando os padrões de projeto Gang of Four (GoF). Existem vários algoritmos, conhecidos e disponíveis, capazes de implementar a funcionalidade C. A aplicação deve deixar o usuário escolher qual algoritmo usar. Sendo assim, Maria decidiu aplicar na funcionalidade C o padrão GoF comportamental que modela algoritmos similares como objetos independentes, porém, passíveis de troca em tempo de execução. Maria decidiu aplicar o padrão GoF:

- a) Visitor;
- b) Strategy;
- c) Mediator;
- d) Interpreter;
- e) Template Method.

Comentários:

- (a) Errado. Visitor serve para adicionar operações a estruturas de objetos, sem mudar suas classes, não para trocar algoritmos de execução.

- (b) Correto. Strategy encapsula algoritmos em objetos independentes e permite trocá-los em tempo de execução, exatamente como o enunciado descreve.
- (c) Errado. Mediator centraliza a comunicação entre objetos, reduzindo acoplamento, mas não tem foco em escolha de algoritmos.
- (d) Errado. Interpreter é usado para representar e interpretar gramáticas e expressões de linguagem, o que não é o caso aqui.
- (e) Errado. Template Method define o esqueleto de um algoritmo numa superclasse, mas não é voltado à troca dinâmica entre algoritmos distintos.

Gabarito: Letra B

22.(FGV / TJ SE - 2023) Pedro está implementando a aplicação web SergipeJusto. Pedro aplicou em SergipeJusto o padrão de projeto comportamental que soluciona o problema de armazenar e, posteriormente, restaurar o estado interno de um objeto sem violar o seu encapsulamento. Pedro aplicou em SergipeJusto o padrão de projeto comportamental:

- a) State;
- b) Iterator;
- c) Mediator;
- d) Memento;
- e) Prototype.

Comentários:

- (a) Errado. State trata da mudança de comportamento conforme o estado do objeto, não do salvamento e restauração desse estado.
- (b) Errado. Iterator serve para percorrer elementos de uma coleção sem expor sua estrutura interna.
- (c) Errado. Mediator centraliza a comunicação entre objetos, reduzindo acoplamento entre eles.
- (d) Correto. Memento é o padrão usado para guardar e depois restaurar o estado interno de um objeto sem quebrar o encapsulamento.
- (e) Errado. Prototype está ligado à criação de objetos por clonagem, não ao armazenamento de estado.

Gabarito: Letra D

23.(FGV / TJ SE - 2023) André está trabalhando em um sistema com atualização de dados de forma assíncrona, onde um gráfico e uma planilha precisam repercutir de forma automática qualquer alteração efetuada na fonte de dados. Para implementar a solução, André deve usar o padrão de desenvolvimento:

- a) Chain of Responsibility;
- b) Adapter;
- c) Factory Method;
- d) Observer;
- e) Front Controller.

Comentários:

- (a) Errado. Chain of Responsibility serve para encadear o tratamento de requisições entre vários objetos, não para notificar automaticamente mudanças em dados.
- (b) Errado. Adapter é usado para compatibilizar interfaces diferentes, facilitando integração entre classes, e não para atualização automática entre elementos.
- (c) Errado. Factory Method trata da criação de objetos, definindo como instâncias são produzidas, sem foco em propagação de alterações.
- (d) Correto. Observer é o padrão ideal quando vários elementos, como gráfico e planilha, precisam ser avisados automaticamente sobre mudanças na fonte de dados.
- (e) Errado. Front Controller centraliza o tratamento de requisições em aplicações, muito comum na web, mas não resolve sincronização assíncrona de atualizações.

Gabarito: Letra D

24.(FGV / TJ SE - 2023) Paulo implementou um sistema na plataforma Java EE, onde foi adotada a arquitetura MVC, colocando Servlets e JSPs na camada View, entidades JPA na Model e Session Beans na Controller. Como os Session Beans são os únicos componentes que instanciam gestores de persistência do JPA, Paulo segue o padrão de desenvolvimento:

- a) Decorator;
- b) Facade;
- c) Template Method;
- d) Abstract Factory;
- e) State.

Comentários:

- (a) Errado. Decorator serve para adicionar responsabilidades a um objeto de forma dinâmica, o que não é o foco do caso apresentado.
- (b) Correto. Facade centraliza o acesso a funcionalidades mais complexas. Aqui, os Session Beans concentram a criação e o uso dos gestores de persistência, simplificando o acesso às regras de negócio e à persistência.
- (c) Errado. Template Method define o esqueleto de um algoritmo, deixando etapas para subclasses, o que não aparece na situação descrita.
- (d) Errado. Abstract Factory é usado para criar famílias de objetos relacionados, sem especificar classes concretas, o que não é a ideia central do enunciado.
- (e) Errado. State trata da mudança de comportamento de um objeto conforme seu estado interno, algo diferente da centralização feita pelos Session Beans.

Gabarito: Letra B**25.(FGV / Sefaz AM - 2022) O padrão de projeto de software orientado a objetos, que pode ser utilizado para garantir que uma determinada classe tenha somente uma única instância, é**

- a) adapter.
- b) flyweight.
- c) memento.
- d) singleton.
- e) visitor.

Comentários:

- (a) Errado. Adapter serve para adaptar interfaces entre classes incompatíveis, não para limitar a quantidade de instâncias.
- (b) Errado. Flyweight busca economizar memória com compartilhamento de objetos, não garantir instância única.
- (c) Errado. Memento é usado para salvar e restaurar estados de objetos, como um histórico.

- (d) Correto. Singleton é justamente o padrão criado para assegurar que uma classe tenha apenas uma única instância e um ponto global de acesso.
- (e) Errado. Visitor permite adicionar operações a estruturas de objetos sem mudar suas classes, não controlar instanciação.

Gabarito: Letra D

26.(FGV / SEMSA Manaus - 2022) No contexto de Design Patterns (Padrões de projetos), um dos padrões mais simples em Java envolve uma classe simples, responsável pela criação de um objeto com a garantia de que apenas um simples objeto é criado. Esse pattern é conhecido como

- a) Adapter.
- b) Builder.
- c) Factory.
- d) Singleton.
- e) State.

Comentários:

- (a) Errado. Adapter serve para adaptar interfaces diferentes, permitindo que classes incompatíveis trabalhem juntas.
- (b) Errado. Builder é usado para construir objetos complexos passo a passo, não para garantir instância única.
- (c) Errado. Factory trata da criação de objetos, mas sem a ideia central de existir apenas uma única instância.
- (d) Correto. Singleton é o padrão que garante que uma classe tenha apenas um único objeto criado e um ponto global de acesso a ele.
- (e) Errado. State é voltado para alterar o comportamento de um objeto conforme seu estado interno muda.

Gabarito: Letra D

27.(FGV / TCE-AM - 2021) O Sistema de Acompanhamento de Mercados (SisMARKET) apoia o processo de controle de transações de compra e venda de ativos financeiros e fornece aos gestores visões distintas contendo gráficos que demonstram continuamente o andamento das transações. Sendo assim, o SisMARKET possui uma funcionalidade para notificar e atualizar as

visões gráficas automaticamente sempre que o quantitativo de um ativo financeiro sofre alterações. No projeto de desenvolvimento do SisMARKET foram empregadas boas práticas, como o uso de padrões de projeto.

Sendo assim, no SisMARKET, o padrão de projeto utilizado na construção da funcionalidade que configura uma dependência um-para-muitos entre objetos, de modo que, quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente, foi o:

- a) Memento;
- b) Proxy;
- c) Observer;
- d) State;
- e) Strategy.

Comentários:

(a) Errado. Memento serve para guardar e restaurar estados anteriores de um objeto, como um "snapshot", não para notificar vários dependentes automaticamente.

(b) Errado. Proxy atua como um intermediário para controlar acesso a outro objeto, mas não tem foco em atualização automática de vários interessados.

(c) Correto. Observer é justamente o padrão de dependência um-para-muitos: quando um objeto muda de estado, os observadores são avisados e atualizados.

(d) Errado. State trata da mudança de comportamento de um objeto conforme seu estado interno, não de notificação para múltiplos objetos.

(e) Errado. Strategy organiza algoritmos intercambiáveis, permitindo trocar a lógica usada, mas não resolve o mecanismo de aviso e atualização automática.

Gabarito: Letra C

28.(FGV / FunSaúde CE - 2021) No contexto de padrões de projeto GoF, há um padrão que tem como definição garantir que uma classe tenha apenas uma instância de si mesma e que forneça um ponto global de acesso. Assinale-o.

- a) Builder.
- b) Factory.
- c) Proxy.

- d) Singleton.
- e) Strategy.

Comentários:

- (a) Errado. Builder é usado para construir objetos complexos em etapas, separando a construção da representação final.
- (b) Errado. Factory trata da criação de objetos, mas não de limitar a classe a uma única instância global.
- (c) Errado. Proxy funciona como um intermediário para controlar o acesso a outro objeto, com outra finalidade.
- (d) Correto. Singleton é justamente o padrão que garante uma única instância da classe e um ponto global de acesso.
- (e) Errado. Strategy serve para encapsular algoritmos e permitir trocar comportamentos, não para restringir instâncias.

Gabarito: Letra D

LISTA DE QUESTÕES

1. (FGV / AMAZUL - 2026) Em programação orientada a objetos, um desenvolvedor precisa garantir que apenas uma instância de uma classe de conexão com banco de dados seja criada durante toda a execução da aplicação, evitando múltiplas conexões desnecessárias. O padrão de projeto que resolve esse problema é o
- a) Factory Method.
 - b) Observer.
 - c) Singleton.
 - d) Strategy.
 - e) Decorator.
2. (FGV / AMAZUL - 2026) Uma equipe de desenvolvimento identificou que o código de um módulo de autenticação está sendo duplicado em vários pontos do sistema, dificultando a manutenção. A equipe decide refatorar o código aplicando um padrão que centralize a criação de objetos de autenticação, permitindo que subclasses decidam qual tipo específico de autenticador instanciar (OAuth, LDAP, ou local). O padrão de projeto que atende a essa necessidade é chamado
- a) Abstract Factory.
 - b) Factory Method.
 - c) Builder.
 - d) Prototype.
 - e) Singleton.
3. (FGV / TCE-RR - 2025) Um bom desenvolvedor de sistemas deve ter conhecimento sobre refatoração, padrões de arquitetura e de projeto, princípios SOLID e outros tópicos importantes sobre programação. Em relação a padrões de projeto, considere o seguinte trecho de código:

```
public interface Heroi {  
    void falar();  
}
```

```
public class HomemDeFerro implements Heroi {  
    public void falar() {  
        System.out.println("Eu sou o Homem de Ferro, da Marvel!");  
    }  
}
```

```
public class SuperHomem implements Heroi {
```

```
public void falar() {  
    System.out.println("Eu sou o Super Homem, da DC!");  
}  
  
public class MarvelOuDC {  
    public static Heroi getHeroi(String nome) {  
        if (nome.equalsIgnoreCase("HomemDeFerro")) {  
            return new HomemDeFerro();  
        } else if (nome.equalsIgnoreCase("SuperHomem"))  
        {  
            return new SuperHomem();  
        }  
    }  
}
```

Marque a opção que corresponde ao padrão de projeto implementado no código acima.

- a) Singleton
- b) Factory
- c) Observer
- d) Flyweight
- e) Builder

4. (FGV / TCE-PE - 2025) Leia a descrição a seguir.

Trata-se de um padrão que fornece uma interface para criar famílias de objetos relacionados ou dependentes, sem especificar suas classes concretas, seja porque estas ainda são desconhecidas ou para permitir maior escalabilidade no futuro.

O padrão de projeto descrito é o:

- a) Builder.
- b) Singleton.
- c) Abstract Factory.
- d) Decorator.
- e) Template Method.

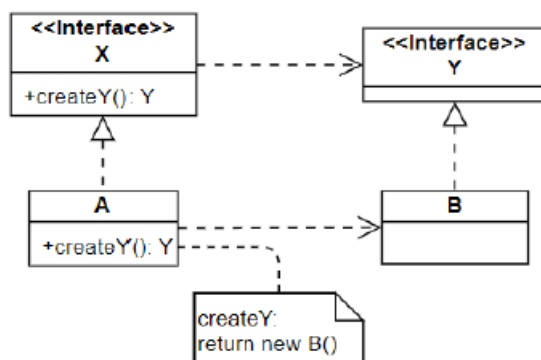
5. (FGV / AgSUS - 2025) Os princípios GRASP (General Responsibility Assignment Software Patterns) são padrões de design de software orientados a objetos que servem como diretrizes para a atribuição de responsabilidades a classes e objetos. Dentre os princípios GRASP podemos identificar os seguintes:

- a) Baixo Acoplamento (Low Coupling), Proteção contra Variações (Protected Variations) e Responsabilidade única (Single Responsibility).
- b) Alta Coesão (High Cohesion), Indireção (Indirection) e Substituição de Liskov (Liskov Substitution).
- c) Controlador (Controller), Especialista na Informação (Information Expert) e Inversão da dependência (Dependency Inversion).
- d) Controlador (Controller), Criador (Creator) e Fabricação Pura/Pura Fabricação (Pure Fabrication).
- e) Baixa Coesão (Low Cohesion), Indireção (Indirection) e Polimorfismo (Polymorphism).

6. (FGV / Prefeitura de BH - 2024) Padrões de projeto são soluções consagradas que se baseiam nas estruturas da orientação a objetos para solucionar problemas comuns em projetos de software. Os padrões são agrupados em tipos. Assinale a opção que indica apenas padrões do tipo comportamental.

- a) Decorator, bridge e adapter.
- b) Singleton, prototype e builder.
- c) Iterator, memento e observer.
- d) Command, mediator e facade.

7. (FGV / TJ AP - 2024) Padrões de projeto são representados por meio de diagramas da UML, sendo possível identificá-los através da análise de métodos e relacionamentos entre classes ou interfaces. Observe o diagrama de classes apresentado a seguir.

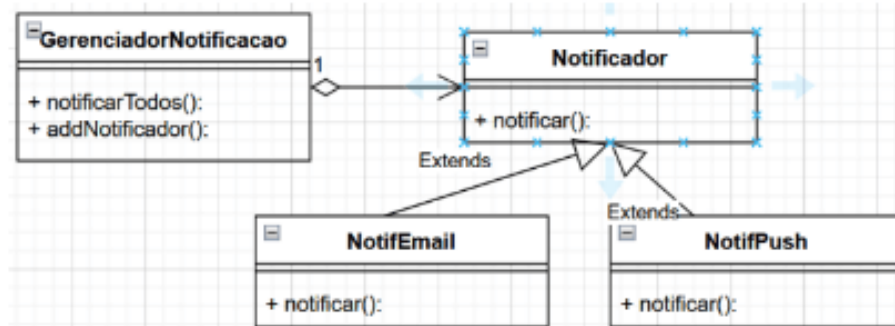


No diagrama é representado o padrão de projeto:

- a) Singleton;
- b) Prototype;
- c) Data Access Object;

- d) Observer;
- e) Abstract Factory.

8. (FGV / ALETO - 2024) O diagrama a seguir mostra um módulo de notificação para uma aplicação de e-commerce para permitir diversos serviços de notificação, tais como, e-mail e push.



Assinale a opção que indica o padrão de projeto usado nesse módulo.

- a) Singleton.
- b) Factory Method.
- c) Abstract Factory.
- d) Observer.
- e) Prototype.

9. (FGV / ALETO - 2024) Assinale a opção que indica o padrão responsável pela criação de objetos de várias classes graças à especialização de sua classe principal.

- a) Abstract Factory.
- b) Decorator.
- c) Strategy.
- d) Builder.
- e) Adapter.

10.(FGV / TJ MS - 2024) O analista Lucas está desenvolvendo o sistema orientado a objeto JustiSul. Em determinado módulo do JustiSul, o sistema instancia um objeto da classe Terceirizado de forma dinâmica, em tempo de execução, por meio da interface I3rd. A classe Terceirizado pertence a uma biblioteca proprietária externa, de forma que o JustiSul conhece apenas a especificação de I3rd, e não a de Terceirizado. Dentro de outro módulo, o sistema precisa instanciar um segundo objeto Terceirizado. Devido ao alto custo de uma nova inicialização dinâmica, Lucas optou por chamar o método `cp3rd` do objeto Terceirizado já criado. O `cp3rd`, declarado em I3rd e implementado na classe Terceirizado, retorna uma nova instância da classe, com um estado idêntico ao do objeto chamador.

Ao permitir a criação de novas instâncias pelo método cp3rd de uma instância existente, Terceirizado implementa o padrão de projeto:

- a) builder;
- b) singleton;
- c) prototype;
- d) factory method;
- e) abstract factory.

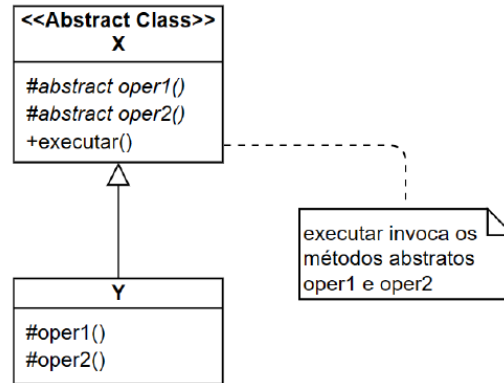
11.(FGV / Prefeitura de Caraguatatuba - 2024) Leia o fragmento a seguir sobre Design Patterns.

Os padrões de design desempenham um papel crucial na arquitetura de software, proporcionando soluções testadas e comprovadas para desafios recorrentes no desenvolvimento de sistemas. Esses padrões são geralmente classificados em três categorias principais: padrões de design _____, que são estratégias que se concentram nos padrões de comunicação e colaboração entre objetos; padrões de design _____, que oferece abordagens flexíveis e reutilizáveis na instanciação e composição de objetos e os padrões de design _____, que referem as estratégias voltadas para a eficiente organização de classes e objetos, visando aprimorar a composição e simplificar a interação entre os componentes de um sistema. Cada categoria oferece um conjunto valioso de diretrizes e abordagens que promovem a _____, e manutenção do código, contribuindo para o desenvolvimento de sistemas robustos e eficientes.

Assinale a opção cujos itens completam corretamente as lacunas do fragmento lido.

- a) comportamental – criacional – estrutural – flexibilidade e segurança
- b) estrutural – criacional – comportamental – reusabilidade e segurança
- c) estrutural – comportamental – criacional – flexibilidade e reusabilidade
- d) comportamental – criacional – estrutural – flexibilidade e reusabilidade
- e) criacional – comportamental – estrutural – reusabilidade e acessibilidade

12.(FGV / CVM - 2024) Os padrões de projeto são extremamente úteis para organizar a arquitetura do sistema e o modelo de programação. Eles são projetados em diagramas da UML, como no modelo a seguir.



O diagrama expressa o padrão de projeto:

- a) Observer;
- b) Singleton;
- c) Data Access Object;
- d) Intercept Filter;
- e) Template Method.

13.(FGV / CVM - 2024) Natália está desenvolvendo um sistema financeiro, no qual os documentos podem receber diferentes tipos de tratamento, e pretende tirar proveito dos padrões de desenvolvimento. Para tratar os documentos, ela criou diversos processos, que serão colocados em uma fila. Assim, o documento passa por um dado processo da fila, é tratado e enviado para o processo seguinte. Adotando essa estratégia, Natália usou o padrão de desenvolvimento denominado:

- a) Chain of Responsibility;
- b) Prototype;
- c) Decorator;
- d) Abstract Factory;
- e) Factory Method.

14.(FGV / SEF MG - 2023) Os padrões de projeto de software (design patterns) tiram proveito máximo dos pilares da orientação a objetos. Usemos como exemplo o padrão Abstract Factory, que é um padrão de projeto criacional que permite produzir famílias de objetos relacionados sem especificar suas classes concretas. O principal pilar da orientação a objetos usado nesse design pattern é

- a) o polimorfismo.
- b) a herança.
- c) o encapsulamento.
- d) a abstração.

e) a sublevação.

15.(FGV / DPE RS - 2023) Os padrões de projeto fornecem soluções para problemas recorrentes no desenvolvimento de sistemas. Maria está desenvolvendo o sistema ComprasWeb e precisa resolver um problema comum de sistemas Web que é a recepção assíncrona de dados, onde deve ocorrer a atualização dos dados na interface do usuário quando ocorre a resposta do servidor, sem que haja o bloqueio das demais funções da interface.

Para tratar o problema do ComprasWeb, Maria deve usar o padrão de projeto:

- a) Observer;
- b) Chain of Responsibility;
- c) Flyweight;
- d) Data Access Object;
- e) Builder.

16.(FGV / DPE RS - 2023) Ana está implementando um sistema de comércio eletrônico e agora está definindo a operação de pagamento para a compra, podendo aceitar cartão de crédito, PIX ou boleto. Como a ação de pagamento envolve apenas a conciliação do valor final da compra, mas segundo diferentes meios de pagamento, Ana, com o objetivo de organizar seu código, permitiu a escolha do meio de pagamento por intermédio do padrão:

- a) State;
- b) Interpreter;
- c) Prototype;
- d) Memento;
- e) Strategy.

17.(FGV / ALEMA - 2023) Um padrão de projeto é uma solução geral para um problema que ocorre com frequência dentro de um determinado contexto no projeto de software. O padrão de projeto de software denominado Bridge é um padrão

- a) estrutural que desacopla uma abstração de sua implementação para que os dois possam variar independentemente.
- b) comportamental destinado a armazenar o estado interno de um objeto em um dado momento para que seja possível retorná-lo a este estado.
- c) de controle de objetos utilizado para garantir que determinada classe só tenha uma única instância e prover um ponto de acesso global a ela.

- d) de criação para instanciar famílias de objetos relacionados por meio de uma única interface e sem que a classe concreta seja especificada.
- e) de interação para prover uma maneira de acessar os elementos de um objeto agregado sequencialmente sem expor sua representação interna.

18.(FGV / TCE SP - 2023) O analista André é o arquiteto da aplicação web TCEDash. O componente Board, da TCEDash, apresenta ao usuário indicadores visuais e manipuláveis, baseados em informações contidas no banco de dados da aplicação. O Board instancia um novo objeto para cada indicador apresentado ao usuário. Sabendo que determinado subconjunto de informações se repete em todos os indicadores, André aplicou no componente Board o padrão de projeto estrutural do Gang Of Four (GoF), que otimiza a alocação de objetos na memória RAM, mediante o compartilhamento de atributos comuns entre os objetos.

André aplicou no componente Board o padrão do GoF:

- a) proxy;
- b) bridge;
- c) facade;
- d) flyweight;
- e) composite.

19.(FGV / TCE SP - 2023) Marcos está efetuando a manutenção de um sistema e notou que em diversos pontos são efetuadas chamadas HTTP para diferentes servidores, utilizando configurações específicas. Ele resolveu refatorar o código, criando uma classe que oferece métodos para cada elemento da configuração, os quais podem ser combinados para gerar um conector personalizado para cada servidor acessado.

Adotando essa abordagem, Marcos aumentou o reuso de processos intermediários na construção de objetos complexos, de acordo com o padrão:

- a) Singleton;
- b) Mediator;
- c) Builder;
- d) Observer;
- e) Chain of Responsibility.

20.(FGV / CAM DEP - 2023) Assinale a opção que indica o padrão de projeto GoF ('Gang of Four') que, com finalidade comportamental, sem violar o encapsulamento, captura e externaliza um estado interno de um objeto, de modo que o mesmo possa posteriormente ser restaurado para este estado.

- a) bridge.
- b) facade.
- c) flyweight.
- d) memento.
- e) visitor.

21.(FGV / TJ SE - 2023) A analista Maria está desenvolvendo a funcionalidade C de uma aplicação, utilizando os padrões de projeto Gang of Four (GoF). Existem vários algoritmos, conhecidos e disponíveis, capazes de implementar a funcionalidade C. A aplicação deve deixar o usuário escolher qual algoritmo usar. Sendo assim, Maria decidiu aplicar na funcionalidade C o padrão GoF comportamental que modela algoritmos similares como objetos independentes, porém, passíveis de troca em tempo de execução. Maria decidiu aplicar o padrão GoF:

- a) Visitor;
- b) Strategy;
- c) Mediator;
- d) Interpreter;
- e) Template Method.

22.(FGV / TJ SE - 2023) Pedro está implementando a aplicação web SergipeJusto. Pedro aplicou em SergipeJusto o padrão de projeto comportamental que soluciona o problema de armazenar e, posteriormente, restaurar o estado interno de um objeto sem violar o seu encapsulamento. Pedro aplicou em SergipeJusto o padrão de projeto comportamental:

- a) State;
- b) Iterator;
- c) Mediator;
- d) Memento;
- e) Prototype.

23.(FGV / TJ SE - 2023) André está trabalhando em um sistema com atualização de dados de forma assíncrona, onde um gráfico e uma planilha precisam repercutir de forma automática qualquer alteração efetuada na fonte de dados. Para implementar a solução, André deve usar o padrão de desenvolvimento:

- a) Chain of Responsibility;
- b) Adapter;
- c) Factory Method;
- d) Observer;
- e) Front Controller.

24.(FGV / TJ SE - 2023) Paulo implementou um sistema na plataforma Java EE, onde foi adotada a arquitetura MVC, colocando Servlets e JSPs na camada View, entidades JPA na Model e Session Beans na Controller. Como os Session Beans são os únicos componentes que instanciam gestores de persistência do JPA, Paulo segue o padrão de desenvolvimento:

- a) Decorator;
- b) Facade;
- c) Template Method;
- d) Abstract Factory;
- e) State.

25.(FGV / Sefaz AM - 2022) O padrão de projeto de software orientado a objetos, que pode ser utilizado para garantir que uma determinada classe tenha somente uma única instância, é

- a) adapter.
- b) flyweight.
- c) memento.
- d) singleton.
- e) visitor.

26.(FGV / SEMSA Manaus - 2022) No contexto de Design Patterns (Padrões de projetos), um dos padrões mais simples em Java envolve uma classe simples, responsável pela criação de um objeto com a garantia de que apenas um simples objeto é criado. Esse pattern é conhecido como

- a) Adapter.
- b) Builder.
- c) Factory.
- d) Singleton.
- e) State.

27.(FGV / TCE-AM - 2021) O Sistema de Acompanhamento de Mercados (SisMARKET) apoia o processo de controle de transações de compra e venda de ativos financeiros e fornece aos gestores visões distintas contendo gráficos que demonstram continuamente o andamento das transações. Sendo assim, o SisMARKET possui uma funcionalidade para notificar e atualizar as visões gráficas automaticamente sempre que o quantitativo de um ativo financeiro sofre alterações. No projeto de desenvolvimento do SisMARKET foram empregadas boas práticas, como o uso de padrões de projeto.

Sendo assim, no SisMARKET, o padrão de projeto utilizado na construção da funcionalidade que configura uma dependência um-para-muitos entre objetos, de modo que, quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente, foi o:

- a) Memento;
- b) Proxy;
- c) Observer;
- d) State;
- e) Strategy.

28.(FGV / FunSaúde CE - 2021) No contexto de padrões de projeto GoF, há um padrão que tem como definição garantir que uma classe tenha apenas uma instância de si mesma e que forneça um ponto global de acesso. Assinale-o.

- a) Builder.
- b) Factory.
- c) Proxy.
- d) Singleton.
- e) Strategy.

GABARITO

- | | | | | | |
|-----|---------|-----|---------|-----|---------|
| 1. | LETRA C | 11. | LETRA D | 21. | LETRA B |
| 2. | LETRA B | 12. | LETRA E | 22. | LETRA D |
| 3. | LETRA B | 13. | LETRA A | 23. | LETRA D |
| 4. | LETRA C | 14. | LETRA D | 24. | LETRA B |
| 5. | LETRA D | 15. | LETRA A | 25. | LETRA D |
| 6. | LETRA C | 16. | LETRA E | 26. | LETRA D |
| 7. | LETRA E | 17. | LETRA A | 27. | LETRA C |
| 8. | LETRA D | 18. | LETRA D | 28. | LETRA D |
| 9. | LETRA A | 19. | LETRA C | | |
| 10. | LETRA C | 20. | LETRA D | | |

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.